

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



TESIS DE GRADO

**PROTOTIPO DE MEDIDOR DE AGUA IOT PARA EL
CONTROL Y MONITOREO DEL CONSUMO DE AGUA
POTABLE EN HOGARES DE LA CIUDAD DE LA PAZ**

Tesis de Grado para obtener el Título de Licenciatura en Informática
Mención Ingeniería de Sistemas Informáticos

POR: JESUS JUAN CARLOS MARAZA VIGABRIEL
TUTOR: M.SC. GROVER ALEX RODRIGUEZ

LA PAZ – BOLIVIA

2021

HOJA DE CALIFICACIONES
UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA

Tesis de Grado:

**PROTOTIPO DE MEDIDOR DE AGUA IOT PARA EL CONTROL Y MONITOREO
DEL CONSUMO DE AGUA POTABLE EN HOGARES DE LA CIUDAD DE LA PAZ**

Presentado por: Jesus Juan Carlos Maraza Vigabriel

Para optar el grado Académico de Licenciado en Informática

Mención Ingeniería en Sistemas Informáticos

Nota Numeral: 70

Nota Literal: SETENTA

Ha sido acreedor al Grado Académico de Licenciado en Informática, mención: Ingeniería de
Sistemas Informáticos.

Director de la Carrera de Informática: Lic. Nancy Imelda Orihuela Sequeiros

Tutor: M.Sc. Grover Alex Rodriguez

Tribunal: Lic. Huanca Quisbert Carmen R.

Tribunal: Lic. Zeballos Daza Reynaldo J.



UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE CIENCIAS PURAS Y NATURALES
CARRERA DE INFORMÁTICA



LA CARRERA DE INFORMÁTICA DE LA FACULTAD DE CIENCIAS PURAS Y NATURALES PERTENECIENTE A LA UNIVERSIDAD MAYOR DE SAN ANDRÉS AUTORIZA EL USO DE LA INFORMACIÓN CONTENIDA EN ESTE DOCUMENTO SI LOS PROPÓSITOS SON ESTRÍCTAMENTE ACADÉMICOS.

LICENCIA DE USO

El usuario está autorizado a:

- a) Visualizar el documento mediante el uso de un ordenador o dispositivo móvil.
- b) Copiar, almacenar o imprimir si ha de ser de uso exclusivamente personal y privado.
- c) Copiar textualmente parte(s) de su contenido mencionando la fuente y/o haciendo la referencia correspondiente respetando normas de redacción e investigación.

El usuario no puede publicar, distribuir o realizar emisión o exhibición alguna de este material, sin la autorización correspondiente.

TODOS LOS DERECHOS RESERVADOS, EL USO NO AUTORIZADO DE LOS CONTENIDOS PUBLICADOS EN ESTE SITIO DERIVARÁ EN EL INICIO DE ACCIONES LEGALES CONTEMPLADOS EN LA LEY DE DERECHOS DE AUTOR.

DEDICATORIA

A mis dos Ángeles que me cuidan desde el Cielo, mi Señor Padre Juan Carlos Maraza Noa y mi Señora Madre Julieta Vigabriel Acarapi, con quienes estaré siempre agradecido por todas las oportunidades que me dieron en vida.

Dedico también la presente Tesis a mi Hermana Gabriela, quien siempre estuvo apoyándome en el crecimiento de mi Vida Universitaria y ahora Profesional.

AGRADECIMIENTOS

Primeramente, agradezco a Dios por darme la vida y permitirme tener éxito en ella.

Agradecer a mis Padres, quienes con sus enseñanzas me guiaron por el camino correcto.

A mi Señora Esposa Yasmina Salcedo, un ejemplo de mujer y coraje, quien siempre me apoyó con su consejo, paciencia y amor para poder salir adelante.

A mi Hermana Gabriela, que siempre estuvo a mi lado en buenos y malos momentos.

Agradecer a mis Tíos Nancy Vigabriel y Javier Sandi, quienes siempre estuvieron preocupados, por mi bienestar y sobre todo por mi desempeño en la Universidad.

A mis Padrinos Antonio Maraza y Lidia Gisbert, quienes siempre han estado a mi lado para recomendarme, apoyarme en todos mis emprendimientos y mi vida Universitaria.

A mi Hermano Gary Sandi por todo el cariño, apoyo, consejos y oportunidades que siempre me ha brindado.

A mi Tutor M.Sc. Grover Alex Rodríguez, por toda la paciencia, recomendaciones y enseñanza impartida en mi carrera universitaria.

Al Lic. Aldo Valdez, quien siempre fue un ejemplo a seguir y una figura a la que deseo aspirar en un futuro, agradecerle por siempre apoyar a los grupos de estudio de los que fui parte, por el apoyo incondicional y por las llamadas de atención que nos sirvieron de gran ayuda.

A mis mejores amigos, Alejandro Alcón y Cristhian Flores, quienes siempre estuvieron a mi lado compartiendo buenas y malas experiencias en nuestra vida Universitaria que los 3 estamos culminando.

Por último, agradecer a todos mis amigos con los que compartí diversas experiencias, a mis alumnos del Grupo de Estudio The Makers Informática y Arduino Open Source, quienes hoy en día son grandes profesionales.

carlosmaraza@gmail.com

RESUMEN

La tecnología IoT incorpora nuevas técnicas que nos permiten automatizar diferentes tareas que son total o parcialmente sistemáticas y/o repetitivas, así como realizar un control y/o monitoreo sobre dichas tareas, muchas veces ahorrando recursos.

En la presente Tesis se desarrolló un prototipo de Medidor de Agua Iot el cual permite al usuario final poder tener un control y monitoreo sobre su propio consumo de agua. El sistema IoT cuenta con una interfaz de Blynk.cc que opera en smartphones con SO Android e IOS y trabaja bajo la plataforma Open Source ESP32, similar a un Arduino común, pero con el potencial de tener la capacidad de conectarse a una red Wifi y tener también integrado BLE.

Para su desarrollo se trabajó bajo la plataforma PlatformIO y lenguaje basado en C y Processing. Se utilizaron los sensores de flujo de agua YF - S201 y de voltaje FZ0430.

Palabras clave: IoT, automatizar, control, monitoreo, agua, Wifi, sensores

Metodología: Metodología de Desarrollo en V o de 4 Nivel.

ABSTRACT

IoT technology incorporates new techniques that allow us to automate different tasks that are totally or partially consistent and/or repetitive, as well as to control and/or monitor these tasks, often saving money.

In this Thesis, a prototype of an Iot Water Meter was demonstrated, which allows the end user to have control and monitoring of their own water consumption. The IoT system has a Blynk.cc interface that operates on smartphones with Android and IOS OS and works under the Open Source ESP32 platform, similar to a common Arduino, but with the potential to have the ability to connect to a Wi-Fi network and also have integrated BLE.

For its development, we worked under the PlatformIO platform and language based on C and Processing. YF-S201 water flow and FZ0430 voltage sensors were used.

Keywords: IoT, automate, control, monitoring, water, Wi-Fi, sensors

Methodology: Development Methodology in V or Level 4.

La Paz, 15 de junio 2021

Señor.-

PhD. José María Tapia Baltazar

DIRECTOR

CARRERA DE INFORMÁTICA

FACULTAD DE CIENCIAS PURAS Y NATURALES

UNIVERSIDAD MAYOR DE SAN ANDRÉS

Presente.-

Ref.- Aval para la defensa de Tesis de Grado

De mi mayor consideración:

Mediante la presente, me dirijo a su autoridad en calidad de Tutor para informar que luego de haber realizado el seguimiento de la Tesis de Grado titulado: **“PROTOTIPO DE MEDIDOR DE AGUA IOT PARA EL CONTROL Y MONITOREO DEL CONSUMO DE AGUA POTABLE EN HOGARES DE LA CIUDAD DE LA PAZ”**, presentado por el Univ. **JESUS JUAN CARLOS MARAZA VIGABRIEL**, con C.I. 8321156 LP, para optar por el título de **LICENCIATURA EN INFORMÁTICA MENCIÓN INGENIERÍA EN SISTEMAS INFORMATICOS**.

En este sentido, presento mi conformidad y aval respectivo para la defensa publica de la Tesis de Grado de acuerdo con Reglamento vigente de la Universidad Mayor de San Andrés.

Sin otro particular, me despido con las consideraciones más distinguidas.

Atentamente.

M.Sc. Grover Alex Rodriguez Ramirez

TUTOR

INDICE GENERAL

DESCRIPCIÓN:	PÁG.
CAPITULO 1	
MARCO REFERENCIAL	1
1.1. INTRODUCCION	1
1.2 PROBLEMA	2
1.2.1 ANTECEDENTES	2
1.2.1.1 ANTECEDENTES DE TRABAJOS SIMILARES	2
1.2.1 PLANTEAMIENTO	3
1.2.3 FORMULACIÓN DEL PROBLEMA	3
1.3. OBJETIVOS	4
1.3.1. OBJETIVO GENERAL	4
1.3.2. OBJETIVOS ESPECÍFICOS	4
1.4 HIPÓTESIS	4
1.4.1 IDENTIFICACION DE VARIABLES	4
1.5. JUSTIFICACION	5
1.5.1. JUSTIFICACION SOCIAL	5
1.5.2. JUSTIFICACION ECONÓMICA	5
1.5.3. JUSTIFICACION TECNOLÓGICA	6
1.5.4. JUSTIFICACION CIENTÍFICA	6
1.6. ALCANCES Y LIMITES	7
1.6.1 ALCANCES	7
1.6.2 LIMITES	7
1.7. METODOLOGÍA DE INVESTIGACIÓN	8
1.7.1 MÉTODO CIENTÍFICO	8
CAPITULO 2	
MARCO TEÓRICO	10
2.1. AGUA POTABLE EN BOLIVIA	10
2.1.1. COSTO DEL AGUA POTABLE EN LA CIUDAD DE LA PAZ	11
2.1.2. CATEGORÍA DE PRECIOS SEGÚN EL USUARIO	11
2.2. INTERNET DE LAS COSAS IOT	13
2.2.1. CAMPOS DE APLICACIÓN DEL IOT	14
2.2.1.1. WEREABLES	14
2.2.1.2. SALUD	14
2.2.1.3. MONITOREO DE TRÁFICO	14
2.2.1.4. AGRICULTURA	14

2.2.1.5. AHORRO ENERGÉTICO	15
2.2.1.6. SUMINISTRO DE AGUA	15
2.2. HARDWARE OPEN SOURCE	15
2.2.1. PLATAFORMAS HARDWARE OPEN SOURCE	16
2.2.2.1.1. ARDUINO	17
2.2.2.1.2. NODEMCU ESP8266	19
2.2.2.1.3. NODEMCU ESP32	20
2.2.3 PLACA DE DESARROLLO NODEMCU ESP8266	22
2.2.3.1 NODEMCU	22
2.2.3.2 PLACA DE DESARROLLO	25
2.2.3.3 PRIMERA GENERACIÓN	26
2.2.3.4 SEGUNDA GENERACIÓN	27
2.2.3.5 TERCERA GENERACIÓN	28
2.2.3.6 ESP32	30
2.2.4. BLYNK IOT	33
2.2.5. SENSORES Y ACTUADORES	36
2.2.5.1. SENSORES	36
2.2.5.1.1 SENSOR DE FLUJO DE AGUA YF S201	37
2.2.5.1.2 SENSOR DE VOLTAJE	39
2.2.5.2. ACTUADORES Y PERIFÉRICOS	41
2.2.6. MEDIDOR DE AGUA CONVENCIONAL	42
2.2.6.1. TIPOS DE MEDIDORES DE AGUA POTABLE	43
2.2.6.1.1. MEDIDOR DE VELOCIDAD DE AGUA POTABLE	43
2.2.6.1.2. CONTADOR DE AGUA ELECTROMAGNÉTICO	44
2.2.6.1.3. MEDIDOR DE ULTRASONIDO.	45
2.2.7 MODELO EN V O DE 4 NIVELES	45
2.2.7.1 DISEÑO DEL MODELO EN V	46
2.2.7.2 FASES DE VERIFICACIÓN	47
2.2.7.3 FASE DE CODIFICACIÓN	48
2.2.7.4 FASE DE VALIDACIÓN	49
2.2.7.5 APLICACIÓN DEL MODELO EN V	50
2.2.7.6 MODELO EN V VENTAJAS Y DESVENTAJAS	50
CAPITULO 3 MARCO APLICATIVO	51
3.3.1 INTRODUCCION	51
3.3.2 DESCRIPCIÓN INFORMAL DEL SISTEMA	51
3.3.2.1 ENTRADAS	52

3.3.2.2 PROCESOS	52
3.3.2.3 SALIDAS	52
3.3.3 DESCRIPCIÓN FORMAL DEL SISTEMA	52
3.3.4 IMPLEMENTACIÓN	55
3.3.4.1 MODELO V	55
3.3.4.2 ANALISIS DE REQUERIMIENTOS	55
3.3.4.3 DISEÑO DE ARQUITECTURA	62
3.3.4.4 DISEÑO DE MODULO	68
3.3.4.5 CODIFICACIÓN	75
3.3.4.6 PRUEBAS	88
CAPITULO 4 PRUEBA DE HIPOTESIS	94
4.4.1 INTRODUCCIÓN	94
4.4.2 CALIDAD DE LAS ACTIVIDADES Y CASOS ENCONTRADOS	95
4.4.3 CASO 1	95
4.4.4 CASO 2	97
4.4.5 CASO 3	98
4.4.6 CASO 4	100
CAPITULO 5 CONCLUSIONES Y RECOMENDACIONES	104
5.5.1 CONCLUSIONES	104
5.5.1.1 ESTADO DE LOS OBJETIVOS	104
5.5.2 RECOMENDACIONES	105
BIBLIOGRAFÍA	107
ANEXO A. CODIGO ESP32	109
ANEXO B. REGLAMENTO TÉCNICO DE MEDIDORES DOMICILIARIOS DE AGUA POTABLE	122

INDICE DE TABLAS

Tabla 3.1: Funciones Interfaz Blynk	56
Tabla 3.2: Funciones de Servidor	57
Tabla 3.3: Funciones Hardware Electrónico	57
Tabla 3.4: Casos de uso Interfaz Blynk	58
Tabla 3.4: Casos de uso Servidor Blynk	59
Tabla 3.6: Casos de uso Hardware Electrónico	60
Tabla 3.7: Componentes del Circuito	63
Tabla 3.8: Software para la Interfaz Blynk	66
Tabla 4.1: Condiciones de Operación Medidores de Agua Potable	98
Tabla 4.2: Condición de Operación Medidores de Agua Potable comparativa con Medidor IOT	99

INDICE DE FIGURAS

Figura 2.1: Arduino UNO	17
Figura 2.2: Placas Arduino Clásicas	18
Figura 2.3: NodeMCU ESP8266	19
Figura 2.4: ESP32	21
Figura 2.5: Logo NodeMCU	23
Figura 2.6: Lua ESP8266	24
Figura 2.7: NodeMCU ESP8266 v0.9	27
Figura 2.8: NodeMCU ESP8266 Amica v1.0/v2	28
Figura 2.9: NodeMCU ESP8266 LOLIN v1.0/v3	29
Figura 2.10: Estructura SOC ESP32	30
Figura 2.11: PinOut ESP32	31
Figura 2.12: Logo Blynk	33
Figura 2.13: Vista de la App Blynk	34
Figura 2.14: Sensor de Flujo YF-S201	37
Figura 2.15: Sensor de Flujo YF-S201 Internamente	38

Figura 2.16: Sensor de Voltaje FZ0430	39
Figura 2.17: Medidor de velocidad de agua potable	43
Figura 2.18: Contador de Agua Electromagnético	44
Figura 2.19: Medidor de Ultrasonido de agua potable	45
Figura 2.20: Fases del Modelo en V	46
Figura 3.1: Entradas y salidas del sistema IoT	51
Figura 3.2: Entradas y salidas de los subsistemas	53
Figura 3.3: Diseño Global	54
Figura 3.4: Casos de uso para la interacción del Usuario	59
Figura 3.5: Casos de uso para el Servidor	60
Figura 3.6: Casos de uso para el Circuito Electrónico	61
Figura 3.7: Diagrama de Secuencias	61
Figura 3.8: Arquitectura funcional del prototipo	62
Figura 3.9: Diseño del circuito	65
Figura 3.10: Diseño Arquitectura Blynk	67
Figura 3.11: Algoritmo principal del Microcontrolador	68
Figura 3.12: Setup	70
Figura 3.13: Loop	71
Figura 3.14: datos_lcd	72
Figura 3.15: volt_med	73
Figura 3.16: grabarDatos	73
Figura 3.17: subirDatos	74
Figura 3.18: Implementación circuito	75
Figura 3.19: Inclusión librerías	76
Figura 3.20: Inicialización de Variables Globales	76
Figura 3.21: Inicialización de Variables Globales	77
Figura 3.22: Función lcd_volt	78
Figura 3.23: Función lcd_volt	79
Figura 3.24: Función lcd_volt	79
Figura 3.25: Función lcd_volt	80
Figura 3.26: Función lcd_volt	80
Figura 3.27: Función lcd_volt	81
Figura 3.28: Función fmap y volt_med	82

Figura 3.29: Función datos_lcd	83
Figura 3.30: Función setup	84
Figura 3.31: grabarDatos	85
Figura 3.32: subirDatos	87
Figura 3.33: Función loop	87
Figura 3.34: Interfaz Blynk inicial	88
Figura 3.35: Interfaz Blynk Conectado	89
Figura 3.36: Prototipo	91
Figura 3.37: Prototipo	92
Figura 3.38: Prototipo	92
Figura 3.39: Prototipo Instalado y en funcionamiento	93
Figura 4.1: Formula para el cálculo de Litros consumidos	95
Figura 4.2: Resultado de la Calibración 1 litro	96
Figura 4.3: Resultado de la Calibración 1 litro	97
Figura 4.4: Fórmula para el Cálculo de Costo total de Agua consumida	97
Figura 4.5: Datos en estado conectado	100
Figura 4.6: Datos en estado desconectado	100
Figura 4.7: Blynk detecta la desconexión del dispositivo	101
Figura 4.8: Hardware prosigue con el registro de datos	101
Figura 4.9: Despliegue del estado conectado al restablecer la conexión a Internet	102
Figura 4.10: Blynk detecta la conexión con el dispositivo y actualiza sus datos	102
Figura 4.11: Datos almacenado en la tarjeta SD	103

CAPITULO I

MARCO REFERENCIAL

1. INTRODUCCIÓN.

Distintas empresas y/o cooperativas de aguas, que brindan el servicio de agua potable, instalan diversos tipos de medidores de agua los cuales son controlados por su personal de manera física, contabilizando el consumo mensual y generando un costo por dicho consumo.

El ciudadano de a pie llegar a ser el usuario final quien recibe este servicio de agua potable, quien muchas veces no puede controlar su propio consumo, ya sea diario, semanal o mensual, tampoco tiene una forma para poder calcular su consumo total en bolivianos, muchas veces se cometen errores el momento en que la empresa de agua realiza la lectura del consumo, llegando costos erróneos, el usuario final no tiene una constancia precisa de su consumo para poder validar su reclamo, generando descontento en el usuario.

Para poder brindar una solución tecnológica que nos permita automatizar y optimizar la obtención de datos del consumo de agua potable, se plantea implementar un Prototipo de Medidor de Agua IoT monitoreado desde una Aplicación Móvil, que nos permita tener un historial de consumo, cálculo de importe en bolivianos, así mismo hacer una tratamiento seguro y eficiente de los datos mencionados.

2. PROBLEMA

2.1 ANTECEDENTES.

En Bolivia el consumo de agua va acrecentando año con año, esto por falta de políticas de concientización, crecimiento poblacional y el poco interés del ciudadano por conocer cuál es el consumo de agua que realiza día a día.

Es su mayoría el ciudadano de a pie desconoce su consumo diario o semanal, conformándose con saber su consumo mediante una factura que entrega la empresa encargada de ofrecer este servicio, existiendo muchas veces reclamos por el incremento de la factura, que deriva del consumo que el usuario considera no haber realizado.

2.1.1 ANTECEDENTES DE TRABAJOS SIMILARES

Lain Holding, empresa Costa Riqueña, ofrece diferentes soluciones tecnológicas, entre ellas se puede mencionar a IoT SigFox, un medidor que permite tener un control sobre el flujo de agua que se consume y generar a partir de ellos diversas tareas, siendo este un Medidor más orientado para empresas de servicios de agua que trabajan con miles de clientes, a pesar de contar con características de un dispositivo IoT, no deja de ser un medidor que aún mantiene un formato analógico para mostrar datos de manera física, así mismo su costo regular en el mercado queda lejos del alcance de un usuario común.

EMI, empresa de Tecnología China, empresa enfocada en soluciones tecnológicas, cuenta con su producto denominado Nb-Iot, el cual permite obtener datos de consumo y proporcionar dicho dato a la empresa que lo esté administrando, al igual que el medidor

antes mencionado de Lain Holding está más enfocado a la empresa y mantiene un enfoque analógico para mostrar físicamente la lectura.

Ambos medidores mencionados están más enforcados a la empresa, llegando a tener costos elevados por unidad, mantenimiento y soporte.

2.2 PLANTEAMIENTO.

Poder tener un control del consumo de agua que se tiene por día, semana o mes, es una tarea complicada para el usuario común, no tener conocimiento del costo por metro cúbico, imposibilita poder realizar un cálculo con precisión, así mismo el hecho de no tener la facilidad para poder leer de manera correcta los datos que se ven expresados de manera física en el medidor de agua, dificulta aún más esta tarea.

En un ámbito más global, como dueño de un edificio, oficinas, empresas con alto consumo de agua potable, es conveniente tener un historial del consumo, así mismo tener gráficas e importe a pagar, tener un registro de qué temporadas, días, semanas, meses se tiene mayor consumo de dicho elemento.

2.3 FORMULACIÓN DEL PROBLEMA

¿Cómo puedo obtener datos sobre el consumo de agua potable en hogares de la ciudad de La Paz, de tal manera que pueda controlar y monitorizar dicho consumo?

3 OBJETIVOS

3.1 OBJETIVO GENERAL

Desarrollar e implementar un Prototipo de Medidor de Agua IoT, para poder obtener los datos sobre el consumo de agua potable en hogares de la ciudad de La Paz para su control y monitoreo.

3.2 OBJETIVOS ESPECÍFICOS

- Diseñar el Prototipo de Medidor de Agua IoT, para la obtención de datos.
- Implementar el Prototipo de Medidor de Agua IoT.
- Generar Graficas de consumo e historiales en la App de Monitoreo.
- El Prototipo de Medidor de Agua IoT deberá cumplir con las especificaciones requeridas por las normas nacionales.
- Evaluar el funcionamiento del Prototipo de Medidor de Agua IoT, realizando pruebas en tiempo real.

4 HIPÓTESIS

El uso de un Prototipo de Medidor de Agua IoT, nos permitirá obtener datos del consumo de agua potable en hogares de la ciudad de La Paz para su control y monitoreo.

4.1 IDENTIFICACIÓN DE VARIABLES

VARIABLES INDEPENDIENTES

Prototipo de Medidor de Agua IoT.

VARIABLES DEPENDIENTES

Obtención de datos del consumo de Agua

5 JUSTIFICACIONES

5.1 JUSTIFICACIÓN SOCIAL

En los últimos años se ha concientizado a cerca del consumo desmedido de agua, siendo este hoy en día un elemento vital que escasea en diferentes partes del mundo, donde se ven en la necesidad de racionar su consumo o en su caso repartir agua embotellada, esta crisis ira incrementando con los años si no tomamos conciencia y no se toman medidas a tiempo.

Con nuestro Prototipo de Medidor de Agua IoT, se busca crear conciencia al usuario sobre su consumo de agua, mostrando en tiempo real su propio consumo en litros y a largo plazo su consumo semanal y/o mensual.

5.2 JUSTIFICACIÓN ECONÓMICA

El consumo de agua potable ha ido incrementando en los últimos años, esto por su uso desmedido y falta de control, derivando en altos costos económicos al momento de cancelar el servicio de este elemento vital.

Con nuestro Prototipo de Medidor de Agua IoT, se busca poder ofrecer al usuario una forma más eficiente de poder controlar su consumo diario, semanal o mensual y a su vez poder calcular un costo aproximado del consumo que se tiene actualmente, con el objetivo de que a futuro se puedan implementar buenas prácticas de consumo y tener menores costos de pago por el servicio.

El prototipo está diseñado para tener una vida útil de aproximadamente 10 años sin interrupción, todos los accesorios, sensores y actuadores de nuestro prototipo serán de origen Hardware Open Source, reduciendo costos de producción y por tal ofreciendo un costo adecuado y sostenible del dispositivo, tomando en cuenta también que todos sus componentes pueden ser fácilmente reemplazados, llegando a ofrecer un gran beneficio a corto y largo plazo para el usuario.

5.3 JUSTIFICACIÓN TECNOLÓGICA

En la actualidad gran mayoría de los hogares dentro de la ciudad cuentan con una red Wifi instalada en su domicilio, las redes 4G se han convertido en una estándar de comunicaciones, permitiéndonos un acceso rápido y eficiente a la Internet, gran parte de usuarios cuenta con un dispositivo móvil con un sistema operativo Android, por su versatilidad y bajo costo.

Con nuestro Prototipo de Medidor de Agua IoT se aprovecha la tecnología existente y la facilidad que tiene el usuario para conectarse a la Internet y tener acceso a móviles inteligentes, con el fin de poder Monitorear el consumo de agua y mejorar la administración de dicho consumo.

5.4 JUSTIFICACIÓN CIENTÍFICA

Actualmente existen diversos estudios sobre el impacto que está teniendo el Internet de las Cosas sobre el mundo moderno, facilitando trabajos operativos que antes necesitaban un control físico por parte de operarios externos, control sobre sistemas de fabricación

controlada por robots, e incluso en la agricultura y ganadería, como el control de invernaderos y ubicación de ganado en específico.

Con nuestro Prototipo de Medidor de Agua IoT, podremos obtener datos claros sobre el consumo de agua potable, datos de los cuales podremos obtener información sobre la tendencia de consumo en ciertas épocas del año, aportando a los diversos estudios que se realizan sobre el consumo excesivo de agua.

6. ALCANCES Y LIMITES

6.1 ALCANCES

- El prototipo toma datos constantemente del flujo de agua.
- El prototipo calcula el costo del consumo de agua, según los parámetros de costos existentes.
- El prototipo puede usarse en el caudal principal de agua, como de manera particular en otros dispositivos como lavadoras, duchas, lavamanos, etc.
- El prototipo de Medidor trabaja con una Aplicación móvil para mostrar los datos capturados.
- Se muestran gráficas y datos en tiempo real mediante la Aplicación móvil.
- El prototipo funciona de manera independiente a la Aplicación Móvil.
- El prototipo trabaja en instalaciones comunes de agua, como una vivienda o departamento.

6.2 LIMITES

- El prototipo en su primera versión necesita de una conexión a Internet mediante Wifi estable y fija, siendo el único medio para transmitir los datos obtenidos.
- El prototipo trabaja necesariamente con una alimentación eléctrica o batería de manera constante.

- El prototipo realizará un cálculo del costo de consumo, pero no tomará en cuenta demás variables que influyen en los costos extras que determina cada Cooperativa o Empresa de Aguas, como alcantarillado o reinstalación de servicios.
- El prototipo en su primera versión podrá trabajar en instalaciones comunes de agua, como una vivienda o departamento, pero estos deberán contar con una conexión a internet mediante una red Wifi necesariamente.

7. METODOLOGÍA

7.1 MÉTODO CIENTÍFICO

La investigación científica se encarga de producir conocimiento. El conocimiento científico se caracteriza por ser:

- Sistemático
- Ordenado
- Metódico
- Racional/Reflexivo
- Crítico/Subversivo

Sistémico significa que no se eliminan pasos de manera arbitraria, si no que se deben seguir de manera rigurosa.

Metódico implica que se debe elegir un camino ya sea, una encuesta, una entrevista o una observación.

Racional/Reflexivo implica una reflexión por parte del investigador y tiene que ver con una ruptura con el sentido común. Hay que alejarse de la realidad construida por uno mismo, alejarse de las nociones, del saber inmediato. Esto permite llegar a la objetividad.

Crítico se refiere a que intenta producir conocimiento, aunque esto pueda jugar en contra.

En la presente investigación se implementará la metodología de investigación científica y deductiva planteada por Mario Bunge que consta de las siguientes etapas:

- **Observación:** Se hará un análisis del funcionamiento de los prototipos IoT actuales, para poder implementar un nuevo prototipo IoT.
- **Planteamiento de hipótesis:** El uso de un Prototipo de Medidor de Agua IoT, nos permitirá obtener datos del consumo de agua potable en hogares de la ciudad de La Paz
- **Diseño de la aplicación:** En esta etapa se hará uso de una metodología de software para realizar el sistema domótico.
- **Casos de prueba:** Durante el desarrollo del sistema se realizarán pruebas de caja blanca, pruebas unitarias, pruebas de sistema. Al finalizar la implementación del sistema se realizarán pruebas de caja negra, para verificar si los datos de salida coinciden con los requisitos del usuario.
- **Conclusiones:** En esta etapa se analizarán los resultados obtenidos para dar pauta a nuevos temas de investigación.

CAPITULO II

MARCO TEORICO

1. AGUA POTABLE EN BOLIVIA

La distribución, consumo y calidad de Agua Potable en Bolivia ha ido acrecentando en las últimas décadas, estudios realizados por la Autoridad de Fiscalización y Control Social de Agua Potable y Saneamiento (AAPS) revelan que el líquido cuenta con un nivel de conformidad del 99% para el consumo humano. El 70% de la población boliviana consume agua potable de calidad, según la Autoridad de Fiscalización y Control Social de Agua Potable y Saneamiento (AAPS, 2013).

(Director de la AAPS, Edson Solares) “Bolivia cuenta con agua potable de calidad. Tomando en cuenta que el estándar aceptable es de 95%, nosotros sobrepasamos ese porcentaje con 99%”.

Destacó los avances en la gestión de Gobierno y auguró que la cobertura de abastecimiento para 2025 se ampliará del 70% al 100% de la población. Entretanto, dijo, “estamos velando para que la población tenga calidad, cobertura y costos justos”.

La regulación y control de calidad del líquido elemento es realizada sobre la base de la norma boliviana 512, que establece seis parámetros esenciales: potencial de hidrógeno (PH), conductividad, turbiedad, cloro de ciudad y coliformes.

A escala nacional, la AAPS recibe reportes de 1.091 Empresa Pública Social de Agua y Saneamiento (Epsas), las que son regularizadas por el Estado. “Antes sólo se regulaban 16

Epsas”, apuntó Solares antes de dar a conocer que “lo que hacemos es aplicar un protocolo de seguimiento a la calidad, a la cobertura y a los costos”. (AAPS, 2018)

1.1 COSTO DEL AGUA POTABLE EN LA CIUDAD DE LA PAZ

En cuanto se refiere a la estructura de precios y tarifas, EPSAS tiene la intención de mantener dichas estructuras: especialmente las referidas a las categorías de usuarios y los rangos de consumo en cada categoría. A continuación, se detalla las características de la estructura de EPSAS. (EPSAS, 2018)

1.2 CATEGORÍA DE PRECIOS SEGÚN EL USUARIO

EPSAS aplica una categorización de usuarios conforme a la definición establecida por la AAPS que se describe a continuación:

- **Categoría Tarifa Doméstica Solidaria:** Pertenecen a esta categoría aquellos Usuarios de bajos recursos, cuyo predio se usa para vivienda y el agua para salud. Cuyo consumo no rebase a los 15 m³/mes.
- **Categoría Doméstica:** Pertenecen a esta categoría los Usuarios domésticos cuyo predio se usa para vivienda y el agua para salud, cuyo consumo sea superior a los 15m³.
- **Categoría Estatal:** Pertenecen a esta categoría los Usuarios en cuyo predio se desarrollan tareas de la administración pública, municipal, educación fiscal, salud pública, policía, militares, parques, plazas y otras actividades que no sean con fines de lucro.

- **Categoría Comercial:** Pertenecen a esta categoría los Usuarios cuyo predio se usa para negocio y el agua para salud. Cuando existan pequeños comercios en las propias viviendas y que en forma evidente no utilicen agua potable para la comercialización de sus productos y servicios, como es el caso de tiendas de abarrotes, dichos predios deberán considerarse, para efectos de la definición tarifaria, como categoría doméstica.
- **Categoría Industrial:** Pertenecen a esta categoría los Usuarios cuyo predio se usa para negocio y el agua para negocio. Adicionalmente pertenecen a esta categoría todas las personas jurídicas inscritas en la Cámara Nacional de Industrias
- **Categoría Solidaria Social:** Pertenecen a esta categoría los Usuarios cuyo predio es utilizado para asilos de ancianos, albergues de niños (orfanatos) y centros de rehabilitación de discapacitados, a cargo de instituciones públicas, religiosas y privadas que cumplan una función de asistencia social de la población altamente vulnerable y que no tengan fines de lucro.

La tarifa asignada para esta categoría es de Bs. 1,78 conforme a las condiciones establecidas por la AAPS.
- **Categoría Seguridad Ciudadana:** Pertenecen a esta categoría los usuarios que comprenda a los módulos policiales, estaciones policiales integrales, módulos fronterizos y puestos de control. La tarifa asignada para esta categoría es de Bs. 1,78 conforme a las condiciones establecidas por la AAPS en su RAR AAPS No. 69/2012. (EPSAS, 2018).

2. INTERNET DE LAS COSAS IOT

En la última década se ha sentido la necesidad de conectar objetos comunes que son de nuestro diario vivir hacia la Internet, esto en busca a soluciones optimas y asequibles.

El tipo de objetos que se busca conectar pueden ser cualquier tipo de elemento, como un foco común, plantas, elementos portátiles y dispositivos inteligentes.

Un sistema de IoT funciona enviando, recibiendo y analizando datos de forma permanente, cumpliendo un ciclo de retroalimentación, manteniendo como clave la operación remota (Rojas, 2017).

Las aplicaciones de esta tecnología son múltiples, porque es ajustable a casi cualquier tecnología que sea capaz de aportar información relevante sobre su propio funcionamiento, sobre el desempeño de una actividad e incluso sobre las condiciones ambientales que necesitemos monitorear y controlar a distancia (Fractal, 2020).

A diferencia de algunas tecnologías, el IoT aún no ha encontrado un mercado de consumo potencialmente estable, quizá esto se debe a sus recientes inicios o que grandes empresas del sector aún no han visto la oportunidad para empezar a explotar dicha tecnología.

2.1 CAMPOS DE APLICACIÓN DEL IOT

El Internet de las cosas no tiene un número de campos de aplicación determinado, pero se pueden detallar los campos donde más ha resaltado su aplicación.

2.1.1 WEREABLES

Se tratan de dispositivos pequeños y autosostenibles, equipados con sensores y el hardware necesario para realizar mediciones y lecturas con el objetivo de poder recopilar y organizar información sobre los usuarios en un software especializado.

2.1.2 SALUD

Mediante el uso de sensores conectados a pacientes, se puede tener un seguimiento de sus condiciones, fuera y adentro del hospital en tiempo real, mediante este seguimiento se puede obtener métricas y alertas automáticas sobre signos vitales.

2.1.3 MONITOREO DE TRÁFICO

El IoT es de gran utilidad en la gestión de tránsito vehicular, de grandes y medianas ciudades, contribuyendo al concepto de las ciudades inteligentes.

2.1.4 AGRICULTURA

La implementación de sensores IoT nos puede ofrecer una cantidad importante de datos sobre el estado y las etapas de los suelos.

Existen diversos sensores especializados para trabajar con los suelo como sensores de humedad del suelo, nivel de acidez, presencia de ciertos nutrientes, así como también la temperatura y otras características.

2.1.5 AHORRO ENERGÉTICO

El uso de medidores de energía inteligentes o medidores equipados con los sensores necesarios nos permitirán realizar un mejor seguimiento y control de la red eléctrica , estableciendo una comunicación entre la compañía que presta el servicio y el usuario final.

2.1.6 SUMINISTRO DE AGUA

Un sensor, incorporado o ajustado de manera externa a un medidor de agua, conectado a internet y acompañado del software necesario, ayuda a recopilar, procesar y analizar datos, que permite comprender el comportamiento de los consumidores finales,, detectar fallas, reportar resultados entre otros beneficios.

Ofrece también a los consumidores finales la posibilidad de hacer seguimiento de su propia información de consumo, a través de una página web y en tiempo real.

2.2 HARDWARE OPEN SOURCE

Se considera Hardware de Código abierto, a todo dispositivo de hardware cuyo datasheet, especificaciones y diagramas son de acceso público, de manera general obtenidos de manera gratuita. La filosofía de Software Open Source también se aplica a la del hardware libre formando parte de la cultura libre.

El Hardware Open source tuvo sus inicios en 2001 con el Challenge to Silicon Valley publicado por Kofi Annan. Debido a que la naturaleza del hardware es diferente a la del software y que el concepto de hardware libre es relativamente nuevo, no teniéndose hasta la fecha una definición exacta del término.

El término más formal definido hasta el momento ha sido planteado por la OSHA (Open Source Hardware Association) quien nos da la siguiente Declaración de Principios en su versión 1.0:

(Cuartielles, 2019): “Hardware de Fuentes Abiertas (OSHW en inglés) es aquel hardware cuyo diseño se hace disponible públicamente para que cualquier persona lo pueda estudiar, modificar, distribuir, materializar y vender, tanto el original como otros objetos basados en ese diseño. Las fuentes del hardware (entendidas como los ficheros fuente) habrán de estar disponibles en un formato apropiado para poder realizar modificaciones sobre ellas. Idealmente, el hardware de fuentes abiertas utiliza componentes y materiales de alta disponibilidad, procesos estandarizados, infraestructuras abiertas, contenidos sin restricciones, y herramientas de fuentes abiertas de cara a maximizar la habilidad de los individuos para materializar y usar el hardware. El hardware de fuentes abiertas da libertad de controlar la tecnología y al mismo tiempo compartir conocimientos y estimular la comercialización por medio del intercambio abierto de diseños.”

2.2.1 PLATAFORMAS HARDWARE OPEN SOURCE.

Las plataformas Hardware Open Source han ido evolucionando con el tiempo y diferentes dispositivos han ido marcando hitos en la evolución de los que es el Hardware

Open Source, mencionaremos datos generales de las más importantes y de mayor relevancia para la presente Tesis.

2.2.1.1 ARDUINO

Arduino.cc (2020) nos dice que Arduino es una Plataforma de Hardware Libre, basado en un microcontrolador Atmel AVR, en particular de la Serie ATmega, cuenta con varios puertos digitales y analógicos de entrada y salida, a través de los cuales podemos leer información del medio que nos rodea, utilizando una diversidad de sensores e interactuar con el mismo a través de actuadores.

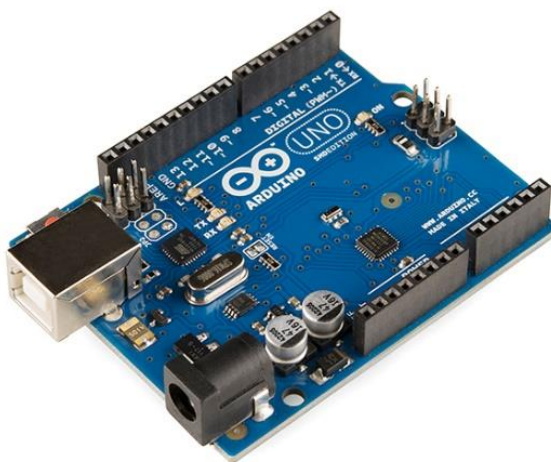


Figura 2.1: Arduino UNO
Fuente: (ElectroTools.com, abril 2016)

Es la Plataformas Open Hardware más difundida a nivel mundial, por ser libre, sencilla y por su bajo costo, esta plataforma tiene diferentes versiones para que podamos elegir la que mejor se ajuste a nuestro proyecto, pudiendo variar entre versiones el tamaño, memoria, velocidad del procesador, cantidad de puertos, etc.

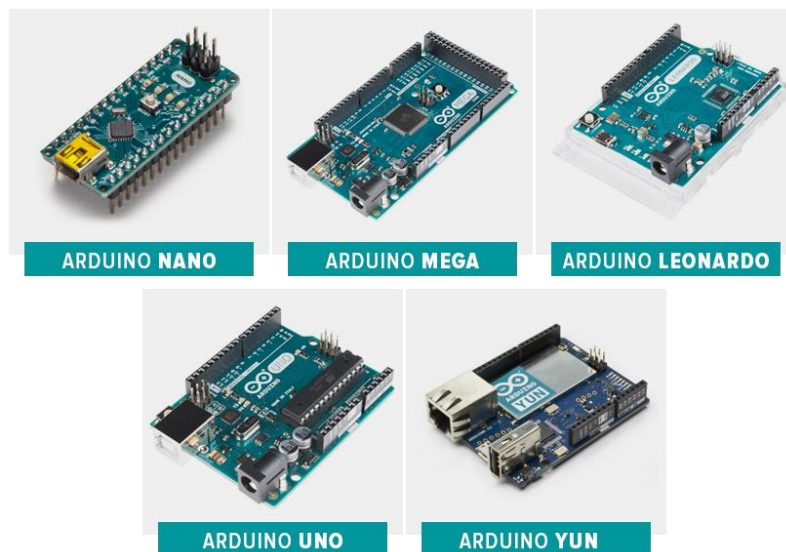


Figura 2.2: Placas Arduino Clásicas

Fuente: (Arduino.cl)

A algunas de estas placas se les puede acoplar shields de expansión las cuales aumentan la versatilidad de un Arduino común, añadiéndole características como conexión a Ethernet, WiFi, GPRS o matrices de LED's.

La plataforma Arduino se programa con el Lenguaje Arduino que básicamente es un híbrido entre Processing y C++. Actualmente existen herramientas que nos permiten programar una placa Arduino de manera visual e intuitiva, es por ellos que también es una de las plataformas más explotadas para la incorporación de la Robótica en Instituciones Educativas de niveles iniciales.

Hoy en día, a pesar de su gran éxito, la Plataforma Arduino a quedado estancada y solo se la considera para prototipos funcionales, pero no está considerada como una solución a

nivel empresarial y/o industrial, los creadores de Arduino van innovando año con año para romper esta brecha y seguir brindando soluciones con su Plataforma.

2.2.1.2 NODEMCU ESP8266

NodeMCU es una placa de desarrollo basada en el ESP12E el cuál es el módulo más popular que integra el SoC ESP8266, expone funcionalidades y capacidad de este.

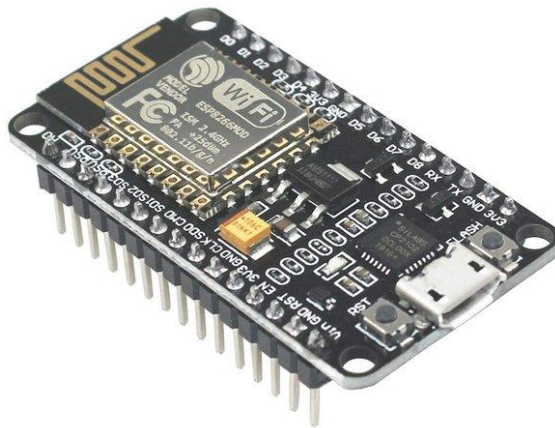


Figura 2.3: NodeMcu ESP8266

Fuente: (Equipo NodeMCU)

Como es de esperarse esta plataforma es de Código abierto además de ello es interactivo, programable, de bajo costo, simple, inteligente y lo que lo hace diferente de otras plataformas, es que ya está habilitado para Wifi, simplificando la tarea de poder integrarse al mundo del IoT. (Equipo NodeMCU, 2018).

Cuenta con una API avanzada para E/S de hardware, que puede reducir drásticamente el trabajo redundante para configurar y manipular hardware, tiene disponible una API impulsada por eventos para aplicaciones de red, que facilita a los desarrolladores escribir código que se ejecuta en la MCU, acelerando enormemente el proceso de desarrollo de su aplicación IoT.

Al igual que la ya mencionada Arduino, la NodeMCU es un kit de desarrollo integrado y fácil de manipular para la creación de prototipos IoT a muy bajo costo. NodeMCU Esp8266 hoy en día se considera como la herramienta más versátil y de fácil desarrollo para soluciones IoT, considerándose como una herramienta que puede brindar soluciones a nivel empresarial. (Equipo NodeMCU, 2018).

2.2.1.3 NODEMCU ESP32

NodeMCU 32 es una herramienta muy potente para el prototipado rápido de proyectos con IoT. La plataforma ESP32 es la evolución del ESP8266 mejorando sus capacidades de comunicación y procesamiento computacional, a nivel de compatibilidad permite utilizar diversos protocolos de comunicación inalámbrica como: Wifi, Bluetooth y BLE (Node MCU Team, 2018).

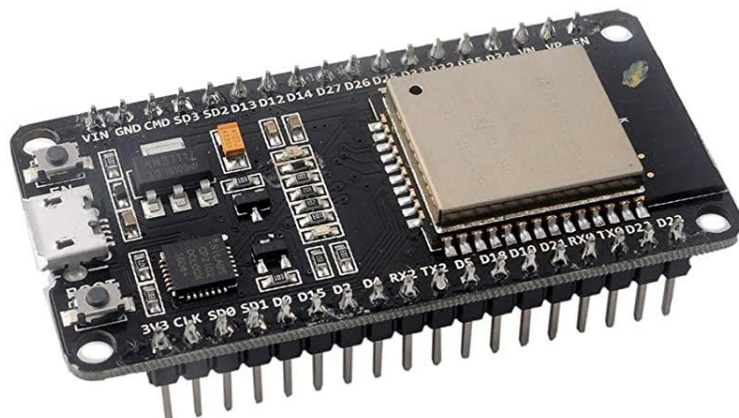


Figura 2.4: ESP32
Fuente: (Espressif Systems)

El ESP32 es capaz de funcionar de forma fiable en entornos industriales, con una temperatura de funcionamiento de -40°C a $+125^{\circ}\text{C}$. alimentado por circuitos de calibración avanzados, ESP32 puede eliminar dinámicamente las imperfecciones del circuito externo y adaptarse a los cambios en las condiciones externas.(Luis Lamas, 2018)

Diseñado para dispositivos móviles, dispositivos electrónicos portátiles y aplicaciones de IoT, ESP32 logra un consumo de energía ultra bajo con una combinación de varios tipos de software patentado. El ESP32 también incluye características de vanguardia, como la sincronización de reloj de grano fino, varios modos de potencia y escalado dinámico de potencia.

ESP32 está altamente integrado con interruptores de antena incorporados, balun de RF, amplificador de potencia, amplificador de recepción de bajo ruido, filtros y módulos de administración de energía. ESP32 agrega funcionalidad y versatilidad invaluable a sus aplicaciones con requisitos mínimos de placa de circuito impreso (PCB).

ESP32 puede funcionar como un sistema independiente completo o como un dispositivo esclavo de una MCU host, lo que reduce la sobrecarga de la pila de comunicación en el procesador de la aplicación principal. ESP32 puede interactuar con otros sistemas para proporcionar funcionalidad Wi-Fi y Bluetooth a través de sus interfaces SPI / SDIO o I2C / UART. (Espressif Systems, 2020).

2.3 PLACA DE DESARROLLO NODEMCU

NodeMCU es una placa de desarrollo en el ESP12E, es probablemente el módulo más popular que integra el SoC ESP8266. Sin embargo, pese a la popularidad de las placas NodeMCU, también existe mucha confusión respecto a la terminología (Lua, Lolin, versiones) y, en ocasiones, se mezclan o incluso se usan como sinónimos.

Se debe tener en cuenta que, a diferencia de Arduino donde hay una compañía que pone un cierto orden en los diseños y modelos disponibles, las placas de desarrollo basadas en ESP8266 han evolucionado un poco a saltos y de la mano de distintos fabricantes.

La cosa se complica más aún si leemos las descripciones de algunos vendedores, que mezclan conceptos y contribuyen a generar aún más confusión. (Luis Lamas, 2018).

2.3.1 NODEMCU

NodeMCU es un nombre que recoge tanto un firmware Open Source y como a una placa desarrollo basados en el ESP8266 o su siguiente generación ESP32.

En principio el nombre NodeMCU se refería principalmente al firmware. Actualmente esto se ha invertido y cuando hablamos de NodeMCU normalmente nos referimos a la placa de desarrollo.



Figura 2.5: Logo NodeMCU

Fuente: (Equipo NodeMCU)

El firmware NodeMCU fue creado poco después de aparecer el ESP8266, el 30 de diciembre de 2013. Unos meses después, en octubre de 2014 se publicó la primera versión del firmware NodeMCU en Github. Dos meses más tarde se publicaba la primera placa de desarrollo NodeMCU, denominada devkit v0.9, siendo también Open Hardware.

Recordemos que en esos primeros momentos del ESP8266 apenas había información y la poca que había era confusa. La mayoría del interés de la comunidad se limitaba al ESP01, que se consideraba más un módulo Wifi barato para procesadores como Arduino que una placa de desarrollo independiente.

El firmware NodeMCU podía grabarse en un ESP8266, tras lo cual podíamos programarlo con el lenguaje script Lua. La programación en Lua permitía la conexión y programación del ESP8266 de una forma mucho más sencilla que las herramientas oficiales proporcionadas por Espressif.



Figura 2.6: Lua ESP8266
Fuente: (Espressif.com)

Con el paso del tiempo y la aparición de otras alternativas para programar ESP8266, como (especialmente) con C++ con el entorno del Arduino y otras como MicroPython, el interés en Lua ha disminuido considerablemente.

A pesar de que la programación en Lua tenía aspectos interesantes, no es un lenguaje tan extendido como C++ y Python. Además, nunca consiguieron hacerlo totalmente estable en el ESP8266. Por otro lado, al ser un lenguaje interpretado (en lugar de compilado) el rendimiento y aprovechamiento de los recursos es inferior.

En 2015 el equipo de desarrollo original dejó de mantener el firmware de NodeMCU. Aunque sigue siendo mantenido por una comunidad de desarrolladores el interés en el firmware ha caído casi por completo. Por este motivo, actualmente nos referimos con NodeMCU más a la placa de desarrollo que al firmware.

No obstante, aunque actualmente el firmware haya caído un poco en el olvido, no hay que olvidar la contribución que el proyecto NodeMCU, tanto firmware como placa de desarrollo, han supuesto para la proliferación e implantación del ESP8266.

2.3.2 PLACA DE DESARROLLO NODEMCU

Básicamente, la placa de desarrollo NodeMCU está basada en el ESP12E y expone las funcionalidades y capacidad del mismo. Pero, además, añade las siguientes ventajas propias de placas de desarrollo:

- Puerto micro USB y conversor Serie-USB
- Programación sencilla a través del Micro-USB
- Alimentación a través del USB
- Terminales (pines) para facilitar la conexión
- LED y botón de reset integrados

No vamos a entrar en los detalles de hardware del ESP12E porque los vimos de forma intensiva en su propia entrada y son comunes a todas las placas de desarrollo basadas en el ESP12E.

En esta nos centraremos en los aspectos propios de la placa NodeMCU más allá de las "heredadas por montar un ESP12E. Como decíamos en la introducción, todo lo relacionado con NodeMCU puede llegar a ser confuso porque existe más de un fabricante y cada uno tiene sus propios criterios.

Para colmo, los vendedores chinos de AliExpress o eBay redactan las descripciones de sus artículos como si les pagaran por palabra, mezclando todo y haciéndolo innecesariamente complicado.

Por resumir, tenemos tres principales fabricantes de NodeMCU, Amica, Lolin/Wemos y DOIT/SmartArduino. Las placas son muy similares (o incluso idénticas), aunque pueden tener alguna diferencia de designación de los pines.

Por otro lado, tenemos tres versiones de la placa NodeMCU, que veremos a continuación. Las designaciones de las versiones también contribuyen a generar confusión. Pero, aunque parece un poco complicado, en realidad es sencillo si sigues la evolución desde el principio. (Luis Llamas, 2018).

2.3.3 PRIMERA GENERACIÓN

La versión original del NodeMCU se denominó devkit v0.9, y montaba un ESP12 junto a 4MB de flash (recordemos que la memoria en el ESP8266 es externa y se conecta por SPI). El ESP12 es similar al ESP12E, pero carece de una hilera de pines por lo que dispone de menos GPIO.

La versión 0.9 era poco atractiva, amarilla-anaranjada y muy ancha. Con unas dimensiones de 47x21mm ocupaba 10 hileras de pins de una placa breadboard, por lo que la tapa por completo. Esto la hacía muy poco práctica de usar porque no deja pines libres en la breadboard para realizar conexiones. (Luis Llamas, 2018).



Figura 2.7: NodeMcu ESP8266 v0.9

Fuente: (Luis Lamas)

2.3.4 SEGUNDA GENERACIÓN V1.0/V2

La siguiente versión del NodeMCU es la v1.0 V2. De forma resumida Amica, una compañía creada por el alemán Gerwin Janssen, fabricó su propia versión mejorada de la v0.9. Al equipo original de NodeMCU le gustó y la declararon versión "oficial" de NodeMCU.



Figura 2.8: NodeMcu ESP8266 Amica v1.0/v2
Fuente: (Equipo NodeMCU)

La principal diferencia de la versión v1.0 v2 es que monta un ESP12E en lugar de un ESP12, por lo que tiene más pines disponibles que el modelo original. Además, es más estrecha que la versión 0.9, tapando sólo 8 hileras de una breadboard. Esto deja una hilera adicional a cada lado para realizar conexiones.

Los tres fabricantes, Amica, Lolin/Wemos y DOIT/SmartArduino fabrican, o han fabricado en algún momento, esta versión v1.0 V2. (Luis Llamas, 2018).

2.3.5 TERCERA GENERACIÓN V1.0/V3

Llegamos a lo que a veces se denomina "tercera generación", la versión 1.0 V3. Básicamente el fabricante Lolin/Wemos decidió crear su propio diseño "mejorado" con unos cuantos cambios menores.

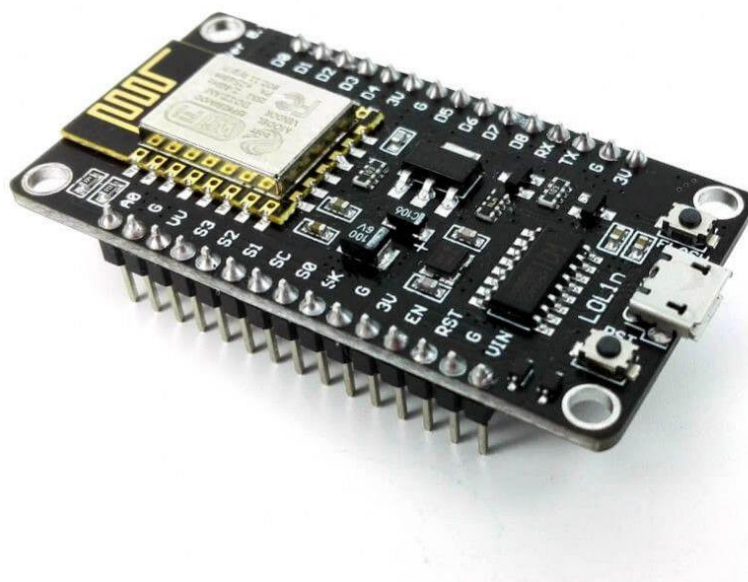


Figura 2.9: NodeMcu ESP8266 LOLIN v1.0/v3

Fuente: (Lolin.com)

El cambio principal es que la V3 monta un conversor serial CH340G en lugar del CP2102. El fabricante asegura que hace que el puerto USB sea más robusto. Por otro lado, reusaron los dos pines reservados de la V2 para sacar un GND y un VUSB.

Por lo demás, las especificaciones son las mismas. Por contra, el modelo v1.0 V3 vuelve a ser ancho y tapar toda la breadboard, lo cual es un auténtico problema a la hora de realizar montajes con breadboard

Aun así, la 1.0 V3 es seguramente el modelo más vendido ahora mismo. Aunque mucha gente busca la V2 porque son prácticamente iguales en funcionalidades, pero al ser estrecha es más cómoda de usar en una breadboard. (Luis Llamas, 2018).

2.3.6 ESP32

El ESP32 es un SoC (System on Chip) diseñado por la compañía china Espressif y fabricado por TSMC. Integra en un único chip un procesador Tensilica Xtensa de doble núcleo de 32bits a 160Mhz (con la posibilidad de hasta 240Mhz), conectividad WiFi y Bluetooth.

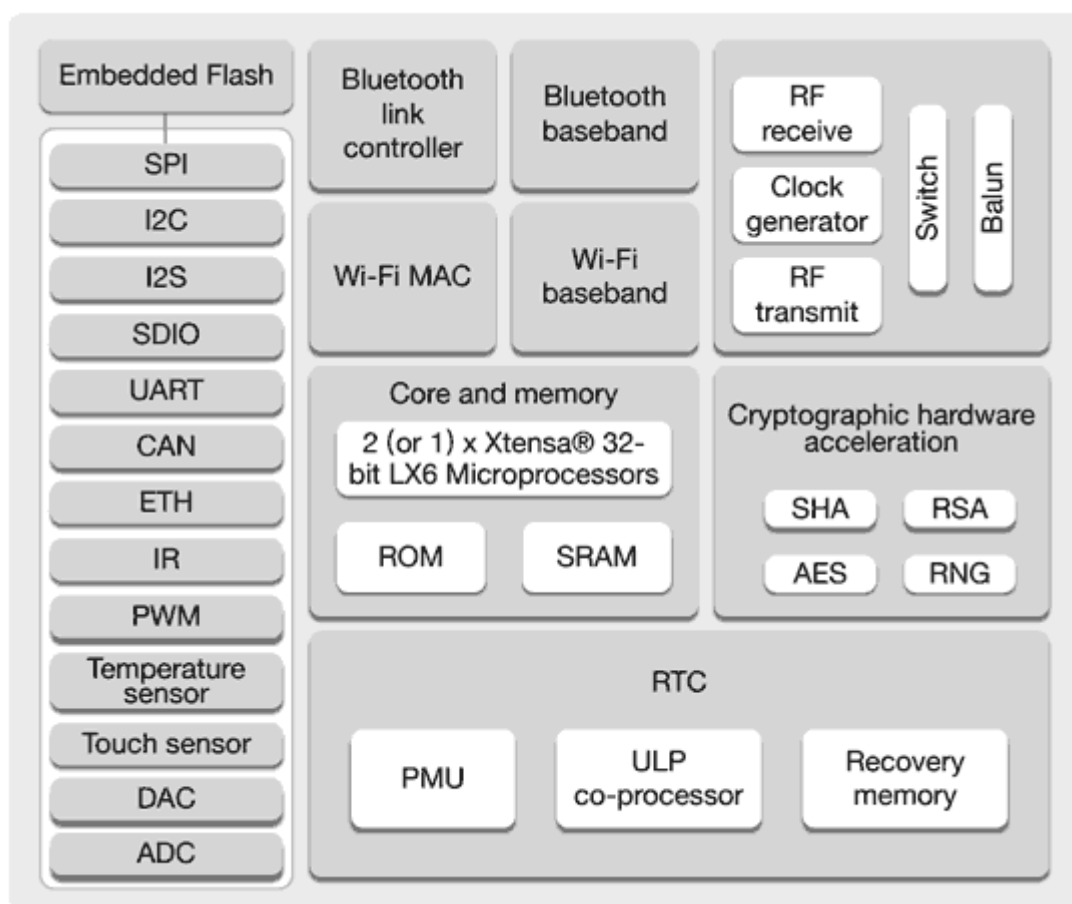


Figura 2.10: Estructura SOC ESP32

Fuente: (Espressif.com)

El ESP32 añade muchas funciones y mejoras respecto a el ESP8266, como son mayor potencia, Bluetooth 4.0, encriptación por hardware, sensor de temperatura, sensor hall, sensor táctil, reloj de tiempo real RTC, más puertos, más buses entre otras características.

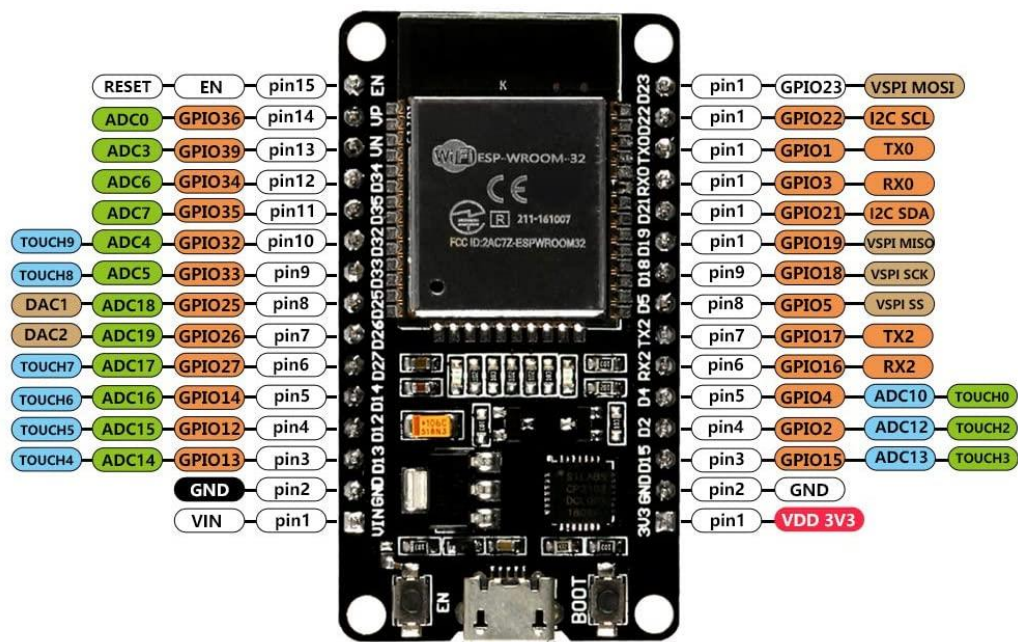


Figura 2.11: PinOut ESP32

Fuente: (Espressif.com)

Como no podía ser de otra forma la comunidad Maker acogió el nuevo ESP32 con los brazos abiertos. Se han desarrollado firmwares, documentación, herramientas y aunque su soporte es aún inferior al del ESP8266, en la actualidad es fácil encontrar tutoriales sobre el mismo y continuamente se publican nuevos artículos. (Luis Lamas, 2018)

Por supuesto, los fabricantes están atentos y han desarrollado numerosas placas de desarrollo que integran el ESP32. Algunas tienen batería LiPo tipo 16050, otras TFT,

otras pantallas OLED, comunicación por Lora y cada vez aparecen nuevas opciones, algunas reamente innovadoras.

En cuanto a lenguajes de programación tenemos varias opciones, básicamente similares a las que vimos en ESP8266. Es posible el IDE de Arduino, instalar MicroPython, RTOS, Mongoose OS, Espruino.

A continuación, detallamos algunas de sus características principales:

- Procesador Xtensa LX6 de 32 bits de doble núcleo.
- Velocidad de 160Mhz (máximo 240 Mhz)
- Co-procesador de ultra baja energía.
- Memoria 520 KiB SRAM.
- Memoria flash externa hasta 16MiB.
- Encriptación de la Flash.
- Arranque seguro.
- Pila de TCP/IP integrada.
- Wifi 802.11 b/g/n 2.4GHz (soporta WFA/WPA/WPA2/WAPI).
- Bluetooth v4.2 BR/EDR y BLE.
- Criptografía acelerada por hardware.
- 32 pins GPIO
- Conversor analógico digital (ADC) de 12 bits y 18 canales.
- 2 conversores digital analógico (DAC) de 8 bits.
- 16 salidas PWM (LED PWM).

- 1 Salida PWM para motores.
- 11 conversor analógico a digital de 10 pin.
- 10x sensores capacitivos (en GPIO)
- 3x UART, 4x SPI, 2x I2S, 2x I2C, CAN bus 2.0.
- Controladora host SD/SDIO/CE-ATA/MMC/eMMC.
- Controladora slave SDIO/SPI.
- Sensor de temperatura.
- Sensor de efecto Hall.
- Generador de números aleatorios.
- Reloj tiempo real (RTC).
- Controlador mando a distancia infrarrojos (8 canales).

2.4 BLYNK IOT

Blynk es una plataforma de IoT independiente del hardware con aplicaciones móviles de marca blanda, nubes privadas, administración de dispositivos, análisis de datos y aprendizaje automático.



Figura 2.12: Logo Blynk
Fuente: (Blynk.cc)

Blynk permite que cualquiera pueda controlar fácilmente su prototipo con un dispositivo con sistema iOS o Android. Los usuarios tienen la posibilidad de crear una interfaz gráfica de usuario de fácil interacción y creación para su proyecto en cuestión de minutos y sin ningún gasto extra, gracias a sus componentes básicos de soluciones completas de IoT.



Figura 2.13: Vista de la App Blynk

Fuente: (Blynk.cc)

Blynk combina una plataforma en la nube con aplicaciones que ponen las cosas, las personas y los datos en el centro de las operaciones comerciales. Su amplia compatibilidad con el Hardware Open Source, convirtiéndose en una solución indispensable para cualquier prototipo (Blynk.IO, 2018). Cuenta con todo lo que se necesite usar para controlar y monitorizar un prototipo, desde deslizadores y pantalla a gráficos y otros widgets

funcionales que se pueden organizar en la pantalla y mostrar los datos de sensores para graficarlos.

Tecnología Humanizada, indica que Blynk permite al usuario controlar hardware de forma remota. Monitorear sensores, guardar datos y visualizarlos. Las partes principales que conforman el sistema son las siguientes:

- La Aplicación Blynk, la cual permite crear las interfaces de control para tu proyecto.
- El servidor Blynk server responsable de todas las comunicaciones entre el teléfono celular y el hardware. Se puede utilizar el servidor online de Lbynk o instalar tu propio servidor Blynk local. El servidor Blynk es una aplicación de código abierto.

Las librerías Blynk se encuentran disponibles para diferentes plataformas de desarrollo permitiendo la comunicación entre el servidor y los recursos del hardware.

Existen los pines virtuales proporcionados por Blynk, que se usan comúnmente para interactuar con otras bibliotecas (Servo, LCD y otras) e implementar una lógica a medida de las necesidades de la aplicación. El dispositivo a su vez puede enviar datos a la aplicación mediante `BLYNK_WRITE(vPin)`. Los datos enviados a un pin virtual son enviados como strings (cadena de caracteres de texto), sin embargo por supuesto hay una limitación dada por el hardware para la manipulación numérica. Por ejemplo Arduino, los enteros son de 16 bits, con un rango de -32768 a 32767. (Tecnología Humanizada, 2018)

2.5 SENSORES Y ACTUADORES

2.5.1 SENSORES

Un sensor es un dispositivo capaz de detectar magnitudes físicas o químicas, llamadas variables de instrumentación, y transformarlas en variables eléctricas. Las variables de instrumentación pueden ser, por ejemplo: temperatura, intensidad lumínica, distancia, aceleración, inclinación, desplazamiento, presión, fuerza, torsión, humedad, movimiento, pH, etc. Una magnitud eléctrica puede ser una resistencia eléctrica (como en una RTD), una capacidad eléctrica (como en un sensor de humedad o un sensor capacitivo), una tensión eléctrica (como en un termopar), una corriente eléctrica (como en un fototransistor), etc.

Los sensores se pueden clasificar en función de los datos de salida en:

- Digitales
- Analógicos
- Comunicación por Bus

Los Sensores van conectados a las entradas de un Arduino o cualquier Microcontrolador usando la electrónica necesaria.

A la hora de elegir un sensor, debemos leer detenidamente las características y elegir uno que sea compatible con nuestro sistema (tensión y voltaje) y que sea sencillo de usar o nos faciliten una librería y potente.

2.5.1.1 SENSOR DE FLUJO DE AGUA YF – S201



Figura 2.14: Sensor de Flujo YF-S201
Fuente: (Dynamo Electronics)

El sensor de Flujo YF-S201 sirve para medir el flujo de agua, por ejemplo, de un invernadero, o en una casa como en un proyecto, resulta muy importante conocer el consumo de líquido. Este sensor se instala en la línea principal del agua, sirviendo para medir la cantidad de líquido que se ha movido a través de él. El aspa del sensor tiene un pequeño imán asegurado y hay un sensor magnético de efecto Hall en el otro lado del tubo de plástico capaz de medir la cantidad de revoluciones del aspa de viento que ha hecho a través de la pared de plástico. Este método permite que el sensor permanezca seguro y seco.



Figura 2.15: Sensor de Flujo YF-S201 abierto

Fuente: (Hetpro-Store.com)

El sensor viene con tres cables: rojo (potencia 5-24VDC), negro (a tierra) y amarillo (salida de impulsos de efecto Hall). Al contar los pulsos de la salida del sensor, puede seguir fácilmente el movimiento del fluido: cada pulso es de aproximadamente 2,25 mililitros.

A continuación, detallamos sus características precisas:

- Modelo: YF – S201.
- Voltaje de Operación: 5V – 18V DC.
- Consumo de Corriente: 15mA(5V).
- Capacidad de carga: 10mA (5 VDC).
- Salida: Onda cuadrada pulsante.
- Rango por litro: 450 pulsos.

- Factor de conversión: 7,5
- Rosca externa: ½" NPS.
- Presión de trabajo máxima: 1.75MPa (17bar).
- Temperatura de funcionamiento: -25°C a 80°C.
- Material: Plástico color negro.
- Trabajo Caudal: de 1 a 30 litros/minuto.
- Humedad de trabajo: 35% - 80% de humedad relativa.
- Precisión: +/- 2%
- Rango Flujo: 1-30 L/min.
- Modo de detección: Vertical.
- Durabilidad: un mínimo de 300.000 ciclos.

2.5.1.2 SENSOR DE VOLTAJE

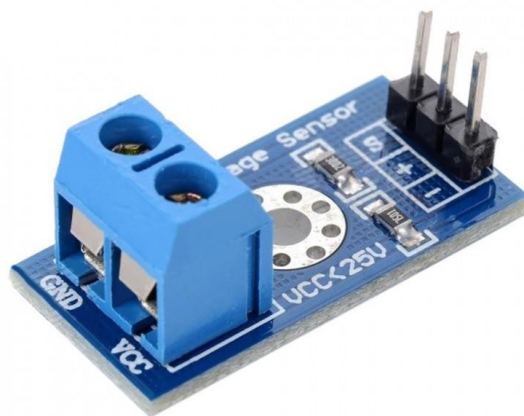


Figura 2.16: Sensor de Voltaje FZ0430
Fuente: (C&D Tecnología)

El medidor de voltaje FZ0430 es un simple divisor de tensión con resistencias de 30K 7K5, lo que supone que la tensión percibida por el módulo sea dividida por un factor de 5 ($7.5/(30+7.5)$).

La tensión máxima que podemos medir será 25v para un procesador de tensión de alimentación 5v (Vcc), y 16,5v para un procesador de 3.3v (Vcc). Superar esta tensión máxima en la entrada del FZ0430 dañara el pin analógico de Arduino. La resolución de la medición del módulo es de 24,45mV.

Este sensor es muy útil para medir el estado de una batería, comprobar la alimentación de un dispositivo de 12v o 24v, como una tira LED, un electroimán, un ventilador, o una célula Peltier por ejemplo.

A continuación, se detalla sus características precisas:

- Rango de entrada de voltaje: 0v a 25v DC.
- Voltaje de detección entrada máximo: 25v ($5v \times 5 = 25v$) o 16.5v ($3.3v \times 5 = 16.5v$).
- Rango de detección de voltaje: 24,41mV – 25v.
- Resolución analógica de tensión: 0,00489v DC.
- Voltaje detección entrada mínimo: 24,45mV ($4,89mV \times 5 = 24,45mV$).

2.5.2 ACTUADORES Y PERIFÉRICOS

Un actuador es un dispositivo capaz de transformar energía hidráulica, neumática o eléctrica en la activación de un proceso con la finalidad de generar un efecto sobre elemento externo. Este recibe la orden de un regulador, controlador o en nuestro caso un Arduino y en función a ella genera la orden para activar un elemento final de control como, por ejemplo, una válvula.

Existen varios tipos de actuadores como son:

- Electrónicos
- Hidráulicos
- Neumáticos
- Eléctricos
- Motores
- Bombas

Los actuadores van conectados a las salidas de Arduino o de cualquier Microcontrolador usando la electrónica necesaria.

Periférico es la denominación genérica para designar al aparato o dispositivo auxiliar e independiente conectado a la unidad central de procesamiento o en este caso a Arduino. Se consideran periféricos a las unidades o dispositivos de hardware a través de los cuales Arduino se comunica con el exterior, y también a los sistemas que almacenan o archivan la información, sirviendo de memoria auxiliar de la memoria principal.

Ejemplos de periféricos:

- Pantallas LCD

- Teclados
- Memorias externas
- Cámaras
- Micrófonos
- Impresoras
- Pantalla táctil
- Displays numéricos
- Zumbadores
- Indicadores luminosos

Para cada actuador o periférico necesitamos un driver o manejador para poder mandar órdenes desde Arduino.

A la hora de seleccionar un actuador o periférico para usar con Arduino habrá que ver sus características y cómo hacer el interface con Arduino o similares. En el playground de Arduino existe una gran base de datos de conocimientos para conectar Arduino con casi cualquier Hardware.

2.6 MEDIDOR DE AGUA CONVENCIONAL

Un medidor de agua potable convencional contabiliza el flujo de agua a través de un conducto, en el cual va incorporado un sistema de despacho de líquido potable y se puede conocer datos de la cantidad que pasa a través de este, como a cantidad de energía consumida por algún aparato si se instala de manera particular (PedroJ, 2020).

Es uno dispositivos más usados en departamentos, hogares, negocios y todo lugar donde se requiera de una instalación de agua potable, permite calcular la factura del cobro de agua

potable de acuerdo al volumen contabilizado en un periodo de tiempo determinado, en el caso particular de nuestro País, se realiza de manera mensual.

2.6.1 TIPOS DE MEDIDORES DE AGUA POTABLE

Existen diversos tipos de medidores de agua potable, mencionaremos los más utilizados por empresas y cooperativas de aguas.

2.6.1.1 MEDIDOR DE VELOCIDAD DE AGUA POTABLE



Figura 2.17: Medidor de velocidad de agua potable
Fuente: (MaterialdeLaboratorio.com)

Dispositivo utilizado para medir la velocidad del flujo de agua a través de una red de agua potable, de este modo se conoce cuál es el caudal y cómo circula.

Son aparatos sencillos de bajo coste y con una precisión aceptable, se puede programar para saber la cantidad de agua que atraviesa en un chorro único, también la velocidad en diversos chorros y en la tangente del fluido cuando se da paso libre.

2.6.1.2 CONTADOR DE AGUA ELECTROMAGNÉTICO



Figura 2.18: Contador de Agua Electromagnético

Fuente: (MaterialdeLaboratorio.com)

Su funcionamiento es similar al anterior, pero en esta ocasión lo que se miden son las propiedades electromagnéticas de la velocidad de agua que pasa por una turbina en vez del chorro. La filosofía de este tipo de medidor está basada en la ley de inducción de Faraday y no disponen de elementos mecánicos, en sustitución, presentan sensores que miden la tensión eléctrica resultante a la velocidad con la que fluye el agua.

2.6.1.3 MEDIDOR DE ULTRASONIDO



Figura 2.19: Medidor de Ultrasonido de agua potable

Fuente: (MaterialdeLaboratorio.com)

Es un tipo de medidor que envía ondas de ultrasonidos para determinar y contar la velocidad con la que atraviesa el fluido de agua por una red de distribución, Es uno de los más usados tanto en los establecimientos comerciales, como en los hogares.

Utilizan dos tecnologías. La primera en el efecto Doppler y el segundo en tránsito que miden el tiempo en que atraviesa el agua en más de dos puntos dentro del aparato.

2.7 MODELO EN V O DE 4 NIVELES

El modelo en V es un modelo SDLC7 donde el proceso de ejecución pasa de forma secuencial, también conocido como modelo de Verificación y Validación.

El modelo en V es una extensión del modelo en Cascada y asocia una fase de prueba a cada etapa de desarrollo. Este es un modelo altamente disciplinado, donde la siguiente fase empieza solo después de haberse completado la fase previa (tutorialespoint.com, 2008).

2.7.1 DISEÑO DEL MODELO EN V

Bajo el modelo en V, la fase de prueba correspondiente a la fase de desarrollo está planeado paralelamente. Así que hay fases de verificación en un lado de la V y fases de validación en el otro lado, ver Figura 2.

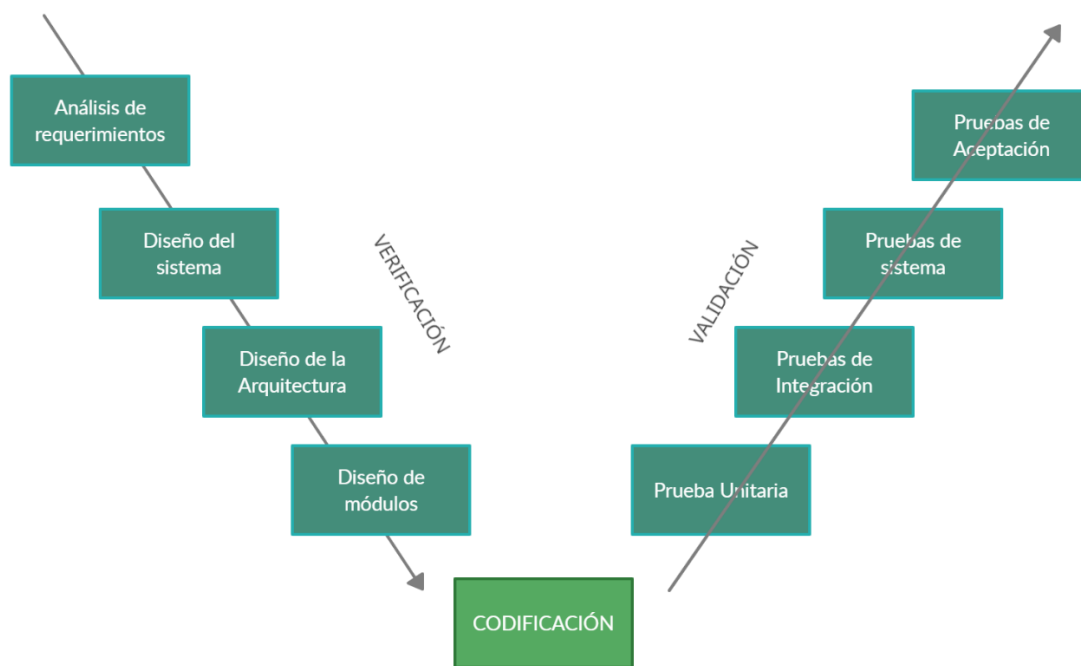


Figura 2.20: Fases del Modelo en V

Fuente: (tutorialespoint.com, 2008)

2.7.2 FASES DE VERIFICACIÓN

Según (tutorialespoint.com, 2008) se tiene las siguientes etapas en la fase de verificación del modelo en V:

- **Análisis de Requerimientos**

Esta es la primera fase en el ciclo de desarrollo donde los requerimientos del producto son entendidos desde la perspectiva del cliente. Esta fase implica comunicación detallada con el cliente para entender sus expectativas y sus requerimientos. Esta es una actividad muy importante y necesita ser bien manejada, ya que muchas veces el usuario no necesariamente sabe lo que necesita.

- **Diseño del Sistema**

Una vez que se tenga claro y detallado los requerimientos del producto, es hora de diseñar el sistema completo. El diseño del sistema debería comprender a detalle el hardware completo y la configuración de la comunicación. El sistema de plan de pruebas está desarrollado basado en el sistema de diseño.

- **Diseño de la Arquitectura**

Las especificaciones de la arquitectura son entendidas y diseñadas en esta etapa. Por lo general más de un enfoque técnico es propuesto y la decisión final se basa en la viabilidad financiera. El diseño del sistema se desglosa en módulos con diferentes funcionalidades.

Los datos transferidos y la comunicación entre los módulos internos y el mundo exterior son claramente entendidos y definidos en esta etapa. Con esta información las pruebas de integración pueden ser diseñadas y documentadas a lo largo de la etapa.

- **Diseño de Módulos**

En esta fase el diseño interno para todos los módulos del sistema es especificado, referidos como LLD₈ o diseño de bajo nivel. Es importante que el diseño sea compatible con otros módulos en la arquitectura del sistema y otros sistemas externos. Las pruebas unitarias son parte de cualquier proceso de desarrollo y ayudan a eliminar fallas y errores en etapas tempranas. Las pruebas unitarias pueden ser diseñadas en esta etapa basadas en el diseño de módulos internos.

2.7.3 FASE DE CODIFICACIÓN

La codificación actual de módulos del sistema diseñada en la fase de diseño es tomada en la fase de codificación. El lenguaje de programación más adecuado es elegido basado en el sistema y lo requerimiento arquitecturales. La codificación se ejecuta basada en directrices de codificación y estándares. El código es pasa a través de numerosas revisiones de código y es optimizada para mejor rendimiento.

2.7.4 FASE DE VALIDACIÓN

Según tutorialpoint.com (2008) las siguientes son las fases de validación en el modelo en V:

- **Pruebas unitarias**

Las pruebas unitarias diseñadas en la fase de módulo de diseño son ejecutadas en el código durante esta fase de validación. Las pruebas unitarias son pruebas a nivel de código y ayudan a eliminar bugs en etapas tempranas.

- **Pruebas de integración**

Las pruebas de integración están asociadas con la fase de diseño de arquitectura. Las pruebas de integración son ejecutadas para probar la coexistencia y comunicación de módulos internos dentro del sistema.

- **Pruebas de sistema**

Las pruebas de sistema están directamente asociadas con la fase de diseño de sistema. Las pruebas de sistema verifican la funcionalidad del sistema entero y la comunicación del sistema en desarrollo con sistemas externos. Mucho de los problemas de compatibilidad de software y hardware pueden ser descubiertos acá.

- **Pruebas de aceptación**

Las pruebas de aceptación están asociadas con los requerimientos del negocio en la fase de análisis e involucra pruebas del producto en el entorno del usuario. Las pruebas de aceptación descubren problemas de compatibilidad con otros sistemas

disponibles en el entorno del usuario, también descubre problemas no funcionales como defectos de carga y rendimiento.

2.7.5 APLICACIÓN DEL MODELO EN V

El modelo en V es casi el mismo que el modelo en cascada, ambos modelos son de tipo secuencial. Los requerimientos tienen que estar muy claros antes de que el proyecto empiece por que por lo general es costoso volver atrás y hacer cambios. Los siguientes escenarios son aconsejables para el uso del modelo en V:

- Los requerimientos deben estar bien definidos y documentados claramente.
- La definición del producto es estable.
- La tecnología no es dinámica y debe estar bien entendida por el equipo de desarrollo.
- El proyecto es pequeño.

2.7.6 MODELO EN V VENTAJAS Y DESVENTAJAS

Las ventajas del modelo en V es que es muy fácil de entender y de aplicar. La simplicidad de este modelo hace que sea muy fácil manejarlo. La desventaja es que este modelo no es flexible para cambios solo en el caso de que haya un cambio en el requerimiento, lo cual es muy común hoy en día. (tutorialespoint.com, 2008).

CAPITULO III

3. MARCO APLICATIVO

3.1 INTRODUCCIÓN

En el presente capitulo se desarrolla el prototipo de Medidor de Agua IoT, para el diseño del prototipo se enfocará en el Hardware que constará del sistema Sensor, para tal objetivo se utilizará Modelo V. Ver Figura 2.20

3.2 DESCRIPCIÓN INFORMAL DEL SISTEMA

El modelo de Sistema IoT contará con una interfaz visual mediante la cual el usuario final podrá visualizar los datos sobre el consumo de agua en su hogar, creando una experiencia de control del sistema IoT. En la figura 3.1, se muestran la entradas y salidas del sistema en general.

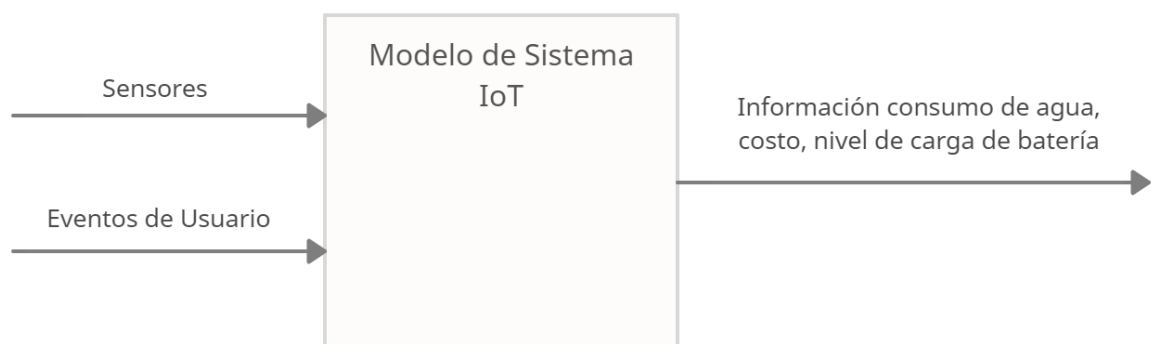


Figura 3.1: Entradas y salidas del sistema IoT

Fuente: (Elaboración propia)

3.2.1 ENTRADAS

En el sistema se detectan dos tipos de entrada, la primera son los datos capturados por el sensor de Flujo de Agua y el Sensor de Nivel de Voltaje, el segundo tipo de entrada, es el evento realizado por el usuario al presionar el Botón Momentáneo.

3.2.2 PROCESOS

Los datos recibidos por el Sensor de Flujo de Agua son procesados para posteriormente enviar el resultado a los periféricos físicos de Salida.

Los datos recibidos por el Sensor de Nivel de Voltaje son procesados para posteriormente enviarlos a los actuadores.

3.2.3 SALIDAS

Las salidas del Sistema se manifiestan mediante evento físicos que son ejecutados por los actuadores, como son graficar los datos procesados del Sensor de Flujo de Agua en la pantalla LCD 16x2 y el nivel de Voltaje que se proyecta en el Display de 10 segmentos, además estos deben visualizarse en la interfaz gráfica de Blynk.

3.3 DESCRIPCIÓN FORMAL DEL SISTEMA

En el modelo del Sistema IoT, se identifican 3 componentes principales que son importantes para cumplir el objetivo propuesto:

- Interfaz Blynk
- Servidor Blynk
- Hardware Electrónico

Los cuales se pueden observar en la Figura 3.2.

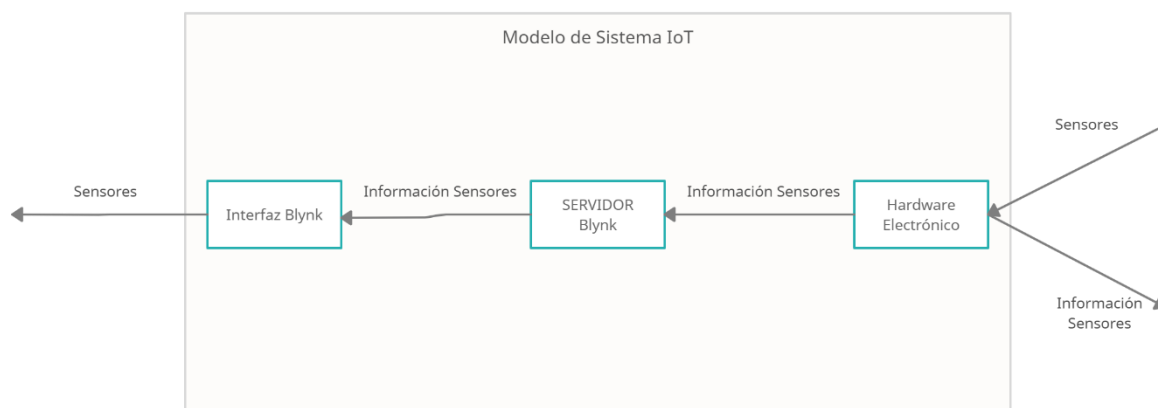


Figura 3.2: Entradas y salidas de los subsistemas

Fuente: (Elaboración propia)

Interfaz Blynk

Este módulo permitirá la interacción del usuario final con el Prototipo de Medidor de Agua IoT, mediante una interfaz prefabricada por Blynk, desde donde el usuario final podrá controlar su consumo de Agua Potable y verificar el estado de su medidor IoT.

Servidor Blynk

El servidor será el mediador para el almacenamiento e intercambio de datos entre el Software Administrador y el Hardware Electrónico, en este caso se utilizará el Servidor proporcionado por Blynk.

Hardware Electrónico

Procesa los datos recibidos por los sensores para convertirlos en acciones que se verán reflejadas en los periféricos y actuadores del mismo Hardware, así como en la interfaz Blynk.

A continuación, se muestra en la figura 3.3 el diseño modular del sistema y la interacción con la interfaz de Blynk.

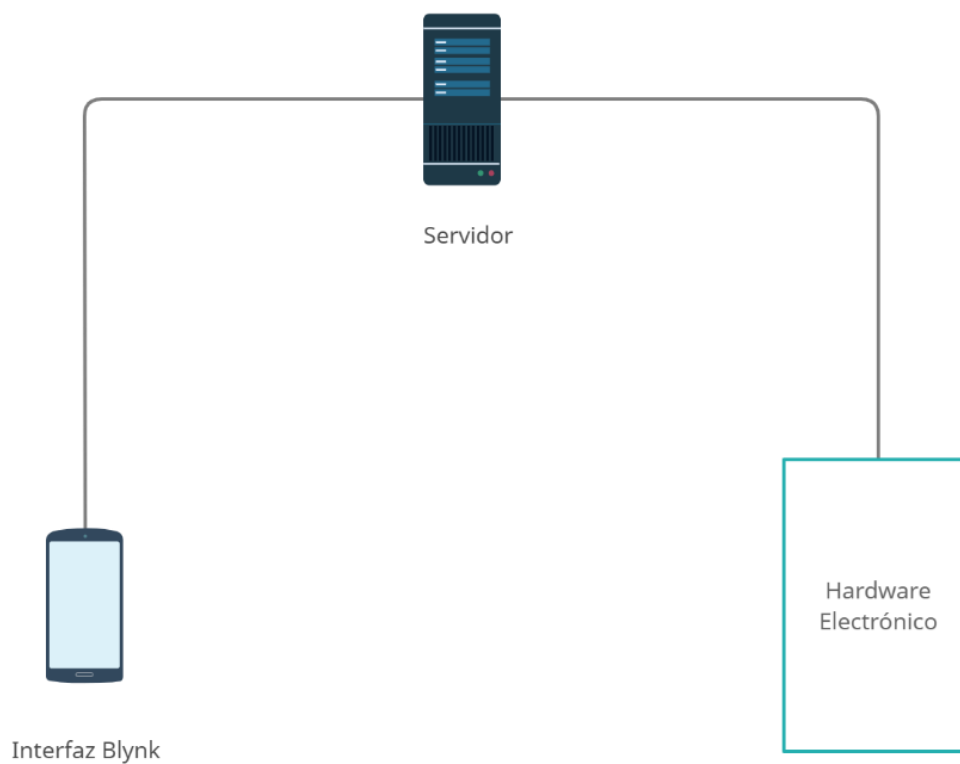


Figura 3.3: Diseño Global
Fuente: (Elaboración propia)

3.4 IMPLEMENTACIÓN

3.4.1 MODELO V

A continuación, se detallan los acápites descritos en las Fases del Modelo en V:

1. Análisis de requerimientos
2. Diseño de arquitectura
 - 2.1 Identificación de elementos de datos
 - 2.2 Identificación de objetos
 - 2.3 Modelado de Objetos
 - 2.3.1 Gráficos
 - 2.3.2 Comportamientos
 - 2.3.3 Interacciones
 - 2.3.4 Comunicaciones Internas
 - 2.3.5 Comunicaciones Externas
3. Diseño de modulo
4. Codificación

3.4.2 ANALISIS DE REQUERIMIENTOS

El objetivo principal en esta etapa es identificar y documentar lo que se necesita, de forma tal que pueda ser fácilmente transmitido al cliente y al desarrollador. Por tal se tocarán los siguientes puntos.

- Panorama.
- Metas.
- Funciones del sistema.

Panorama general

Dar al usuario final una experiencia de control y monitoreo sobre el consumo de agua potable.

Metas

Este proyecto tiene como objetivo crear un prototipo de medidor de Agua IoT que permita controlar y monitorizar el consumo de agua potable.

Funciones del Sistema

Las funciones del sistema son lo que este deberá hacer. Las funciones se agrupan en 3 grupos que son:

- Funciones de interfaz Blynk.
- Funciones de servidor.
- Funciones del hardware electrónico.

En la tabla 3.1 se describen las funciones que debe cumplir a interfaz Blynk para poder controlar y monitorizar el consumo de Agua.

Tabla 3.1: Funciones Interfaz Blynk
Fuente: (Elaboración propia)

Funciones de interfaz Blynk		
Referencia	Función	Categoría
R1.1	Visualización del consumo de Agua en el tiempo.	Evidente
R1.2	Visualización del consumo de agua actual.	Evidente
R1.3	Visualización del costo de consumo actual.	Evidente
R1.4	Visualización del Nivel de Batería	Evidente

En la tabla 3.2 se describen las funciones del servidor para registrar datos de usuarios y sensores.

Tabla 3.2: Funciones de Servidor

Fuente: (Elaboración propia)

Funciones de Servidor		
Referencia	Función	Categoría
R2.1	Envía y recibe información de los Sensores	Evidente
R2.2	Almacena Información de los Sensores	Evidente

En la tabla 3.3 se describen que debe cumplir el Hardware Electrónico para recibir datos de los sensores y posteriormente interactuar con los periféricos y actuadores.

Tabla 3.3: Funciones Hardware Electrónico

Fuente: (Elaboración propia)

Funciones del Hardware Electrónico		
Referencia	Función	Categoría
R3.1	Leer la información de los sensores	Evidente
R3.2	Encendido y apagado display 10 segmentos	Evidente
R3.3	Encendido, apagado y actualización del Display LCD	Evidente
R3.4	Envía/recibe información en forma de bytes	Oculto

Actores

Para la utilización de los casos de uso primeramente se deben reconocer un conjunto coherente de roles que juegan los actores al momento de interactuar con el sistema.

Según las tablas de funciones del sistema se pueden reconocer dentro del sistema los siguientes actores:

- Usuario
- Interfaz Blynk
- Servidor
- Hardware electrónico

Actor Usuario

El cliente es el usuario que podrá manipular la interfaz Blynk, así como el prototipo IoT.

Casos de Uso

Los casos de uso describirán la interacción que tienen los actores con el sistema y la interacción entre subsistemas.

En la tabla 3.4 y figura 3.4 se describe la interacción que tienen los actores Usuario y Servidor con la Interfaz Blynk.

Tabla 3.4: Casos de uso Interfaz Blynk
Fuente: (Elaboración propia)

Caso de Uso	Software Cliente
Actores	Usuario, Servidor
Propósito	Proporciona una Interfaz en un smartphone, mediante el cual el usuario podrá interactuar con el prototipo IoT.
Resumen	El usuario podrá observar la información asociada al prototipo IoT, como el consumo de agua actual, costo del consumo y nivel de batería
Tipo	Primario
Referencias	R1.1, R1.2, R1.3, R1.4, R2.1, R2.2,

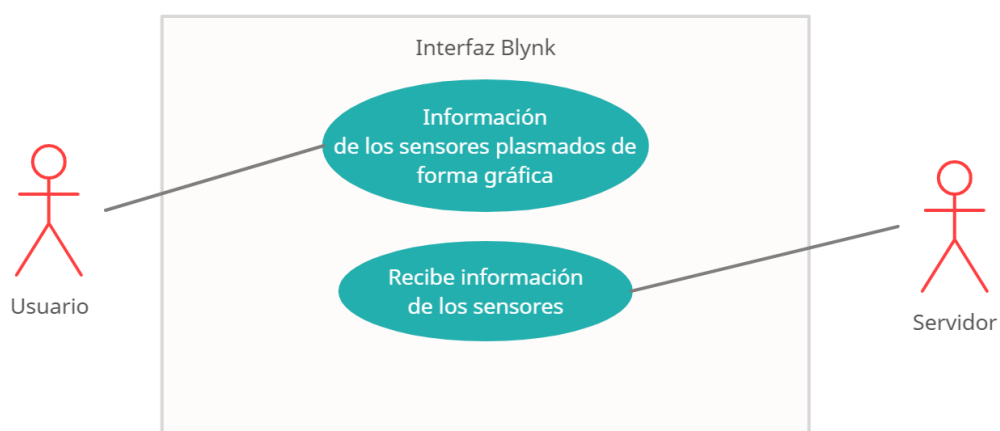


Figura 3.4: Casos de uso para la interacción del Usuario

Fuente: (Elaboración propia)

En la tabla 3.5 y figura 3.5 se describe la interacción que tienen los Actores Interfaz Blynk y circuito electrónico con el Servidor.

Tabla 3.5: Casos de uso Servidor Blynk

Fuente: (Elaboración propia)

Caso de Uso	Servidor Blynk
Actores	Hardware Electrónico, Interfaz Blynk
Propósito	Gestiona y almacena datos
Resumen	Gestiona y almacena la información de los sensores
Tipo	Primario
Referencias	R2.1, R2.2

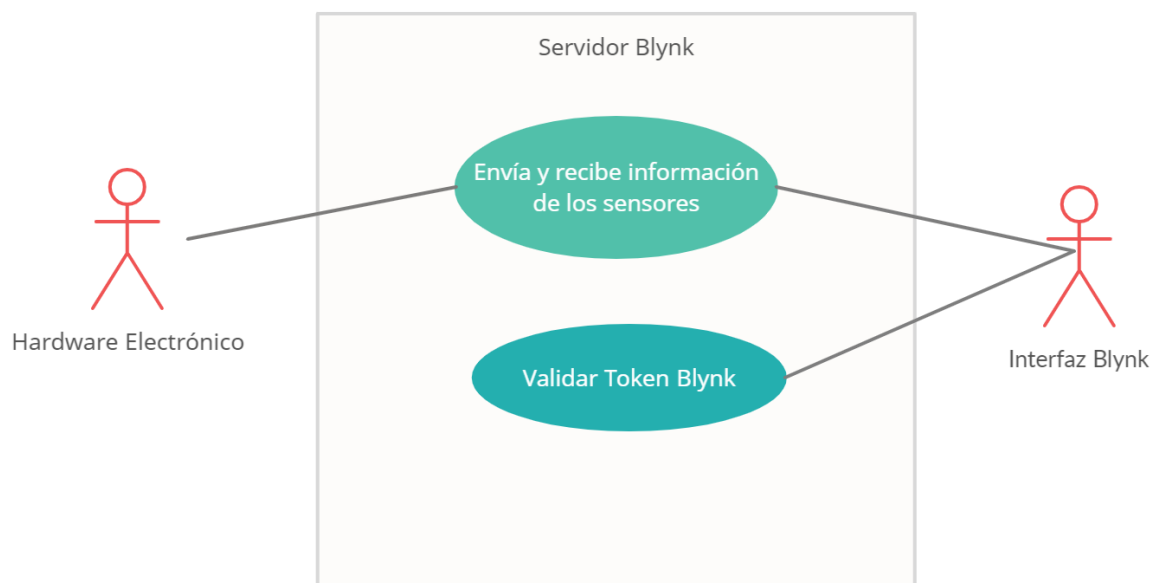


Figura 3.5: Casos de uso para el Servidor

Fuente: (Elaboración propia)

En la figura 3.6 y la tabla 3.6 se describe las interacciones que tiene el circuito electrónico con el Servidor.

Tabla 3.6: Casos de uso Hardware Electrónico

Fuente: (Elaboración propia)

Caso de Uso	Hardware Electrónico
Actores	Servidor
Propósito	Procesa los datos recibidos de los sensores para enviarlos al servidor y actuadores
Resumen	El sistema capta la información recibida de los sensores, para procesarlos y luego enviarlos al servidor y actuadores.
Tipo	Primario
Referencias	R3.1, R3.2, R3.3, R3.4

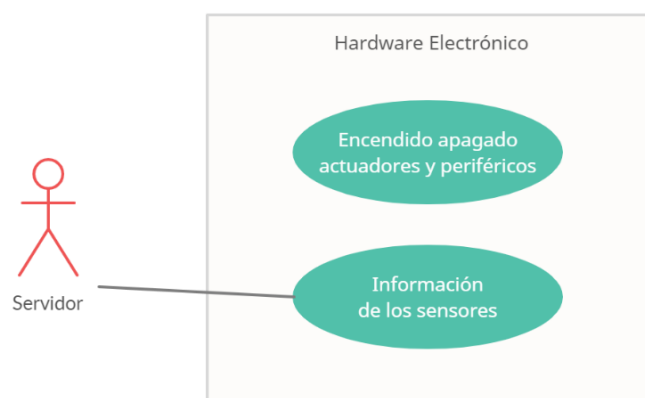


Figura 3.6: Casos de uso para el Circuito Electrónico
Fuente: (Elaboración propia)

Diagrama de Secuencias

Los diagramas de secuencia muestran los eventos que originan los actores y que impactan en el sistema. La creación de los diagramas de secuencias dependerá de la formulación de los casos de uso. Los diagramas de secuencia representan el comportamiento del sistema, como se muestra en la figura 3.7.

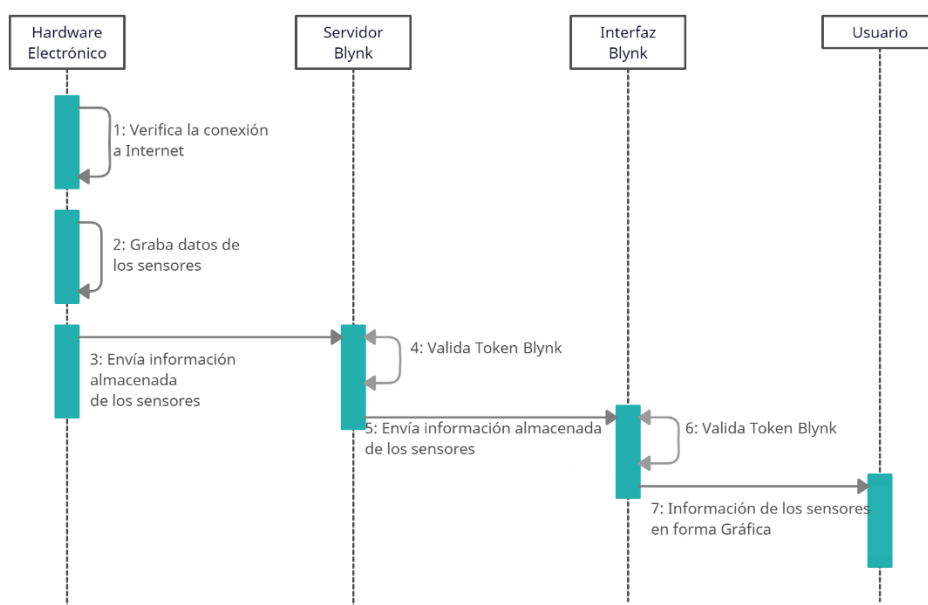


Figura 3.7: Diagrama de Secuencias
Fuente: (Elaboración propia)

3.4.3 DISEÑO DE ARQUITECTURA

La arquitectura del Sistema IoT será de tipo cliente servidor, donde el cliente es la Interfaz Blynk que es una app móvil, este hará peticiones al servidor para poder extraer información sobre el Hardware electrónico y ver el estado de los sensores. La Arquitectura para este prototipo será centralizada debido a que el servidor es el centro de la red y puede suministrar recursos al cliente, ver la figura 3.8

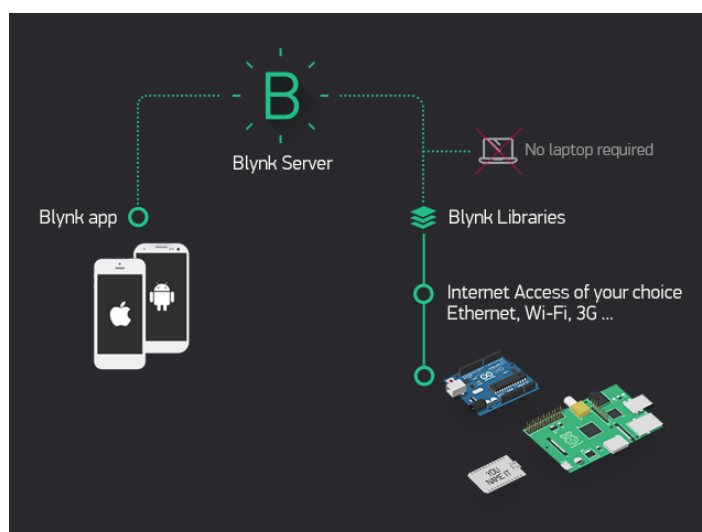


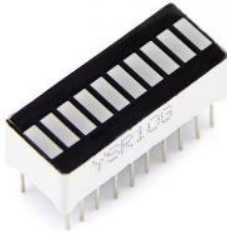
Figura 3.8: Arquitectura funcional del prototipo
Fuente: (Blynk.cc)

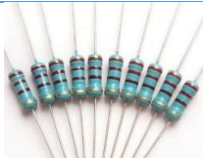
Diseño de Arquitectura Hardware Electrónico

En la tabla 3.7 se muestran los componentes de hardware necesarios que se utilizarán para la integración del circuito electrónico:

Tabla 3.7: Componentes del Circuito

Fuente: (Elaboración propia)

Componente.	Cantidad.	Gráfico.
ESP32	1	
Cable Micro USB	1	
Display LCD 16x2 1602 Azul	1	
Display 10 Segmentos	1	
Sensor de Flujo ½" YF-S201	1	

Sensor de Voltaje 25V DC FZ0430	1	
Módulo cargador de Li-Ion 18650	1	
Módulo MT3608 DC-DC Step Up	1	
Interruptor Momentáneo INOX 16mm	1	
Interruptor de Palanca MTS- 102	1	
Conector Plug Hembra DC con Terminal	1	
Conector Plug Macho DC 2.1mm	1	
Resistencias de 220 ohm	10	
Resistencia 1k	1	

Una vez definidos los componentes, se procede al diseño del circuito como se muestra en la figura 3.9.

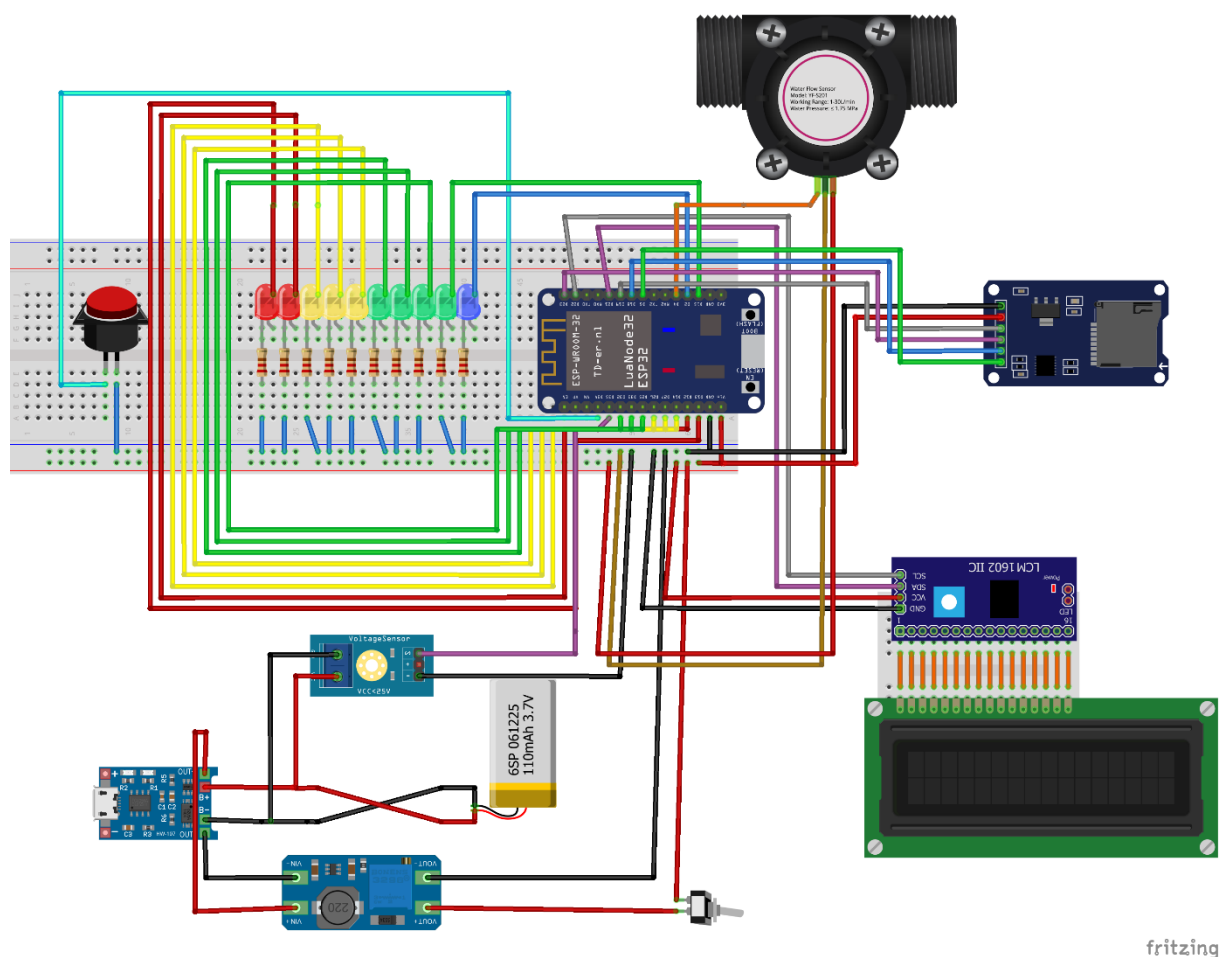


Figura 3.9: Diseño del circuito
Fuente: (Elaboración propia)

Como se puede apreciar el circuito estará alimentado por una fuente de voltaje de 3.7V que suministra la Batería Mini Li Po, el voltaje de 3.7V se utilizará para alimentar el Módulo elevador de Voltaje, el cual suministrará con 5V nuestra placa ESP32, el Display LCD 16x2 y el Sensor de Flujo YF-S201, el Display de 10 segmentos estará conectado a los pines GPIO D13, D12, D14, D27, D26, D25, D33, D32, D15 y D2 de nuestro ESP32, esto para expresar el nivel de carga de nuestra Batería Li Po.

El sensor de Flujo YF-S20, consta de tres pines, alimentación 5V, señal y GND o conexión a tierra, el sensor enviará los datos de lectura al GPIO18.

El sensor de Voltaje FZ 0430, consta de 3 pines, alimentación 5V, señal y GND o conexión a tierra, el sensor enviará la señal analógica de lectura al GPIO35.

Para la programación del Microcontrolador se utilizará el IDE PlatformIO.

Diseño de la Arquitectura de la Interfaz Blynk

Las herramientas que se utilizarán para el desarrollo de la Interfaz Blynk se describen en la tabla 3.8

Tabla 3.8: Software para el la Interfaz Blynk

Fuente: (Elaboración propia)

Componente	Descripción
Blynk CC	Es una Aplicación Móvil que nos permite crear interfaces para el control y monitoreo de diferentes dispositivos IoT, trabaja bajo plataformas Android y IOS

Para el diseño de la Interfaz Blynk se tendrán recursos indispensables que son, ver figura 3.10

- Widgets
- Container

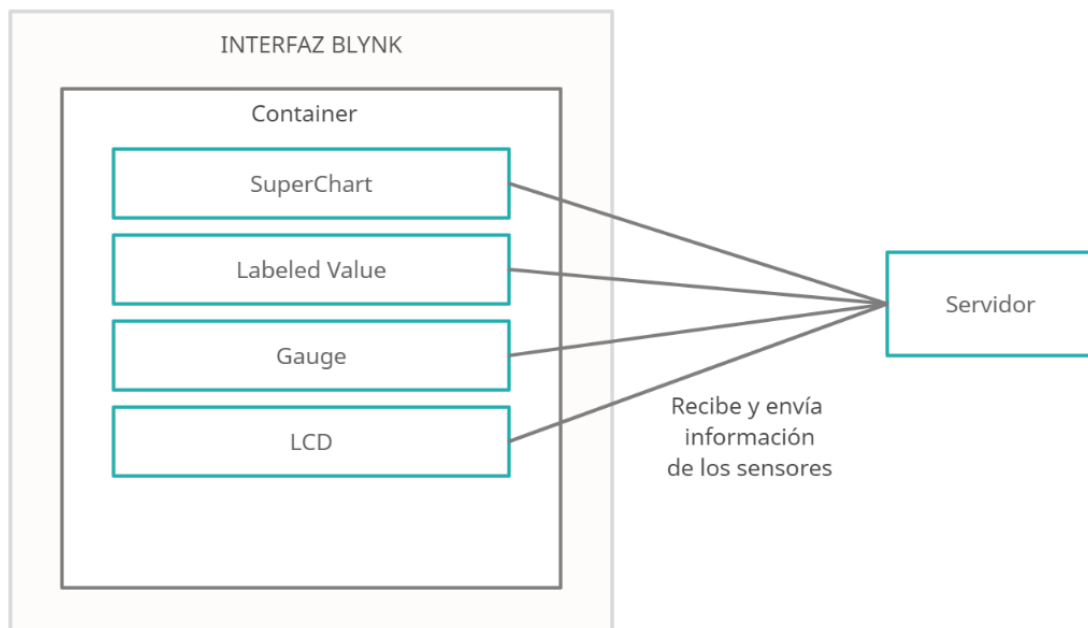


Figura 3.10: Diseño Arquitectura Blynk

Fuente: (Elaboración propia)

Widgets responden a los diferentes componentes que nos permitirán observar en tiempo real los datos procesados por el Hardware Electrónico.

Container contiene los Widgets de la Interfaz Blynk, existe un único container principal en el cual también se guardan las variables de conexión hacia nuestro hardware electrónico.

3.4.4 DISEÑO DE MODULO

En esta etapa se detallará el funcionamiento del sistema a través de algoritmos.

Algoritmo de programación del microcontrolador

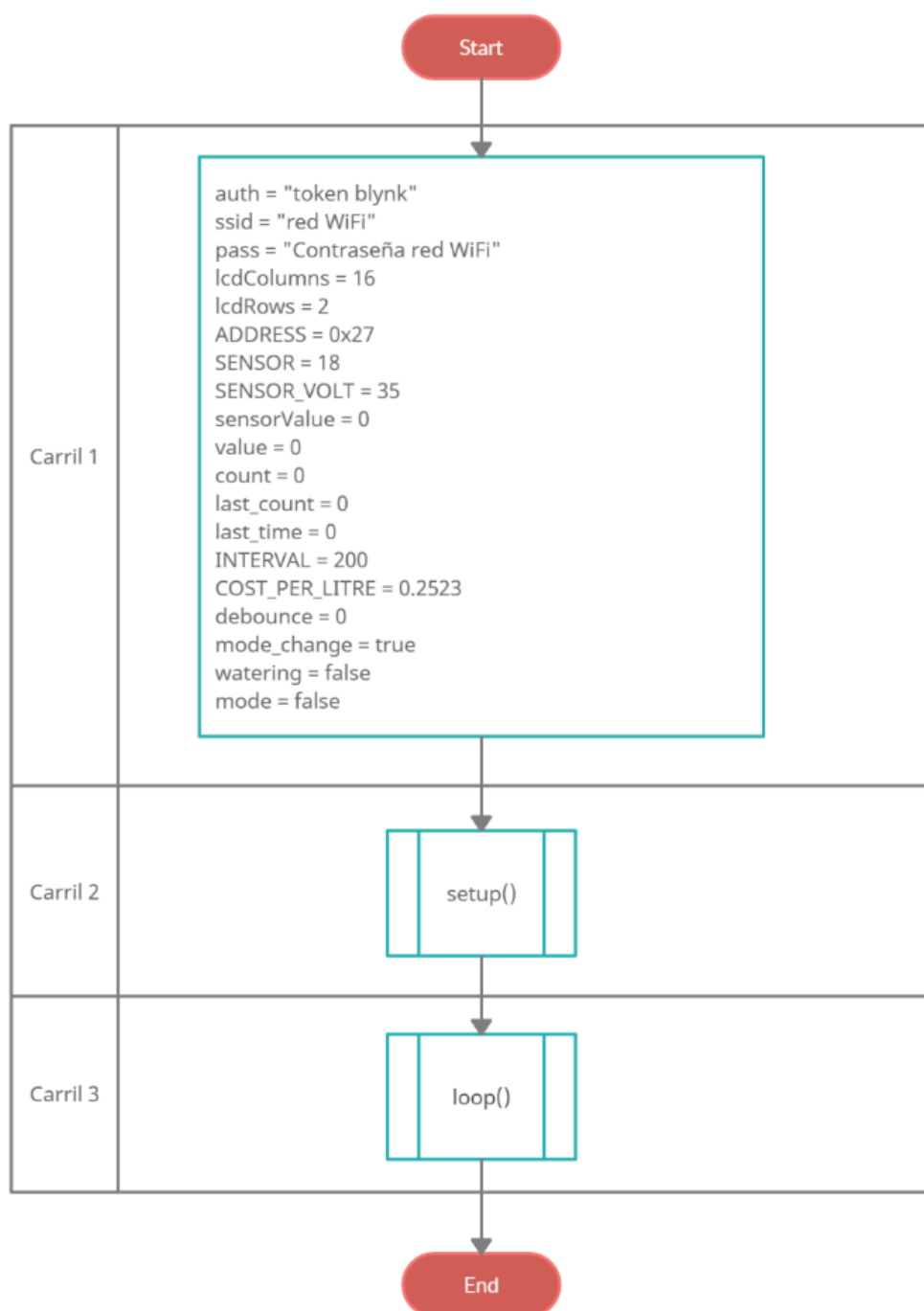


Figura 3.11: Algoritmo Principal del Microcontrolador
Fuente: (Elaboración propia)

Carril 1. Se declaran las variables Globales necesarias para el funcionamiento del prototipo.

Carril 2. Setup(), realiza la configuración necesaria para el funcionamiento correcto de nuestra Placa ESP32

Carril 3. Loop(), realiza los procesos necesarios para el funcionamiento del prototipo.

En las figuras 3.12, y 3.13 se mostrará el algoritmo de las funciones setup() y loop().

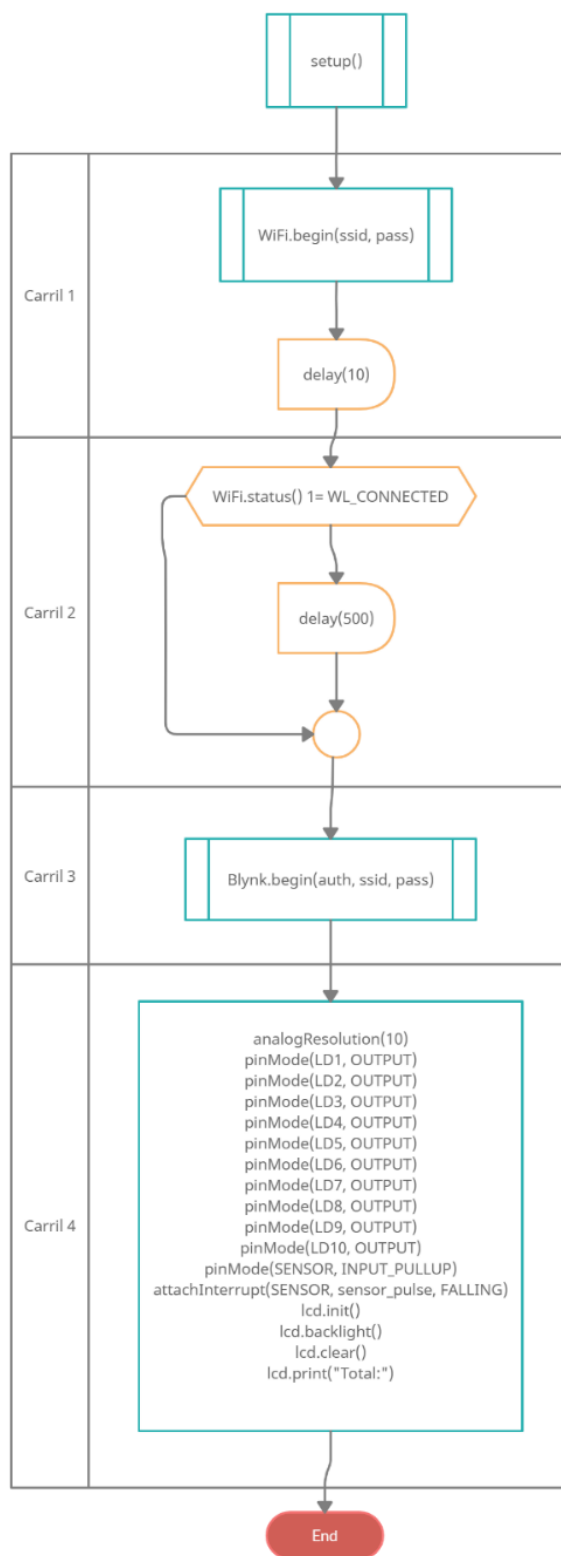


Figura 3.12: setup
Fuente: (Elaboración propia)

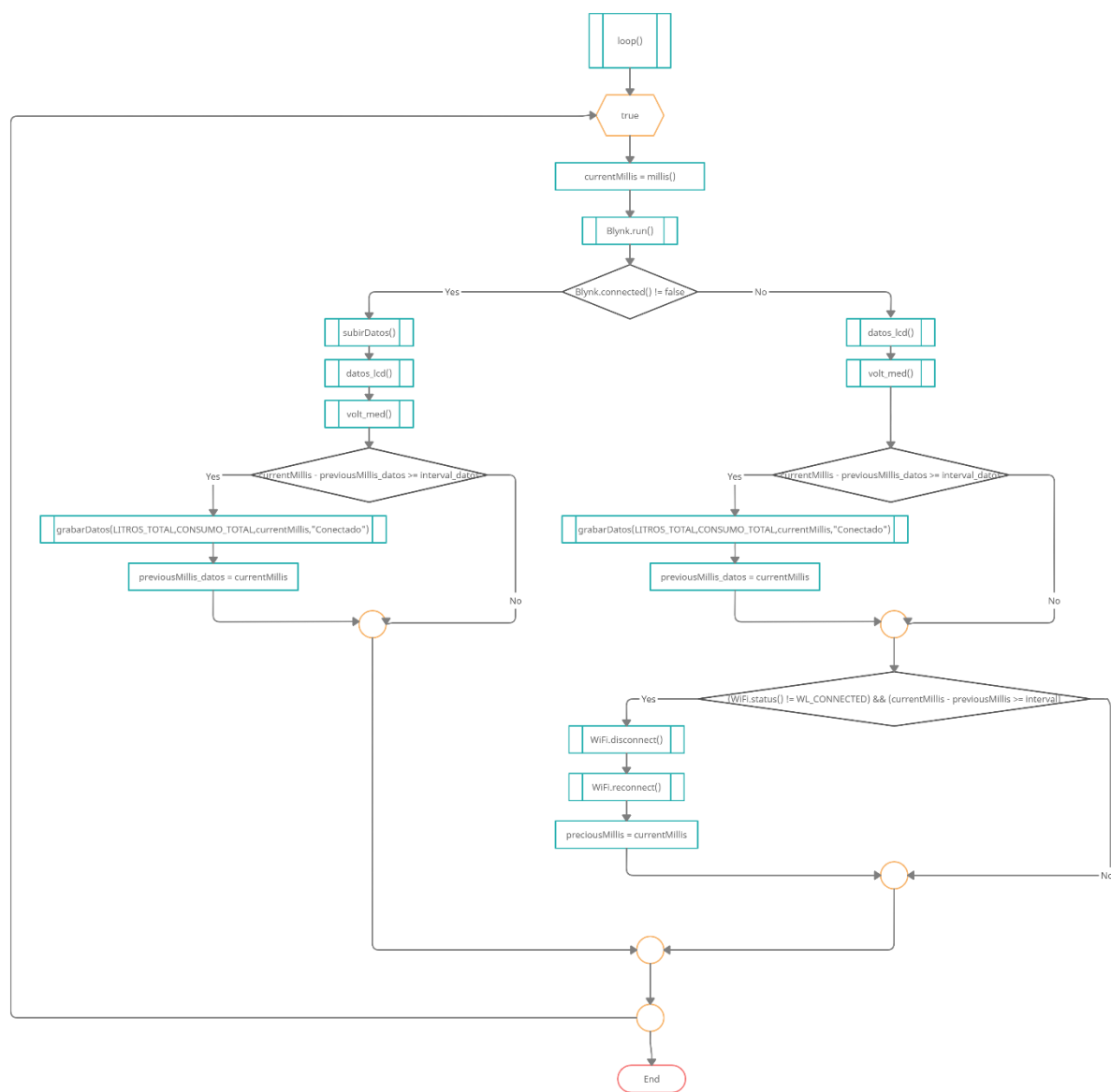


Figura 3.13: loop
Fuente: (Elaboración propia)

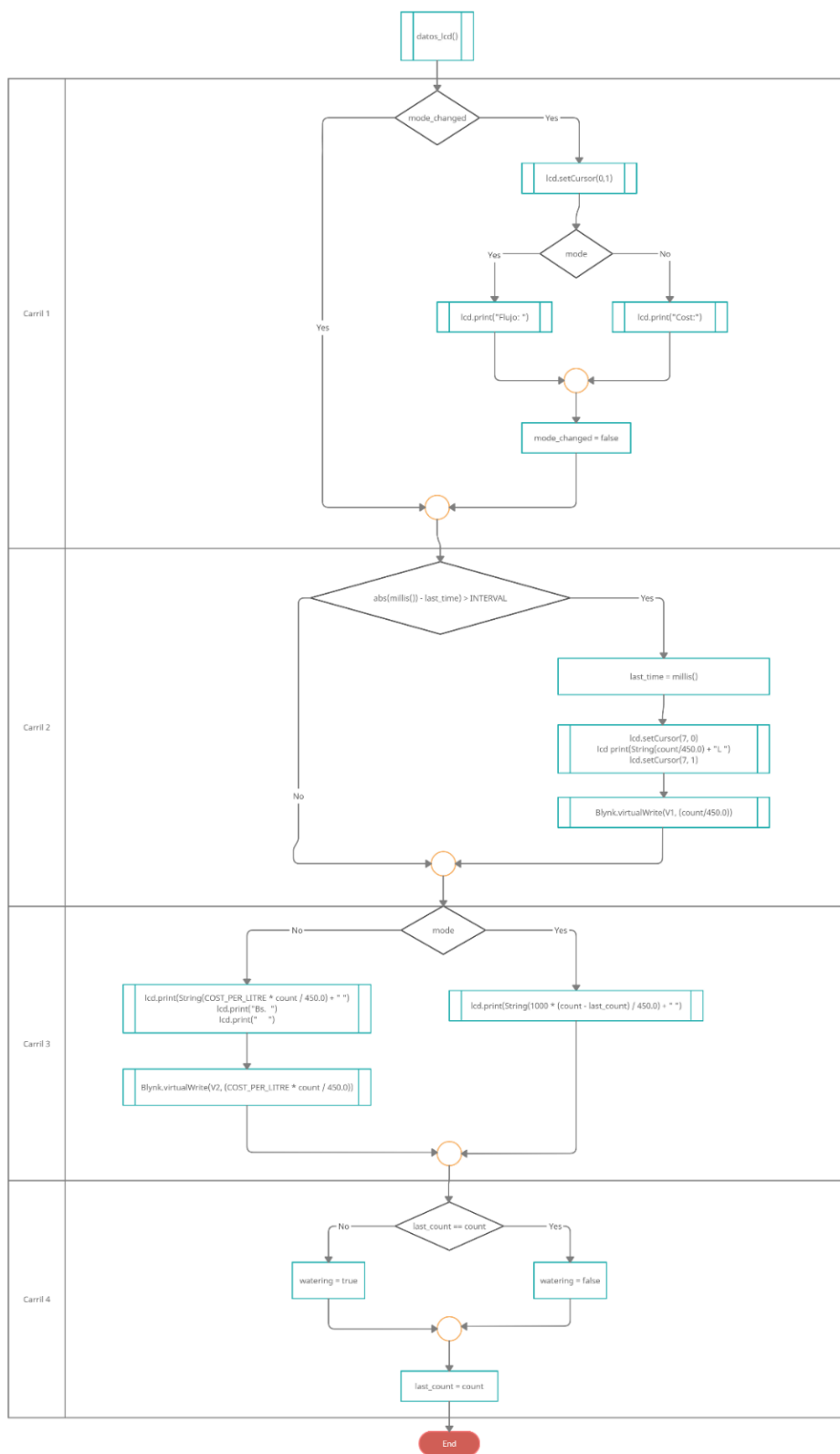


Figura 3.14: datos_lcd
Fuente: (Elaboración propia)

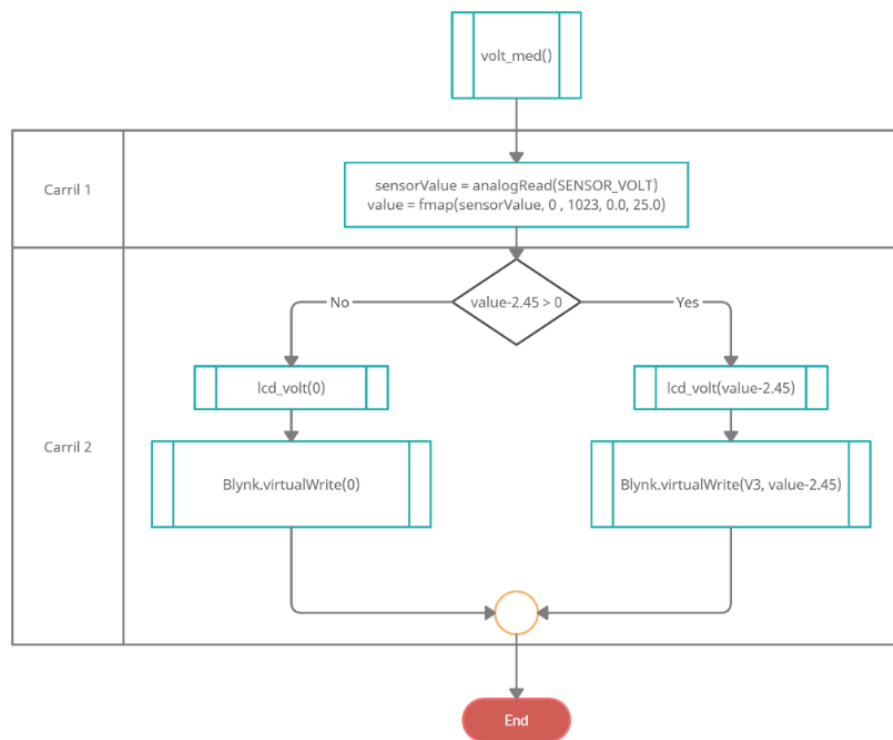


Figura 3.15: `volt_med`
Fuente: (Elaboración propia)

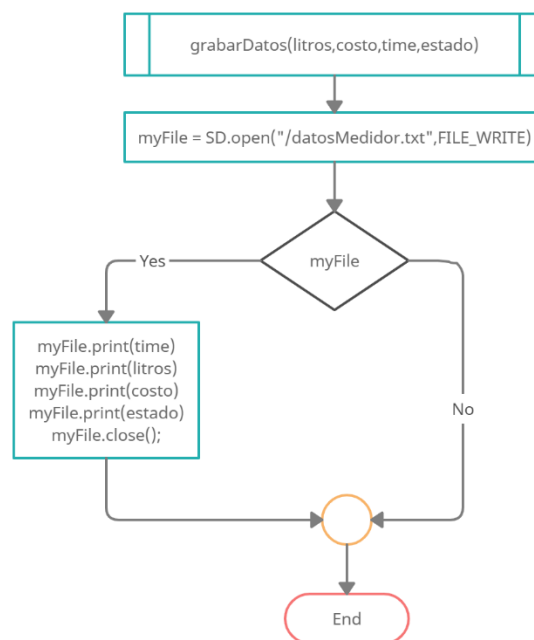


Figura 3.16: `grabarDatos`
Fuente: (Elaboración propia)

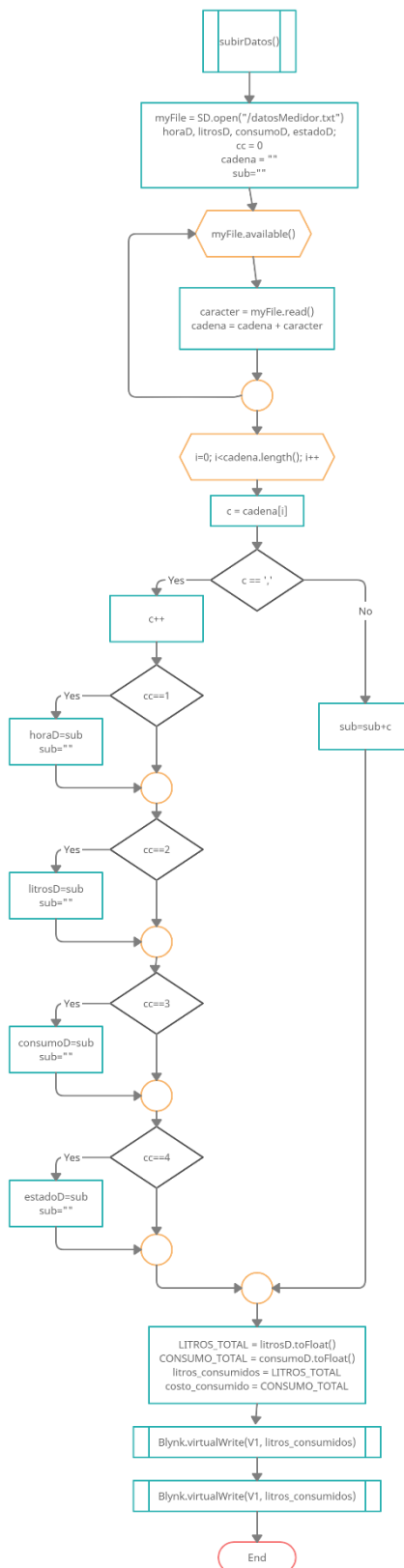


Figura 3.17: subirDatos
Fuente: (Elaboración propia)

3.4.5 CODIFICACIÓN

En esta fase se lleva a cabo la programación del Hardware Electrónico aplicando los componentes de software y hardware mencionados anteriormente y materializando la funcionalidad que se describió en los diagramas anteriores, para lo que se elabora un plan de trabajo en términos de los requerimientos, que consta de los siguientes pasos:

- Implementación del circuito.
- Implementación de la Interfaz Blynk.

Circuito Electrónico

El circuito se armará de acuerdo con los componentes descritos en la tabla 3.7 y de acuerdo con el diseño de la figura 3.8, en la figura 3.18 se puede apreciar el resultado.

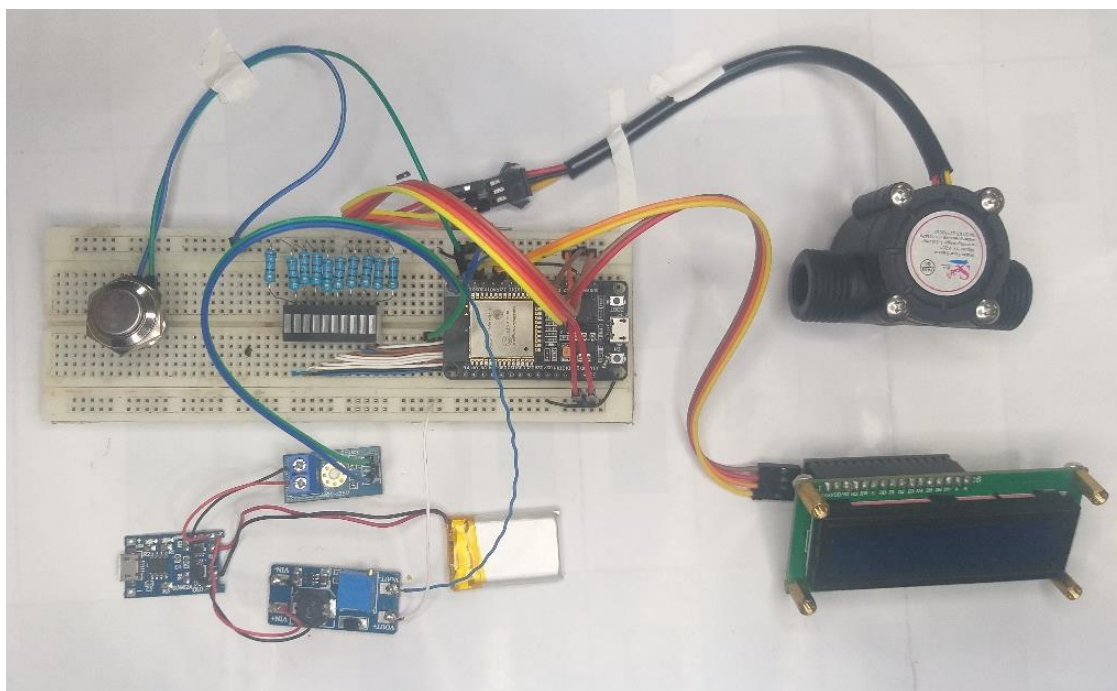


Figura 3.18: Implementación del circuito
Fuente: (Elaboración propia)

Para la codificación del microcontrolador primeramente se incluirán las siguientes librerías como se muestra en la figura 3.19:

```
src > main.cpp > ...
1  <#include <Arduino.h>
2  <#include <LiquidCrystal_I2C.h>
3  <#include <WiFi.h>
4  <#include <WiFiClient.h>
5  <#include <BlynkSimpleEsp32.h>
```

Figura 3.19: Inclusión de Librerías

Fuente: (Elaboración propia)

Se inicializan las variables globales para el funcionamiento del prototipo como se puede ver en las figuras 3.20 y 3.21:

```
src > main.cpp > ...
7  //VARIABLES DE CONEXIÓN
8  char auth[] = "rKv8p1TXvTAuTtCA1wWE5_RBS1aCVUQg"; //Colocar aquí el token que fue enviado a su mail.
9  char ssid[] = "ZTE-ec47d0"; //Red WiFi personal
10 char pass[] = "78312bec"; // Contraseña para WiFi personal
11
12 int lcdColumns = 16;
13 int lcdRows = 2;
14 const byte ADDRESS = 0x27; // dirección I2C para el LCD (0x3F & 0x27 regularmente)
15 LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);
16
17 const byte SENSOR = 18; // sensor en GPIO18
18 const byte BUTTON = 23; // boton en GPIO23
19
20 #define LD1 13
21 #define LD2 12
22 #define LD3 14
23 #define LD4 27
24 #define LD5 26
25 #define LD6 25
26 #define LD7 33
27 #define LD8 32
28 #define LD9 15
29 #define LD10 2
```

Figura 3.20: Inicialización de Variables Globales

Fuente: (Elaboración propia)

```

src > main.cpp > ...
30
31 #define SENSOR_VOLT 35
32 int sensorValue;      // variable que almacena el valor raw (0 a 1023)
33 float value;          // variable que almacena el voltaje (0.0 a 25.0)
34
35 unsigned long count = 0;    // número de pulsos del sensor de flujo (un pulso es 1/450 de litro)
36 unsigned long last_count = 0; // temporizador para rastrear el último pulso del sensor
37 unsigned long last_time = 0; // temporizador para rastrear la última actualización de LCD
38 const int INTERVAL = 200;   // frecuencia de actualización para la lectura de la pantalla
39 const float COST_PER_LITRE = 0.2523; // costo por litro, en centavos//
40 unsigned long debounce = 0;  // temporizador para botón antirrebote
41
42 bool mode_changed = true; // una bandera para actualizar la etiqueta de texto "Tarifa:" o "Costo:"
43 bool watering = false;    // una bandera para indicar que el flujo de agua está activo
44 bool mode = false;        // mostrar el caudal si es verdadero, o el costo si es falso
45

```

Figura 3.21: Inicialización de Variables Globales

Fuente: (Elaboración propia)

A continuación se implementan las funciones necesarias para el calculo del Nivel de Batería, la función `lcd_volt()` implementa la lógica necesaria para enviar señales digitales hacia nuestro display de 10 segmentos, mostrando el nivel de batería restante como se puede ver en las figuras 3.22, 3.23, 3.24, 3.25, 3.26 y 3.27.

```
src > main.cpp > ...  
46  ///////////////////////////////////////////////////////////////////  
47  ///////////////////////////////////////////////////////////////////FUNCIONES PARA EL MÓDULO DE CONTR  
48  ///////////////////////////////////////////////////////////////////  
49  void lcd_volt(float valueX){  
50      int valorVolt = map(valueX, 0.0, 4.50, 0, 10);  
51  
52      switch (valorVolt)  
53      {  
54          case 1:  
55              digitalWrite(LD1, HIGH);  
56              digitalWrite(LD2, LOW);  
57              digitalWrite(LD3, LOW);  
58              digitalWrite(LD4, LOW);  
59              digitalWrite(LD5, LOW);  
60              digitalWrite(LD6, LOW);  
61              digitalWrite(LD7, LOW);  
62              digitalWrite(LD8, LOW);  
63              digitalWrite(LD9, LOW);  
64              digitalWrite(LD10, LOW);  
65              break;  
66  
67          case 2:  
68              digitalWrite(LD1, HIGH);  
69              digitalWrite(LD2, HIGH);  
70              digitalWrite(LD3, LOW);  
71              digitalWrite(LD4, LOW);  
72              digitalWrite(LD5, LOW);  
73              digitalWrite(LD6, LOW);  
74              digitalWrite(LD7, LOW);  
75              digitalWrite(LD8, LOW);  
76              digitalWrite(LD9, LOW);  
77              digitalWrite(LD10, LOW);  
78              break;
```

Figura 3.22: Función lcd_volt

Fuente: (Elaboración propia)

```

src > main.cpp > ...
80     case 3:
81         digitalWrite(LD1, HIGH);
82         digitalWrite(LD2, HIGH);
83         digitalWrite(LD3, HIGH);
84         digitalWrite(LD4, LOW);
85         digitalWrite(LD5, LOW);
86         digitalWrite(LD6, LOW);
87         digitalWrite(LD7, LOW);
88         digitalWrite(LD8, LOW);
89         digitalWrite(LD9, LOW);
90         digitalWrite(LD10, LOW);
91         break;
92
93     case 4:
94         digitalWrite(LD1, HIGH);
95         digitalWrite(LD2, HIGH);
96         digitalWrite(LD3, HIGH);
97         digitalWrite(LD4, HIGH);
98         digitalWrite(LD5, LOW);
99         digitalWrite(LD6, LOW);
100        digitalWrite(LD7, LOW);
101        digitalWrite(LD8, LOW);
102        digitalWrite(LD9, LOW);
103        digitalWrite(LD10, LOW);
104        break;

```

Figura 3.23: Función lcd_volt

Fuente: (Elaboración propia)

```

src > main.cpp > ...
106    case 5:
107        digitalWrite(LD1, HIGH);
108        digitalWrite(LD2, HIGH);
109        digitalWrite(LD3, HIGH);
110        digitalWrite(LD4, HIGH);
111        digitalWrite(LD5, HIGH);
112        digitalWrite(LD6, LOW);
113        digitalWrite(LD7, LOW);
114        digitalWrite(LD8, LOW);
115        digitalWrite(LD9, LOW);
116        digitalWrite(LD10, LOW);
117        break;
118
119    case 6:
120        digitalWrite(LD1, HIGH);
121        digitalWrite(LD2, HIGH);
122        digitalWrite(LD3, HIGH);
123        digitalWrite(LD4, HIGH);
124        digitalWrite(LD5, HIGH);
125        digitalWrite(LD6, HIGH);
126        digitalWrite(LD7, LOW);
127        digitalWrite(LD8, LOW);
128        digitalWrite(LD9, LOW);
129        digitalWrite(LD10, LOW);
130        break;

```

Figura 3.24: Función lcd_volt

Fuente: (Elaboración propia)

```

src > main.cpp > ...
132     case 7:
133         digitalWrite(LD1, HIGH);
134         digitalWrite(LD2, HIGH);
135         digitalWrite(LD3, HIGH);
136         digitalWrite(LD4, HIGH);
137         digitalWrite(LD5, HIGH);
138         digitalWrite(LD6, HIGH);
139         digitalWrite(LD7, HIGH);
140         digitalWrite(LD8, LOW);
141         digitalWrite(LD9, LOW);
142         digitalWrite(LD10, LOW);
143         break;
144
145     case 8:
146         digitalWrite(LD1, HIGH);
147         digitalWrite(LD2, HIGH);
148         digitalWrite(LD3, HIGH);
149         digitalWrite(LD4, HIGH);
150         digitalWrite(LD5, HIGH);
151         digitalWrite(LD6, HIGH);
152         digitalWrite(LD7, HIGH);
153         digitalWrite(LD8, HIGH);
154         digitalWrite(LD9, LOW);
155         digitalWrite(LD10, LOW);
156         break;

```

Figura 3.25: Función lcd_volt

Fuente: (Elaboración propia)

```

src > main.cpp > ...
158     case 9:
159         digitalWrite(LD1, HIGH);
160         digitalWrite(LD2, HIGH);
161         digitalWrite(LD3, HIGH);
162         digitalWrite(LD4, HIGH);
163         digitalWrite(LD5, HIGH);
164         digitalWrite(LD6, HIGH);
165         digitalWrite(LD7, HIGH);
166         digitalWrite(LD8, HIGH);
167         digitalWrite(LD9, HIGH);
168         digitalWrite(LD10, LOW);
169         break;
170
171     case 10:
172         digitalWrite(LD1, HIGH);
173         digitalWrite(LD2, HIGH);
174         digitalWrite(LD3, HIGH);
175         digitalWrite(LD4, HIGH);
176         digitalWrite(LD5, HIGH);
177         digitalWrite(LD6, HIGH);
178         digitalWrite(LD7, HIGH);
179         digitalWrite(LD8, HIGH);
180         digitalWrite(LD9, HIGH);
181         digitalWrite(LD10, HIGH);
182         break;

```

Figura 3.26: Función lcd_volt

Fuente: (Elaboración propia)

```
src > main.cpp > ...  
183  
184     default:  
185         digitalWrite(LD1, HIGH);  
186         digitalWrite(LD2, LOW);  
187         digitalWrite(LD3, LOW);  
188         digitalWrite(LD4, LOW);  
189         digitalWrite(LD5, LOW);  
190         digitalWrite(LD6, LOW);  
191         digitalWrite(LD7, LOW);  
192         digitalWrite(LD8, LOW);  
193         digitalWrite(LD9, LOW);  
194         digitalWrite(LD10, LOW);  
195         break;  
196     }  
197 }
```

Figura 3.27: Función lcd_volt

Fuente: (Elaboración propia)

Se implementan las funciones `fmap` y `volt_med`, `fmap` nos permite mapear números de tipo `Float`, dicha función nos permitirá mapear la señal analógica a un nivel de voltaje manipulable como se puede ver en la figura 3.28.

`Volt_med` realiza la lectura del pin `GPIO35` configurado como entrada Analógica, para posteriormente realizar el mapeo con la función `fmap` y una corrección a la lectura, si la lectura es mayor a cero se envían los datos a la función `lcd_volt` y al `virtualWrite` de Blynk, caso contrario envían datos nulos, como se puede apreciar en la figura 3.28

```
src > main.cpp > ...
199 // cambio de escala entre floats
200 float fmap(float x, float in_min, float in_max, float out_min, float out_max)
201 {
202     return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
203 }
204
205 void volt_med(){
206     sensorValue = analogRead(SENSOR_VOLT); // realizar la lectura
207     value = fmap(sensorValue, 0, 1023, 0.0, 25.0); // cambiar escala a 0.0 - 25.0 para nuestro caso 3.7
208     if(value-2.45 > 0){
209         lcd_volt(value-2.45);
210         Blynk.virtualWrite(V3, value-2.45);
211     }
212     else{
213         lcd_volt(0);
214         Blynk.virtualWrite(0);
215     }
216 }
```

Figura 3.28: Función `fmap` y `volt_med`

Fuente: (Elaboración propia)

La función `datos_lcd` se encarga de la gestión de la Pantalla LCD 16x2 así como el tratamientos de datos antes de ser mostrados en la pantalla, al mismo tiempo estos datos procesados son enviados a la Interfaz Blynk como se puede apreciar en la figura 3.29

```
src > main.cpp > datos_lcd()
238 void datos_lcd(){
239     if (mode_changed) { // si el modo ha cambiado, imprima la etiqueta de texto actualizada
240         lcd.setCursor(0, 1); // mover el puntero a segunda línea
241         if (mode)
242             lcd.print("Flujo:      "); // el espacio en blanco es para borrar cualquier carácter extraño
243         else
244             lcd.print("Cost:      "); // el espacio en blanco es para borrar cualquier carácter extraño
245         mode_changed = false; // limpiar bandera
246     }
247     if (abs(millis() - last_time) > INTERVAL) { // ha pasado más de INTERVAL-time, así que actualice la pantalla
248         last_time = millis(); // establecer la última hora de actualización a la hora actual
249         lcd.setCursor(7, 0);
250         lcd.print(String(count / 450.0) + " L "); // calcular e imprimir litros totales
251         Blynk.virtualWrite(V1, (count / 450.0));
252         lcd.setCursor(7, 1);
253         if (mode) { // modo de caudal, muestra el caudal actual
254             lcd.print(String(1000 * (count - last_count) / 450.0 / INTERVAL) + (" L/s  "));
255         }
256         else { // modo de costo, muestra el costo total
257             lcd.print(String(COST_PER_LITRE * count / 450.0) + " ");
258             Blynk.virtualWrite(V2, (COST_PER_LITRE * count / 450.0));
259             lcd.print("Bs.");
260             lcd.print(" "); // imprimir espacios finales para borrar caracteres extraños
261         }
262         if (last_count == count) // así es como determinamos si el flujo está funcionando(paso de agua)
263             watering = false; // caudal off
264         else
265             watering = true; // caudal on
266
267         last_count = count;
268     }
269 }
```

Figura 3.29: Función `datos_lcd`

Fuente: (Elaboración propia)

A continuación, se implementó la función `setup`, encargada de realizar las configuraciones de pines de salida y entrada, así como configurar la conexión WiFi y Blynk, también se configuran el pin para interrupciones y se inicializa la pantalla LCD, como se puede apreciar en la figura 3.30

```
src > main.cpp > setup()
271 void setup() {
272   Serial.begin(9600);
273   delay(10);
274   Serial.println(ssid);
275   WiFi.begin(ssid, pass);
276   int wifi_ctr = 0;
277   while(WiFi.status() != WL_CONNECTED){
278     delay(500);
279     Serial.print(".");
280   }
281   Serial.println("WiFi Conectado");
282   Blynk.begin(auth, ssid, pass);
283   analogReadResolution(10);  ///CONFIGURACIÓN DE PINES PARA EL MÓDULO DE CONTROL DE NIVEL DE BATERÍA///
284   pinMode(LD1, OUTPUT);
285   pinMode(LD2, OUTPUT);
286   pinMode(LD3, OUTPUT);
287   pinMode(LD4, OUTPUT);
288   pinMode(LD5, OUTPUT);
289   pinMode(LD6, OUTPUT);
290   pinMode(LD7, OUTPUT);
291   pinMode(LD8, OUTPUT);
292   pinMode(LD9, OUTPUT);
293   pinMode(LD10, OUTPUT);
294   lcd.init();  // Inicializar LCD
295   lcd.backlight();
296   lcd.clear();
297   lcd.print("---MEDIDOR IOT---");
298   delay(500);
299   pinMode(SENSOR, INPUT_PULLUP);  // Configure las funciones de los pines y habilite las resistencias pu
300   pinMode(BUTTON, INPUT_PULLUP);
301   attachInterrupt(SENSOR, sensor_pulse, FALLING);  // Inicializar interrupciones
302   attachInterrupt(digitalPinToInterrupt(BUTTON), button_press, FALLING);
303 }
```

Figura 3.30: Función `setup`
Fuente: (Elaboración propia)

Se implementa `grabarDatos`, el cual nos permite abrir y cerrar el archivo `datosMedidor.txt`, en el cual se guardan serapados por comas, datos relevantes como son los litros consumidos y el costo por consumo, como se puede apreciar en la figura 3.31.

```
void grabarDatos(float litros, float costo, unsigned long time, String estado) {  
    myFile = SD.open("/datosMedidor.txt", FILE_WRITE); //abrimos el archivo  
    if (myFile) {  
  
        myFile.print(time);  
        myFile.print(",");  
        myFile.print(litros);  
        myFile.print(",");  
        myFile.print(costo);  
        myFile.print(",");  
        myFile.print(estado);  
        myFile.print(",");  
  
        myFile.close(); //cerramos el archivo  
    }  
}
```

Figura 3.31: `grabarDatos`
Fuente: (Elaboración propia)

subirDatos, es la encargada de leer los datos almacenados de la tarjeta SD, para posteriormente enviarlos a Blynk, como se aprecia en la figura 3.32.

```

395 void subirDatos(){
396     myFile = SD.open("/datosMedidor.txt"); //abrimos el archivo
397     String horaD;
398     String litrosD;
399     String consumoD;
400     String estadoD;
401     int cc= 0;
402     String cadena = "";
403     while(myFile.available()) {
404         char caracter = myFile.read();
405         cadena = cadena + caracter;
406     }
407     myFile.close();
408     String sub = "";
409     for (int i = 0; i < cadena.length(); i++)
410     {
411         char c = cadena[i];
412
413         if (c == ','){
414             cc++;
415             if(cc==1){
416                 horaD=sub;
417                 sub="";
418             }
419             if(cc==2){
420                 litrosD=sub;
421                 sub="";
422             }
423             if(cc==3){
424                 consumoD=sub;
425                 sub="";
426             }
427             if(cc==4){
428                 estadoD=sub;
429                 sub="";
430             }
431             } else {
432                 sub=sub+c;
433             }
434     }
435     LITROS_TOTAL = litrosD.toFloat();
436     CONSUMO_TOTAL = consumoD.toFloat();
437     litros_consumidos = LITROS_TOTAL;
438     costo_consumido = CONSUMO_TOTAL;
439     Blynk.virtualWrite(V1, litros_consumidos);
440     Blynk.virtualWrite(V2, costo_consumido);
441 }

```

Figura 3.32: subirDatos

Fuente: (Elaboración propia)

Por último, se implementa la función `loop`, que es un bucle infinito que invoca las funciones principales mencionadas anteriormente, a su vez verifica que la conexión con el servidor Blynk sea conectada, en caso de estar conectado, llama a `subirDatos` para obtener los datos almacenado en la tarjeta SD, llama a las funciones `datos_lcd` y `volt_med` para actualizar la información, en caso de estar desconectado solo se llama a las funciones `datos_lcd` y `volt_med` para mantener actualizada la información a pesar de no tener una conexión estable, posterior a ellos se realiza la reconexión con la red WiFi con intervalos de 10 segundos hasta lograr la reconexión.

En ambos casos mencionados, la información actualizada del consumo se almacena en la Tarjeta SD en un intervalo de tiempo de 10 segundos, como puede apreciarse en la figura 3.33

```

469 void loop() {
470     unsigned long currentMillis = millis();
471     Blynk.run();
472     if (Blynk.connected() != false){
473         subirDatos();
474         datos_lcd();
475         volt_med();
476         if (currentMillis - previousMillis_datos >= interval_datos){
477             grabarDatos(LITROS_TOTAL, CONSUMO_TOTAL, currentMillis, "Conectado");
478             previousMillis_datos = currentMillis;
479         }
480     } else {
481         datos_lcd();
482         volt_med();
483         if (currentMillis - previousMillis_datos >= interval_datos){
484             grabarDatos(LITROS_TOTAL, CONSUMO_TOTAL, currentMillis, "Desconectado");
485             previousMillis_datos = currentMillis;
486         }
487         if ((WiFi.status() != WL_CONNECTED) && (currentMillis - previousMillis >= interval)){
488             WiFi.disconnect();
489             WiFi.reconnect();
490             previousMillis = currentMillis;
491         }
492     }
493 }

```

Figura 3.33: Función `loop`
Fuente: (Elaboración propia)

3.4.6 PRUEBAS

Prueba de la Interfaz Blynk

Se empezará en con la implementación de la interfaz Blynk, mostrando los componentes de manera secuencial.

Cuando ingresamos a nuestra aplicación Blynk observamos los componentes que previamente habíamos añadido a nuestro panel de trabajo, como se puede observar en la imagen 3.34

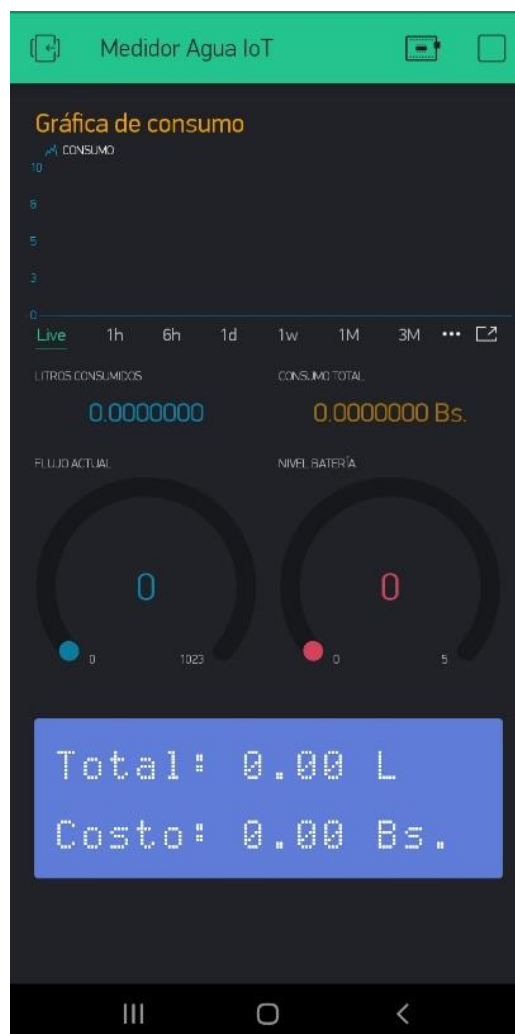


Figura 3.34: Interfaz Blynk Inicial
Fuente: (Elaboración propia)

Una vez ingresado a nuestra aplicación blynk realizamos la prueba de conectarnos a nuestro hardware electrónico.

Si la conexión esta correcta podremos ver la siguiente pantalla con los datos iniciales de lectura que nos llegan desde nuestro Hardware Electrónico, como se puede observar en la figura 3.35.



Figura 3.35: Interfaz Blynk Conectado
Fuente: (Elaboración propia)

Nuestro primer componente denominado Gráfica de consumo, nos muestra en tiempo real el flujo actual del agua potable, teniendo la opción de poder ver el consumo en el pasar del tiempo, como se aprecia en la figura 3.35

A continuación, tenemos la representación en litros del flujo actual del agua así como su equivalente en consumo total en bolivianos, como se aprecia en la figura 3. 35

Podemos ver una gráfica del flujo actual, el cual muestra la candencia con la que el flujo es usado, también podemos apreciar el nivel de batería que aun le resta a nuestro dispositivo, como se puede ver en la figura 3. 35

Por último, tenemos una representación de la pantalla LCD que tenemos en nuestro Hardware para ver en tiempo real los mismo datos que se aprecian en la pantalla del dispositivo. Ver la figura 3. 35

Prueba del Prototipo

Una vez con el circuito armado y probado, bastará con integrarlo al case impreso en 3D el cual fue diseñado para tal propósito

A continuación, en las figuras 3.36, 3.37 y 3.38, se muestra el prototipo ya armado con valores de prueba, también diferentes perspectivas de su interior y ubicaciones de los distintos componentes.



Figura 3.36: Prototipo
Fuente: (Elaboración propia)



Figura 3.37: Prototipo
Fuente: (Elaboración propia)



Figura 3.38: Prototipo
Fuente: (Elaboración propia)



Figura 3.35: Prototipo Instalado y en funcionamiento
Fuente: (Elaboración propia)

CAPITULO IV

4. PRUEBA DE HIPOTESIS

4.1 INTRODUCCIÓN

La prueba de hipótesis es una suposición, la cual en este capítulo verificaremos, cabe destacar que la investigación gira alrededor de un diseño cualitativo, por lo que no se trata de probar o de medir en qué grado una cierta cualidad se encuentra en un cierto acontecimiento dado, sino de descubrir tantas cualidades como sea posible. La investigación cualitativa requiere un profundo entendimiento del comportamiento humano y las razones que lo gobiernan, en otras palabras, investiga por qué y el cómo se tomó una decisión.

La investigación cualitativa se basa en la observación de muestras pequeñas, esto es la observación de grupos de población reducidos.

En el presente capítulo trataremos de descubrir tantas cualidades como sea posible bajo nuestro planteamiento de hipótesis.

“El uso de un Prototipo de Medidor de Agua IoT, nos permitirá obtener datos del consumo de agua potable en hogares de la ciudad de La Paz”.

4.2 CALIDAD DE LAS ACTIVIDADES Y CASOS ENCONTRADOS

Se determinan 4 casos de prueba, para comprobar el funcionamiento del entorno Blynk junto con el Prototipo de medidor de Agua.

- **Caso 1:** Calculo de los Litros consumidos de acuerdo con el sensor de Flujo de Agua.
- **Caso 2:** Calculo del costo total del consumo de agua.
- **Caso 3:** Condiciones de operación.
- **Caso 4:** Guardar información del consumo de Agua cuando no exista conexión a Internet.

Para las pruebas se utilizó un Smartphone Android Samsung J8 2018 con SO Android 10.0, 4GB de RAM. Para las pruebas de Internet se utilizó un Router WiFi TP Link TL WR841HP, con el servicio de Internet de la Empresa Entel de 15MB de velocidad.

4.3 CASO 1.

El cálculo de los litros consumidos es un proceso que trabaja de la mano con las interrupciones en el microcontrolador como su calibración y posterior procesamiento.

Después de varias pruebas y consultando la documentación correspondiente sobre el sensor de Flujo se obtiene la siguiente formula que se puede ver en la figura 4.1

$$\text{litrosConsumidos} = \frac{\text{contadorDeInterrupciones}}{450}$$

Figura 4.1: Formula para el cálculo de Litros consumidos
Fuente: (Elaboración propia)

Teniendo un margen de error de +/- 2%.

El punto 5.5 Clase de exactitud y error máximo permisible del reglamento técnico de medidores domiciliarios elaborado por el ministerio de desarrollo productivo y economía plural, nos indica que los medidores estarán clasificados de acuerdo con su exactitud, como clase de Exactitud 1 o Clase de Exactitud 2.

Una vez obtenido nuestro margen de error, nuestro prototipo de medidor IoT entra dentro de los Medidores con Clase de Exactitud 2, cumpliendo la siguiente premisa:

- El EMP para la zona superior del rango del Flujo es $\pm 2\%$ para temperaturas de $0,1^{\circ}\text{C}$ a 30°C y de $\pm 3\%$ para temperaturas superiores a 30°C . El EMP para la zona inferior del rango de flujo es de $\pm 5\%$, independientemente del alcance de temperatura.

Al medir un litro de agua se puede observar en la figura 4.2 y 4.3 el siguiente resultado:



Figura 4.2: Resultado de la calibración 1Litro

Fuente: (Elaboración propia)



Figura 4.3: Resultado de la calibración 1Litro

Fuente: (Elaboración propia)

4.4 CASO 2.

El cálculo para el costo por el consumo total de agua se basa en la fórmula que se puede ver en la figura 4.1, a continuación, describimos dicha formula, ver figura 4.4

$$consumo_total = \frac{costoPorLitro * contadorDeInterrupciones}{450}$$

Figura 4.4: Formula para el cálculo de costo total de agua consumida

Fuente: (Elaboración propia)

El margen de error del cálculo del costo variará según el costo por litro consumido.

4.5 CASO 3.

Según el subíndice 5.1 del Artículo 5 del reglamento técnico de medidores domiciliarios, elaborado por el ministerio de desarrollo productivo y economía plural, nos describe las condiciones de operación para los medidores de agua potable en la tabla 4.1

Tabla 4.1: Condiciones de Operación Medidores de Agua potable

Fuente: (Reglamento técnico de medidores domiciliarios)

Rango	Nivel
Rango caudal	Q1 a Q3 (Inclusive)
Temperatura del agua	Entre 0,1 °C y 30 °C
Rango de trabajo de temperatura ambiente	+5 °C a +55 °C
Rango de trabajo de humedad relativa ambiental (HR)	0 %HR a 100%HR excepto para dispositivos indicadores remotos, donde el rango será de 0 %HR a 93 %HR
Rango de presión de trabajo	Para: $DN \leq 50mm$ $0,03 \text{ Mpa} < PW < 1 \text{ MPa}$ $0,3 \text{ bar} < PW < 10 \text{ bar}$

Ahora realizamos una comparación de las características de nuestro medidor de Agua IoT en la tabla 4.2 para corroborar que cumple con las condiciones de operación requeridas especificadas en la tabla 4.1.

Tabla 4.2: Condiciones de Operación Medidores de Agua potable Comparativa con medidor IoT

Fuente: (Reglamento técnico de medidores domiciliarios)

	MÍNIMO REQUERIDO	OBTENIDO
Rango	Nivel	Nivel Medidor IoT
Rango caudal	Q1 a Q3 (Inclusive)	0.06 a 1.8 (Inclusive)
Temperatura del agua	Entre 0,1 °C y 30 °C	Entre 0,1 °C y 35 °C
Rango de trabajo de temperatura ambiente	+5 °C a +55 °C	-25°C a +80°C
Rango de trabajo de humedad relativa ambiental (HR)	0 %HR a 100%HR excepto para dispositivos indicadores remotos, donde el rango será de 0 %HR a 93 %HR	0% a 95%
Rango de presión de trabajo	Para: DN ≤ 50mm 0,03 Mpa < PW < 1 MPa 0,3 bar < PW < 10 bar	0,03 Mpa < PW < 1,75 Mpa 0,3 bar < PW < 17bar

Todos los requerimientos mínimos de operación han sido cumplidos por nuestro prototipo sin observación alguna, obteniendo un resultado favorable en este caso.

4.6 CASO 4.

Para tener un correcto control y monitoreo del consumo de Agua, se ha implementado la funcionalidad de grabar los datos del sensor de flujo de agua en una Tarjeta SD cuando se pierda la Conexión con el Servidor Blynk a falta de una conexión estable a Internet.

Cuando nuestro Hardware tiene una conexión estable despliega por puerto serie el siguiente estado, ver figura 4.5

```
ltss: 548.10  
cons: 1.09  
estd: Conectado  
-----
```

Figura 4.5: Datos en estado conectado

Fuente: (Elaboración propia)

Cuando se pierde la conexión se despliega por puerto serie el siguiente estado, ver figura 4.6

```
ltss: 594.37  
cons: 1.16  
estd: Desconectado  
-----
```

Figura 4.6: Datos en estado desconectado

Fuente: (Elaboración propia)

A su vez en la aplicación Blynk se detecta la desconexión del dispositivo como se puede ver en la figura 4.4, teniendo como últimos datos los obtenidos en la figura 4.7.

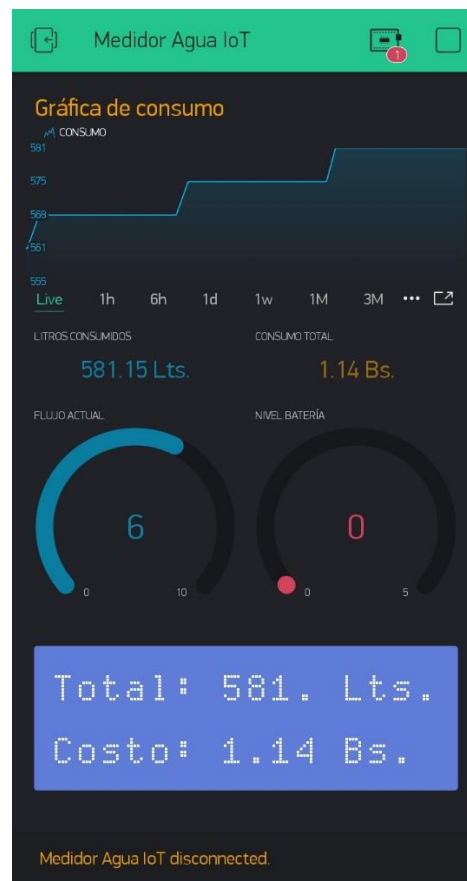


Figura 4.7: Blynk detecta la desconexión del dispositivo
Fuente: (Elaboración propia)

A pesar de estar desconectado, nuestro hardware prosigue con su tarea de registrar los datos de consumo como se puede ver en la figura 4.8

```
ltss: 603.07
cons: 1.18
estd: Desconectado
-----
```

Figura 4.8: El Hardware prosigue con el registro de datos
Fuente: (Elaboración propia)

Una vez reconectado el servicio de Internet se despliega el siguiente estado como se aprecia en la figura 4.9.

```
ltss: 649.00  
cons: 1.27  
estd: Conectado  
-----
```

Figura 4.9: Despliegue del estado Conectado al restablecer la conexión a Internet
Fuente: (Elaboración propia)

A su vez Blynk detecta la reconexión del dispositivo, actualizando sus datos con los almacenados por el Hardware, ver figura 4.10

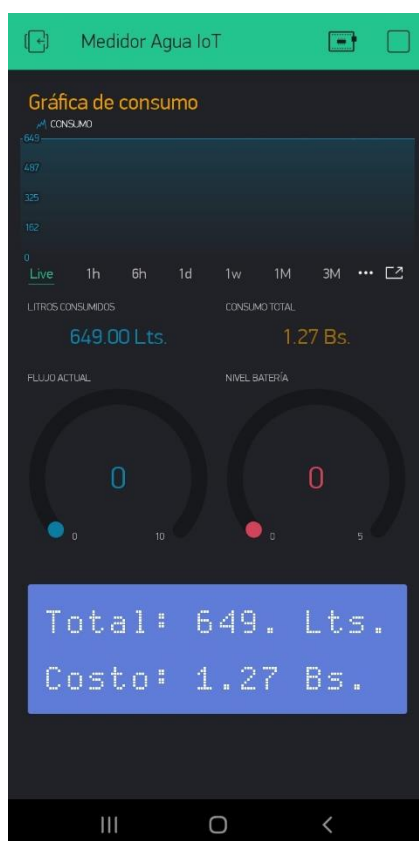


Figura 4.10: Blynk detecta la conexión con el dispositivo y actualiza sus datos
Fuente: (Elaboración propia)

El Hardware almacena en su Tarjeta SD la información del consumo en un archivo txt ver figura 4.11

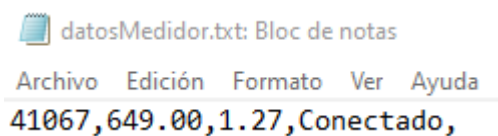


Figura 4.11: Dato almacenado en la Tarjeta SD
Fuente: (Elaboración propia)

Una vez realizadas todas las pruebas que se pueden apreciar, se obtiene un resultado favorable en el presente caso.

CAPITULO V

5. CONCLUSIONES Y RECOMENDACIONES

5.1 CONCLUSIONES

En el presente capitulo se detallan las conclusiones respecto al presente trabajo de investigación junto a los objetivos cumplidos tanto general como específicos.

Para llegar a cubrir las expectativas del objetivo general se realizaron pruebas utilizando el prototipo, de esa forma ir modificando el funcionamiento de este de tal forma que el funcionamiento sea óptimo.

5.1.1 ESTADO DE LOS OBJETIVOS

Estado del objetivo general

Se desarrolló e implemento un prototipo de medidor de agua IoT para poder obtener datos sobre el consumo de agua potable en hogares de la ciudad de La Paz para su control y monitoreo.

Estado de los objetivos específicos

- Se diseño el prototipo de medidor de agua IoT, para la obtención de datos.
- Se implemento el prototipo de medidor de agua IoT.
- Se generó graficas de consumo en la App de monitoreo.
- Se evaluó el funcionamiento del prototipo de medidor de agua IoT realizando pruebas en tiempo real.

Estado de la hipótesis.

La hipótesis del presente trabajo sostiene que “El uso de un Prototipo de Medidor de Agua IoT, nos permitirá obtener datos del consumo de agua potable en hogares de la ciudad de La Paz, para su control y monitoreo.”

Con la construcción e implementación del prototipo, el sistema realiza un monitoreo y control sobre el consumo de agua, permitiendo al usuario final visualizar estos datos, como se observa en la prueba de hipótesis del capítulo anterior.

5.2 RECOMENDACIONES

Para la elaboración de trabajos similares se recomienda lo siguiente:

Utilizar un servidor más avanzado para poder realizar interacción con estos datos hacia una interfaz web.

Si bien la Placa ESP32 es una de las más actuales en su clase, al ser Open Source está libre de ser mejorada tanto en estructura como componentes, se recomienda el uso de la plataforma TTGO la cual cuenta con la facilidad de tener una SIM700 que nos permitirá tener nuestra propia red 4G y no depender de una red WiFi, siendo toda la codificación compatible con dicha plataforma.

El sensor de voltaje debe ser calibrado y conectado de manera tal que se pueda reducir el ruido electrónico que afecta a la lectura final, concluyendo en el parpadeo de los leds relacionados con el mismo.

Actualmente en este prototipo los datos son registrados en tiempo real, creando enésimas lecturas que se van reciclando de manera automática en el Servidor Blynk, si se desea trabajar a futuro con un gestor de base de datos se recomienda cambiar la lógica de registro de datos.

BIBLIOGRAFÍA

- Fracctal, (2020). Las 9 aplicaciones más importantes del Internet de las Cosas (IoT). Recuperado de <https://www.fractal.com/blog/2018/10/10/9-aplicaciones-importantes-iot>
- Cantero, G. (2012, 10 de diciembre). Plataformas de Hardware. Programando a Media Noche. Recuperado de <https://www.programandoamedianoche.com/2012/12/plataformas-de-hardware/>
- Hernandez Salinas, V. J. (2017, 12 de octubre). Soluciones IoT mediante software y hardware libres. U-GOB. Recuperado de <https://u-gob.com/soluciones-iot-mediante-software-y-hardware-libres/>
- Editorial Geekflare, (2020, 21 de agosto). 12 Plataformas y herramientas de Internet de las cosas (IoT) de código abierto. Geekflare Desarrollo. Recuperado de <https://geekflare.com/es/iot-platform-tools/>
- Equipo NodeMcu, (2018). NodeMcu. Conecte las cosas Fácil. Recuperado de https://www.nodemcu.com/index_en.html
- Espressif Systems, (2020). ESP32. Recuperado de https://www.nodemcu.com/index_en.html
- Naylamp Mechatronics, (2028). NodeMCU 32. Naylamp Productos. Recuperado de <https://www.naylampmechatronics.com/espressif-esp/384-nodemcu-32-esp32-wifi.html>
- PedroJ, (2020, 18 de abril). Medidor de Agua Potable. Materiales para Laboratorio. Recuperado de <https://www.materialdelaboratorio.top/medidor-de-agua-potable/>
- Blynk Inc, (2020). Hacemos que el internet de las cosas sea sencillo para usted. Recuperado de [Blynk IoT platform: for businesses and developers](#)
- Oshwa Org, (2020). Open Source Hardware. Recuperado de <https://www.oshwa.org/definition/spanish/>

- Wikipedia. (2020, 21 de noviembre). Hardware Libre. Recuperado de https://es.wikipedia.org/wiki/Hardware_libre
- Rojas, S. (2020, 14 de junio). El Internet de las cosas o IoT. Vs Sistemas. Recuperado de <https://www.vs-sistemas.com/Blog/TIC's/internet-de-las-cosas>
- EPSAS (2018), Plan de Desarrollo Quinquenal EPSAS S.A.2018-2022. Recuperado de http://www.epsas.com.bo/web/wp-content/uploads/2019/05/PDQ_2018.pdf
- Enrique Crespo (2019), Aprendiendo Arduino Sensores y Actuadores. Recuperado de <https://aprendiendoarduino.wordpress.com/2016/12/18/sensores-y-actuadores/>
- Luis Llamas (2018, 1 de Junio), NodeMCU, La popular placa de desarrollo con ESP8266. Recuperado de <https://www.luisllamas.es/esp8266-nodemcu/>
- AAPS (2013, 23 de Septiembre), El 70% de la población de Bolivia consume agua potable de calidad. Recuperado de <https://www.iagua.es/noticias/bolivia/13/09/23/el-70-de-la-poblacion-de-bolivia-consume-agua-potable-de-calidad-36952>
- Tecnología Humanizada (2018, 6 de noviembre), Blynk, plataforma de internet de las cosas en la red. Recuperado de <https://humanizationoftechnology.com/blynk-plataforma-de-internet-de-las-cosas-en-la-red/revista/2018/volumen-4-2018/11/2018/>

ANEXO A

CODIGO COMPLETO ESP32

```
#include <Arduino.h>
#include <LiquidCrystal_I2C.h>
#include <WiFi.h>
#include <WiFiClient.h>
#include <BlynkSimpleEsp32.h>
#include <SD.h>
#include <SPI.h>
//#include <FS.h>

//VARIABLES DE CONEXIÓN
char auth[] = "rKvBp1TXvTAuTtCAIWWE5_RBS1aCVUQg"; //Colocar aquí el token
que fue enviado a su mail.
//char ssid[] = "ZTE-ec47d0"; //Red WiFi personal
//char pass[] = "78312bec"; // Contraseña para WiFi personal

char ssid[] = "MarazaVigabriel"; //Red WiFi personal
char pass[] = "25351010"; // Contraseña para WiFi personal

File myFile;
unsigned long previousMillis = 0;
unsigned long interval = 10000;
unsigned long previousMillis_datos = 0;
unsigned long interval_datos = 10000;

int lcdColumns = 16;
int lcdRows = 2;
const byte ADDRESS = 0x27; // dirección I2C para el LCD (0x3F & 0x27 regularmente)
LiquidCrystal_I2C lcd(0x27, lcdColumns, lcdRows);
WidgetLCD lcdBlynk(V4);

const byte SENSOR = 4; // sensor en GPIO04 //antes GPIO18
//const byte BUTTON = 23; // boton en GPIO23

float LITROS_TOTAL = 0;
float CONSUMO_TOTAL = 0;

#define LD1 13
```

```

#define LD2 12
#define LD3 14
#define LD4 27
#define LD5 26
#define LD6 25
#define LD7 33
#define LD8 32
#define LD9 15
#define LD10 2

#define SENSOR_VOLT 35
int sensorValue;    // variable que almacena el valor raw (0 a 1023)
float value;        // variable que almacena el voltaje (0.0 a 25.0)

unsigned long count = 0;    // número de pulsos del sensor de flujo (un pulso es 1/450 de
                             litro)
unsigned long last_count = 0; // temporizador para rastrear el último pulso del sensor
unsigned long last_time = 0; // temporizador para rastrear la última actualización de LCD
const int INTERVAL = 200;    // frecuencia de actualización para la lectura de la pantalla
float COST_PER_LITRE = 0.002; // costo por litro 0.002, en centavos//2bs cada 1000
                             litros// 2bs x 1m3 CostoInicial
unsigned long debounce = 0; // temporizador para botón antirrebote
float litros_consumidos = 0;
float costo_consumido = 0;
bool mode_changed = true; // una bandera para actualizar la etiqueta de texto "Tarifa:" o
                             "Costo:"
bool watering = false;    // una bandera para indicar que el flujo de agua está activo
bool mode = false;        // mostrar el caudal si es verdadero, o el costo si es falso

////////////////////////////////////
////////////////////////////////////FUNCIONES PARA EL MÓDULO DE CONTROL DEL NIVEL DE
LA BATERÍA////////////////////////////////////
////////////////////////////////////

void lcd_volt(float valueX){
    int valorVolt = map(valueX, 0.0, 5.25, 0, 10);

    switch (valorVolt)
    {
    case 1:
        digitalWrite(LD1, HIGH);
        digitalWrite(LD2, LOW);

```

```
digitalWrite(LD3, LOW);  
digitalWrite(LD4, LOW);  
digitalWrite(LD5, LOW);  
digitalWrite(LD6, LOW);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 2:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, LOW);  
digitalWrite(LD4, LOW);  
digitalWrite(LD5, LOW);  
digitalWrite(LD6, LOW);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 3:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, LOW);  
digitalWrite(LD5, LOW);  
digitalWrite(LD6, LOW);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 4:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);
```

```
digitalWrite(LD5, LOW);  
digitalWrite(LD6, LOW);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 5:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, LOW);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 6:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, HIGH);  
digitalWrite(LD7, LOW);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 7:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, HIGH);
```

```
digitalWrite(LD7, HIGH);  
digitalWrite(LD8, LOW);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 8:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, HIGH);  
digitalWrite(LD7, HIGH);  
digitalWrite(LD8, HIGH);  
digitalWrite(LD9, LOW);  
digitalWrite(LD10, LOW);  
break;
```

case 9:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, HIGH);  
digitalWrite(LD7, HIGH);  
digitalWrite(LD8, HIGH);  
digitalWrite(LD9, HIGH);  
digitalWrite(LD10, LOW);  
break;
```

case 10:

```
digitalWrite(LD1, HIGH);  
digitalWrite(LD2, HIGH);  
digitalWrite(LD3, HIGH);  
digitalWrite(LD4, HIGH);  
digitalWrite(LD5, HIGH);  
digitalWrite(LD6, HIGH);  
digitalWrite(LD7, HIGH);  
digitalWrite(LD8, HIGH);
```



```

    digitalWrite(LD9, HIGH);
    digitalWrite(LD10, HIGH);
    break;

default:
    digitalWrite(LD1, HIGH);
    digitalWrite(LD2, LOW);
    digitalWrite(LD3, LOW);
    digitalWrite(LD4, LOW);
    digitalWrite(LD5, LOW);
    digitalWrite(LD6, LOW);
    digitalWrite(LD7, LOW);
    digitalWrite(LD8, LOW);
    digitalWrite(LD9, LOW);
    digitalWrite(LD10, LOW);
    break;
}
}

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
void validar_costos(float consumo){
    if(consumo > 10000 && consumo < 20000){
        COST_PER_LITRE = 0.005;
    } else {
        if(consumo > 20000 && consumo < 30000){
            COST_PER_LITRE = 0.01;
        } else{
            if(consumo > 30000){
                COST_PER_LITRE = 0.015;
            }
        }
    }
}

// cambio de escala entre floats
float fmap(float x, float in_min, float in_max, float out_min, float out_max)
{
    return (x - in_min) * (out_max - out_min) / (in_max - in_min) + out_min;
}

void volt_med(){

```

```

sensorValue = analogRead(SENSOR_VOLT);      // realizar la lectura
value = fmap(sensorValue, 0, 1023, 0.0, 25.0); // cambiar escala a 0.0 - 25.0
if(value > 0){
  lcd_volt(value);
  Blynk.virtualWrite(V3, value);
}
else{
  lcd_volt(0);
  Blynk.virtualWrite(V3, 0);
}
}

////////////////////////////////////
////////////////////////////////////
void button_press() {
  if (abs(millis() - debounce) > 250) { // Retardo antirrebote para evitar pulsaciones de
  botones no deseadas
    debounce = millis(); // restablecer el temporizador antirrebote
    if (watering) { // si el flujo está corriendo, el botón cambia el modo
      mode = !mode;
      mode_changed = true; // bandera activa
    }
    else { // si el flujo no está corriendo, el botón borra el total
      count = 0;
      last_count = 0;
    }
  }
}

void IRAM_ATTR sensor_pulse() {
  count++; // aumentar el recuento en uno
}

void datos_lcd(){
  if (mode_changed) { // si el modo ha cambiado, imprima la etiqueta de texto
  actualizada
    lcd.print("Total:      ");
    lcd.setCursor(0, 1); // mover el puntero a segunda linea
    if (mode)
      lcd.print("Flujo:      "); // el espacio en blanco es para borrar cualquier carácter
  extraño

```

```

else
    lcd.print("Costo:      "); // el espacio en blanco es para borrar cualquier carácter
    extraño
    mode_changed = false; // limpiar bandera
}
if (abs(millis() - last_time) > INTERVAL) { // ha pasado más de INTERVAL-time, así
que actualice la pantalla
    last_time = millis(); // establecer la última hora de actualización a la hora actual
    lcd.setCursor(7, 0);
    if (litros_consumidos > 1000){
        lcd.print(String((litros_consumidos+(count / 450.0))/1000) + " m3 "); // calcular e
imprimir litros totales
    } else {
        lcd.print(String(litros_consumidos+(count / 450.0)) + " Lts "); // calcular e imprimir
litros totales
    }
    //lcd.print(String(litros_consumidos+(count / 450.0)) + " L "); // calcular e imprimir
litros totales
    LITROS_TOTAL = litros_consumidos+(count / 450.0);
    //Blynk.virtualWrite(V1, litros_consumidos+(count / 450.0));
    Blynk.virtualWrite(V5, (count/450));

    lcd.setCursor(7, 1);
    if (mode) { // modo de caudal, muestra el caudal actual
        lcd.print(String(1000 * (count - last_count) / 450.0 / INTERVAL) + (" L/s  "));
    }
    else { // modo de costo, muestra el costo total
        validar_costos(costo_consumido);
        CONSUMO_TOTAL = costo_consumido+(COST_PER_LITRE * count / 450.0);
        lcd.print(String(costo_consumido+(COST_PER_LITRE * count / 450.0)) + " ");
        //Blynk.virtualWrite(V2, costo_consumido+(COST_PER_LITRE * count / 450.0));
        lcd.print("Bs.");
        lcd.print(" "); // imprimir espacios finales para borrar caracteres extraños
    }
    if (last_count == count) // así es como determinamos si el flujo está funcionando(paso
de agua)
        watering = false; // caudal off
    else
        watering = true; // caudal on

    last_count = count;

```

```

    }
}

BLYNK_WRITE(V1){
    litros_consumidos = param.asFloat(); // get the value from the server and save it back to
my_data_value
}

BLYNK_WRITE(V2){
    costo_consumido = param.asFloat(); // get the value from the server and save it back to
my_data_value
}

void setup() {
    Serial.begin(9600);
    delay(10);
    Serial.println(ssid);
    WiFi.begin(ssid, pass);
    //int wifi_ctr = 0;
    while(WiFi.status() != WL_CONNECTED){
        delay(500);
        Serial.print(".");
    }

    Serial.println("WiFi Conectado");
    Blynk.begin(auth, ssid, pass);

    Serial.print("Iniciando SD...");
    if(!SD.begin(SS)){
        Serial.println("No se pudo inicializar");
        return;
    }
    Serial.println("Inicialización exitosa");

    analogReadResolution(10); ///CONFIGURACIÓN DE PINES PARA EL MÓDULO DE
CONTROL DE NIVEL DE BATERÍA///
    pinMode(LD1, OUTPUT);
    pinMode(LD2, OUTPUT);
    pinMode(LD3, OUTPUT);
    pinMode(LD4, OUTPUT);
    pinMode(LD5, OUTPUT);

```

```

pinMode(LD6, OUTPUT);
pinMode(LD7, OUTPUT);
pinMode(LD8, OUTPUT);
pinMode(LD9, OUTPUT);
pinMode(LD10, OUTPUT);
lcd.init(); // Inicializar LCD
lcd.backlight();
lcd.clear();
lcd.print("---MEDIDOR IOT---");
lcd.clear();
delay(1000);
pinMode(SENSOR, INPUT_PULLUP); // Configure las funciones de los pines y
habilite las resistencias pull-up internas
attachInterrupt(SENSOR, sensor_pulse, FALLING); // Inicializar interrupciones
}

void grabarDatos(float litros, float costo, unsigned long time, String estado) {
  myFile = SD.open("/datosMedidor.txt", FILE_WRITE); //abrimos el archivo
  if (myFile) {
    //Serial.print("Escribiendo en SD:");
    //myFile.print("Tiempo(ms)= ");
    myFile.print(time);
    myFile.print(",");
    myFile.print(litros);
    myFile.print(",");
    myFile.print(costo);
    myFile.print(",");
    myFile.print(estado);
    myFile.print(",");
    //myFile.print("\n");
    myFile.close(); //cerramos el archivo

    Serial.print("hora: ");
    Serial.print(time);
    Serial.println("");
    Serial.print("ltss: ");
    Serial.print(litros);
    Serial.println("");
    Serial.print("cons: ");
    Serial.print(costo);
    Serial.println("");
  }
}

```

```

    Serial.print("estd: ");
    Serial.println(estado);
    Serial.println("-----");
}
}

void subirDatos(){
    myFile = SD.open("/datosMedidor.txt"); //abrimos el archivo
    String horaD;
    String litrosD;
    String consumoD;
    String estadoD;
    int cc= 0;
    String cadena = "";
    while(myFile.available()) {
        char character = myFile.read();
        cadena = cadena + character;
    }
    Serial.print("DATOS: SPT ");
    //Serial.print(cadena);
    myFile.close();
    //String datos = cadena;
    String sub = "";
    for (int i = 0; i < cadena.length(); i++)
    {
        char c = cadena[i];

        if (c == ','){
            cc++;
            if(cc==1){
                horaD=sub;
                sub="";
            }
            if(cc==2){
                litrosD=sub;
                sub="";
            }
            if(cc==3){
                consumoD=sub;
                sub="";
            }
        }
    }
}

```

```

    if(cc==4){
        estadoD=sub;
        sub="";
    }
    } else {
        sub=sub+c;
    }
}

Serial.print("hora: ");
Serial.print(horaD);
Serial.println("");
Serial.print("ltss: ");
Serial.print(litrosD);
Serial.println("");
Serial.print("cons: ");
Serial.print(consumoD);
Serial.println("");
Serial.print("estd: ");
Serial.println(estadoD);
Serial.println("-----");
LITROS_TOTAL = litrosD.toFloat();
CONSUMO_TOTAL = consumoD.toFloat();
litros_consumidos = LITROS_TOTAL;
costo_consumido = CONSUMO_TOTAL;
Blynk.virtualWrite(V1, litros_consumidos);
Blynk.virtualWrite(V2, costo_consumido);
lcdBlynk.print(0,0,"Total:");
lcdBlynk.print(7,0,LITROS_TOTAL);
lcdBlynk.print(11,0," Lts.");
lcdBlynk.print(0,1,"Costo:");
lcdBlynk.print(7,1,CONSUMO_TOTAL);
lcdBlynk.print(11,1," Bs. ");
}

BLYNK_CONNECTED(){
    subirDatos();
}

void loop() {
    unsigned long currentMillis = millis();

```

```

Blynk.run();
if (Blynk.connected() != false){
  subirDatos();
  datos_lcd();
  volt_med();
  if (currentMillis - previousMillis_datos >= interval_datos){
    grabarDatos(LITROS_TOTAL, CONSUMO_TOTAL, currentMillis, "Conectado");
    previousMillis_datos = currentMillis;
  }
} else {
  datos_lcd();
  volt_med();
  if (currentMillis - previousMillis_datos >= interval_datos){
    grabarDatos(LITROS_TOTAL, CONSUMO_TOTAL, currentMillis, "Desconectado");
    previousMillis_datos = currentMillis;
  }
  if ((WiFi.status() != WL_CONNECTED) && (currentMillis - previousMillis >=
interval)){
    WiFi.disconnect();
    WiFi.reconnect();
    previousMillis = currentMillis;
  }
}
}

```


ANEXO B

REGLAMENTO TÉCNICO DE MEDIDORES DOMICILIARIOS DE AGUA

POTABLE

CAPITULO I.

OBJETO, CAMPO DE APLICACIÓN, DEFINICIONES, SIGLAS Y

REFERENCIAS.

Artículo 1. OBJETO.

El objeto del presente Reglamento Técnico es establecer los requisitos que deben cumplir los medidores domiciliarios de agua potable fría a ser usados en conductos cerrados, con el fin de prevenir las prácticas que puedan inducir al error de las usuarias y usuarios.

Artículo 2. CAMPO DE APLICACIÓN.

El presente Reglamento Técnico aplica a los medidores para agua potable fría con diámetro nominal de hasta 50 mm, con Clase de Exactitud 1 y Clase de Exactitud 2, a ser usados en conductos cerrados, cuyas mediciones se utilicen para transacciones comerciales; y que se comercialicen en el Estado Plurinacional de Bolivia, sean de fabricación nacional o importados y que se encuentran en la subpartida arancelaria de la Tabla 1.

Tabla 1. Subpartida arancelaria aplicada al Reglamento Técnico.

Código Descripción Observaciones

90.28 Contadores de gas, líquido o electricidad,
incluidos los de calibración.

9028.20 - Contadores de líquido:

9028.20.10.00 - - Contadores de agua.

Solo aplica a medidores de agua
potable con diámetros nominales
de hasta 50 mm

Artículo 3. DEFINICIONES, SIGLAS Y DOCUMENTOS DE REFERENCIA.

3.1. Definiciones.

Para los efectos del presente Reglamento Técnico se adoptan las definiciones que a continuación se detallan:

Autoridad

Reguladora

Autoridad de Fiscalización y Control Social de Agua Potable y
Saneamiento básico.

Autoridad de

supervisión.

Autoridad nacional competente para supervisar el
cumplimiento del presente Reglamento Técnico.

Calculador. La parte del medidor que transforma las señales de salida del
transductor de medición y posiblemente de los instrumentos
de medición asociados, los transforma y si corresponde,
almacena los resultados en la memoria hasta que sean
utilizados.

Nota 1. En un medidor mecánico, el engranaje es considerado como
el calculador.

Nota 2. El calculador debe tener la capacidad de comunicarse con
los dispositivos auxiliares ambas vías.

Caudal. Cociente del volumen real de agua que pasa a través del medidor y el tiempo transcurrido para que este volumen pase a través del mismo.

Nota: Para el presente Reglamento Técnico, se clasifica a los caudales como: mínimo, de transición, permanente y de sobrecarga conforme el siguiente gráfico:

Caudal mínimo. Es el caudal más pequeño al cual operará el medidor dentro de los errores máximos permisibles.

**Caudal de
sobrecarga.**

Es el caudal más alto al cual el medidor puede operar durante un corto periodo de tiempo dentro de los errores máximos permisibles, manteniendo su desempeño metrológico, cuando posteriormente opera dentro de sus condiciones nominales de operación.

**Caudal de
transición.**

Es el que se encuentra entre el caudal permanente y el caudal mínimo que divide el rango de caudal en dos zonas, la zona de caudal superior y la zona de caudal inferior, cada una caracterizada por sus propios errores máximos permisibles.

**Caudal
permanente.**

Es el máximo caudal dentro de las condiciones nominales de

operación, al cual debe operar el medidor dentro de los errores máximos permisibles.

Condiciones de referencia.

Es la condición de operación establecida para evaluar el desempeño de un medidor o para comparar resultados de medición.

Condiciones nominales de operación.

la medición con el fin de que un medidor se desempeñe según su diseño.

Nota: Las condiciones nominales de operación especifican intervalos para el caudal y para las cantidades de influencia para los cuales los errores (de indicación), deben estar dentro de los errores máximos permisibles.

Dispositivo de ajuste.

Es la parte del medidor que permite ajustar el medidor de modo que su curva de error sea ajustada generalmente en paralelo a ella misma, para hacer que los errores se encuentren dentro de los límites de errores máximos permisibles.

Dispositivo auxiliar. Es un dispositivo previsto para desarrollar una función

específica, directamente relacionada con la elaboración, transmisión o exhibición de los valores medidos.

Nota 1. Para el caso del presente Reglamento Técnico, los dispositivos auxiliares son complementarios al medidor de agua y su uso o aplicación es opcional, sin embargo, pueden estar sujetos a control metrológico legal, en caso de ser necesario.

Nota 2. Los principales dispositivos auxiliares son:

- a) Dispositivo de puesta a cero.
- b) Dispositivo indicador de precio.
- c) Dispositivo indicador de repetición.
- d) Dispositivo de impresión.
- e) Dispositivo memorizador.
- f) Dispositivo de control tarifario.
- g) Dispositivo predeterminador y.
- h) Dispositivo de auto servicio.

Dispositivo

indicador.

Parte del medidor que indica el volumen de agua que pasa por el medidor.

Durabilidad. Es la capacidad de un medidor para mantener sus características de desempeño durante un periodo de uso.

Error (de la indicación).

Volumen indicado menos el volumen real.

Error intrínseco**inicial.**

Error intrínseco de un medidor según se determine antes de las pruebas de rendimiento y las evaluaciones de durabilidad.

Error intrínseco. Error de un medidor determinado bajo las condiciones de referencia.

Error máximo**permisible.**

Valor extremo del error de medición, respecto a un valor de magnitud de referencia conocido, permitido por el presente Reglamento.

Importador. Persona natural o jurídica, que presenta mediante una agencia despachante de aduana, la declaración de mercancías para el despacho, con el cumplimiento de las formalidades aduaneras

Marca de**verificación.**

Identificación o distintivo otorgado a cada medidor de agua, indicando que el mismo cumple con todos los requisitos especificados en el presente Reglamento Técnico.

Medidor de agua. Instrumento diseñado para medir continuamente, memorizar y mostrar el volumen de agua que pasa a través de un transductor de medición en condiciones medibles.

Nota 1: Un medidor de agua incluye, al menos, un transductor de medición, un calculador (incluyendo dispositivos de ajuste o

corrección, si los hay) y un dispositivo indicador. Estos tres dispositivos pueden estar en ubicaciones diferentes.

Nota 2: En el presente Reglamento Técnico, un medidor de agua también es llamado “medidor”.

Nota 3: En el presente Reglamento Técnico, se consideran medidores de agua también a los de “chorro único” y “chorro múltiple”.

Pérdida de presión. Es la disminución irre recuperable en la presión, a un caudal determinado, causada por la presencia del medidor en la tubería.

Presión de trabajo. Presión promedio (estimada) del agua en la tubería, medida a la entrada y a la salida del medidor.

Presión máxima

admisible.

Presión interna máxima que un medidor puede soportar permanentemente, dentro de sus condiciones nominales de operación, sin deteriorar su desempeño metrológico.

Temperatura de

trabajo.

Temperatura del agua en la tubería, medida a la salida del medidor.

Temperatura

máxima permisible.

Temperatura máxima del agua que un medidor puede soportar permanentemente, dentro de sus condiciones nominales de

operación, sin deteriorar su desempeño metrológico.

Nota: La temperatura máxima permisible es la condición nominal de operación más alta para la temperatura.

Temperatura

mínima permisible.

Temperatura mínima del agua que un medidor puede soportar permanentemente, dentro de sus condiciones nominales de operación, sin deteriorar su desempeño metrológico.

Nota: La temperatura mínima permisible del agua es la condición nominal de operación más baja para la temperatura

Volumen indicado. Volumen de agua indicado por el medidor y que corresponde al volumen real.

Volumen real. Volumen total de agua que pasa por el medidor, independientemente del tiempo que le tome.

Nota 1: Este es el mensurando.

Nota 2: El volumen real se calcula desde un volumen de referencia, según se determine por un estándar de medición adecuado, teniendo en cuenta las diferencias en las condiciones de medición, según sea apropiado.

3.2. Siglas.

Las siglas usadas en el presente Reglamento Técnico son descritas a continuación:

AAPS Autoridad de Fiscalización y Control Social de Agua Potable y Saneamiento Básico.

DN Diámetro nominal.

EMP Error máximo permisible.

IBMETRO Instituto Boliviano de Metrología.

OIML Organización Internacional de Metrología Legal.

PMA Presión máxima admisible.

PW Presión de trabajo.

Q Caudal.

Q1 Caudal mínimo.

Q2 Caudal de transición.

Q3 Caudal permanente.

Q4 Caudal de sobrecarga.

SI Sistema Internacional de Unidades.

TMA Temperatura máxima permisible.

TmA Temperatura mínima permisible.

3.3. Documentos de Referencia.

Las fuentes de información consideradas para la elaboración del presente Reglamento Técnico son las siguientes:

- Norma Internacional. Recomendación Internacional. Medidores de agua potable fría y caliente. Parte 1: Requisitos técnicos y metrológicos. Organización Internacional de Metrología Legal, OIML R 49-1:2013.
- Norma Internacional. Recomendación Internacional. Medidores de agua potable fría y caliente. Parte 2: Métodos de ensayo. Organización Internacional de Metrología Legal, OIML R 49-2:2013.

CAPITULO II.

REQUISITOS.

Artículo 4. CONDICIONES GENERALES.

Todos los medidores de agua potable, contemplados en el presente Reglamento Técnico, deben cumplir lo siguiente:

4.1. Materiales y construcción de los medidores.

4.1.1. Los medidores se construirán a partir de materiales con resistencia y durabilidad suficientes para el propósito para el que van a ser usados.

4.1.2. Los medidores serán contruidos con materiales que no sean afectados por las variaciones de temperatura del agua, dentro del alcance de temperatura de operación, (ver Tabla 3).

4.1.3. Todas las piezas del medidor en contacto con el agua estarán fabricadas con materiales comúnmente reconocidos como no tóxicos, no contaminantes y biológicamente inertes.

4.1.4. El medidor de agua completo será fabricado con materiales resistentes a la corrosión interna y externa, o que estén adecuadamente protegidos mediante un tratamiento superficial.

4.1.5. El dispositivo indicador del medidor de agua estará provista por una ventanilla transparente. También debe contar con una cubierta de material adecuado como protección adicional.

4.1.6. Cuando exista el riesgo de que se forme condensación en la parte inferior de la ventana del dispositivo indicador de un medidor de agua, este deberá incorporar dispositivos para eliminar dicha condensación.

4.1.7. Las conexiones terminales roscadas deben cumplir los requisitos técnicos

indicados en la norma ISO 4064-4.

4.2. Corrección y ajuste.

El medidor debe contar con un dispositivo de ajuste y/o de corrección que permita desplazar la curva de error con miras a llevar los errores, dentro de los errores máximos permisibles. Sí estos dispositivos están montados en la parte exterior del medidor, deben tener una cavidad que permita el precintado de este dispositivo de tal manera que no permita ajustes posteriores al precintado.

4.3. Requerimientos del Indicador.

4.3.1. Función.

- a) El dispositivo indicador del medidor de agua deberá proporcionar una indicación de fácil lectura, confiable y clara del volumen indicado en m³.
- b) El dispositivo deberá incluir medios visuales para verificaciones y calibraciones.
- c) El dispositivo de indicación podrá incluir elementos adicionales para verificaciones y calibraciones por otros métodos, por ejemplo, elementos automáticos.

4.3.2. Unidad de medida, símbolo y su ubicación.

La indicación del volumen de agua debe estar expresada en metros cúbicos. El símbolo correspondiente m³, debe aparecer en el dial o inmediatamente junto al número exhibido.

4.3.3. Rango (alcance) de indicación.

El dispositivo debe tener la capacidad de registrar el volumen indicado en metros cúbicos. Esto queda expresado en la siguiente Tabla 2 sin pasar por el cero.

Tabla 2. Rango de indicación de un medidor de agua.

Q3 (m³/h) Valores mínimos del rango del indicador

(m3)

$Q3 \leq 6,3$

$6,3 < Q3 \leq 63$

$63 < Q3 \leq 630$

$630 < Q3 \leq 6.300$

9.999

99.999

999.999

9.999.999

4.3.4. Codificación por color del dispositivo.

- a) El color negro debe usarse para indicar metros cúbicos y sus múltiplos.
- b) El color rojo para indicar submúltiplos del metro cúbico.
- c) Estos colores se aplicarán a los punteros, agujas, números, ruedas, discos y diales.

4.3.5. Tipos de dispositivos indicadores.

Se podrán utilizar cualquiera de los siguientes tipos:

a) Tipo 1 - Dispositivo analógico.

- i. El volumen es indicado por el movimiento continuo de:
 - Uno o más punteros que se mueven en relación a escalas graduadas.
 - Una o más escalas circulares o tambores.
- ii. Cada escala estará graduada en valores expresados en metros cúbicos o bien estará acompañada por un factor multiplicador (x 0,001; x 0,01; x 0,1; x 1; x 10, x 100, x 1000, etc.)
- iii. El sentido de rotación de los punteros o de las escalas circulares debe ser en

sentido horario.

iv. El movimiento lineal de los punteros o escalas será de izquierda a derecha.

v. El movimiento de los indicadores numerados (tambores o rodillos) será hacia arriba.

b) Tipo 2 - Dispositivo digital.

i. El volumen indicado estará dado por una línea de dígitos adyacentes que aparecen en una o más aperturas. El avance de un dígito dado será completado mientras el dígito de la siguiente decena inmediatamente inferior cambia de 9 a 0.

ii. El movimiento de los tambores indicadores será hacia arriba.

iii. La decena de menor valor puede tener un movimiento continuo, siendo la apertura suficientemente grande para permitir que un dígito se lea sin confusión.

iv. La altura de los dígitos será de por lo menos 4 mm.

c) Tipo 3 - Combinación de los dispositivos analógico y digital.

El volumen indicado está dado por la combinación de dispositivos Tipo 1 (Dispositivo analógico) y Tipo 2 (Dispositivo digital) y serán aplicables los requisitos respectivos.

4.3.6. Dispositivos auxiliares.

a) Además de los dispositivos indicadores descritos, el medidor puede incluir dispositivos suplementarios los cuales pueden estar incorporados permanentemente o ser agregados temporalmente.

b) Estos dispositivos pueden usarse para detectar el movimiento del sensor de flujo, antes de que sea claramente visible en el indicador.

c) Estos dispositivos podrán usarse para los ensayos y verificaciones o lectura

remota del indicador, siempre que por otros medios se garantice el correcto funcionamiento del medidor.

4.4. Dispositivos de Verificación.

4.4.1. Todo indicador debe proveer medios para verificación, ensayo y calibración visual en forma clara. El visor de verificación puede o no tener un movimiento continuo.

4.4.2. Además del visor, un dispositivo indicador puede incluir elementos complementarios para una comprobación rápida por la inclusión de elementos complementarios (p.ej. ruedas dentadas o discos), que provean señales a través de sensores adosados externamente.

4.4.3. Visores de verificación.

a) Valor del intervalo de la escala de verificación.

- i.** Los valores del intervalo de la escala de verificación expresado en metros cúbicos tendrán la forma: 1×10^n , 2×10^n o 5×10^n , donde “n” es un número entero positivo o negativo o cero.
- ii.** Para dispositivos analógicos o digitales con movimiento continuo del primer elemento o elemento de control, la escala de verificación puede formarse a partir de la división en 2, 5 o 10 partes iguales del intervalo entre dos dígitos consecutivos del primer elemento o elemento de control. No se debe aplicar numeración a estas divisiones.
- iii.** Para los dispositivos digitales con movimiento discontinuo del primer elemento o elemento de control, la escala de verificación es el intervalo entre dos dígitos consecutivos o movimientos en aumento del primer elemento.

b) Marcas de verificación y dispositivos de protección

- i. Debe ser provisto en el medidor de agua, un lugar para poner la marca de verificación principal que deberá ser visible sin desarmar el medidor.
- ii. Los medidores incluirán dispositivos de protección que deben estar sellados para evitar el desarmado o modificación del mismo, su dispositivo de ajuste, antes y después de la correcta instalación del medidor, sin dañar estos dispositivos.

Artículo 5. REQUISITOS ESPECÍFICOS.

5.1. Condiciones nominales de operación.

Las condiciones de operación para los medidores de agua potable se encuentran en la Tabla 3.

Tabla 3. Condiciones de Operación para medidores de agua potable.

Rango Nivel

Rango de caudal Q1 a Q3 (Inclusive)

Temperatura del agua Entre 0,1 °C y 30 °C

Rango de trabajo de temperatura

ambiente +5 °C a +55 °C

Rango de trabajo de humedad relativa

ambiental (HR)

0 %HR a 100 %HR excepto para dispositivos

indicadores remotos, donde el rango será de 0

%HR a 93 %HR

Rango de presión de trabajo

Para: DN < 50 mm

0,03 MPa < PW < 1 MPa

$$0,3 \text{ bar} < PW < 10 \text{ bar}$$

5.2. Pérdida de Presión.

La pérdida de presión a través del medidor de agua, incluyendo su filtro que forma una parte integral del medidor, no será mayor de 0,063 MPa (0,63 bar) entre Q1 (Caudal mínimo) y Q3 (Caudal permanente).

5.3. Visores de verificación.

a) Forma del intervalo de la escala de verificación.

i. En los indicadores con movimiento continuo del primer elemento, la longitud del intervalo de la escala no será menor que 1 mm ni mayor que 5 mm. La escala constará de:

- Líneas de igual espesor que no excedan un cuarto del espacio entre ejes de dos líneas consecutivas y que se diferencian sólo en longitud.
- Bandas contrastantes de un ancho constante igual al valor del intervalo de la escala de verificación.

ii. El ancho de la punta de la aguja no excederá un cuarto del valor del intervalo de la escala de verificación y en ningún caso será mayor de 0,5 mm.

b) Resolución del Indicador.

i. Las subdivisiones de la escala de verificación serán lo suficientemente pequeñas para asegurar que el error de resolución del indicador no excederá 0,25 % para los medidores de Clase de exactitud 1, y 0,5 % para los de Clase de exactitud 2, respecto del volumen real que pasa durante una hora y 30 minutos a régimen de flujo Q1.

ii. Elementos adicionales de verificación pueden ser usados para que la incertidumbre de lectura no sea mayor que 0,25 % del volumen bajo prueba,

para medidores de Clase de exactitud 1 y 0,5% para medidores de Clase de exactitud 2.

Cuando el visor del primer elemento es continuo, se debe fijar un error máximo permisible en cada lectura de no más de la mitad del intervalo de la escala.

iii. Cuando el visor es discontinuo, se debe fijar un error máximo permisible en cada lectura de no más de un dígito de la escala de verificación.

5.4. Características de Caudal

5.4.1. Las características de caudal de un medidor de agua estarán definidas por los valores Q1, Q2, Q3 y Q4.

5.4.2. Se designará un medidor de agua por el valor numérico de Q3 expresado en m³/h y por la proporción de Q3/Q1 (Alcance de medición).

5.4.3. El valor de Q3 debe ser seleccionado de la Lista 1.

Lista 1. Valores de Q3 expresado en m³/h

1,0 1,6 2,5 4,0 6,3

10 16 25 40 63

100 160 250 400 630

1000 1600 2500 4000 6300

5.4.4. Los valores de la lista pueden ser extendidos hacia arriba o hacia abajo en las series.

5.4.5. La relación Q3/Q1 (Alcance de medición) debe ser seleccionada de la Lista 2.

Lista 2. Valores de relación Q3/Q1 (Alcance de medición)

40 50 63 80 100

125 160 200 250 315

400 500 630 800 1000

5.4.6. Los valores de la lista pueden ser extendidos hacia arriba en las series.

5.4.7. La relación Q_2/Q_1 debe ser igual a 1,6.

5.4.8. La relación Q_4/Q_3 debe ser igual a 1,25.

5.5. Clase de exactitud y error máximo permisible.

5.5.1. Los medidores estarán clasificados, de acuerdo a su exactitud, como Clase de Exactitud 1 o Clase de Exactitud 2.

5.5.2. Los medidores deben estar fabricados para que sus Errores (de indicación) no excedan los EMP, dentro del régimen de condiciones de operación del medidor.

5.5.3. La totalización del medidor (registro de medición) no cambiará cuando el régimen de flujo sea cero.

5.6. Medidores con Clase de Exactitud 1.

5.6.1. El EMP para la zona superior del rango de flujo ($Q_2 \leq Q \leq Q_4$) es $\pm 1 \%$ para temperaturas de $0,1^\circ\text{C}$ a 30°C y $\pm 2 \%$ para temperaturas superiores a 30°C .

5.6.2. El error máximo permisible para la zona inferior del rango de flujo ($Q_1 \leq Q \leq Q_2$) es $\pm 3 \%$, independiente del alcance de temperatura.

5.7. Medidores con Clase de Exactitud 2.

5.7.1. El EMP para la zona superior del rango de flujo ($Q_2 \leq Q \leq Q_4$) es $\pm 2 \%$ para temperaturas de $0,1^\circ\text{C}$ a 30°C y de $\pm 3 \%$ para temperaturas superiores a 30°C . El EMP para la zona inferior del rango de flujo ($Q_1 \leq Q \leq Q_2$) es de $\pm 5 \%$, independiente del alcance de temperatura.

5.8. Flujo inverso.

5.8.1. El fabricante debe especificar si el medidor ha sido diseñado para medir el flujo en sentido inverso o no. Si un medidor ha sido diseñado para medir en

condiciones de flujo inverso, el volumen que pasa durante dicha condición debe ser restado del volumen indicado o el medidor debe registrarlo separadamente. En el caso de los medidores diseñados para medir el flujo inverso, el caudal permanente y el rango de medición podrán ser diferentes en cada dirección.

5.8.2. Si un medidor no está diseñado para medir en condiciones de flujo inverso, el medidor deberá evitar dicho flujo o soportarlo, ante un caso accidental hasta un caudal de Q_3 , sin deterioro o cambio en sus características metrológicas para el funcionamiento con caudal normal.

5.8.3. El EMP será el mismo tanto para el flujo normal como para el inverso.

Artículo 6. REQUISITOS DE ENVASE, EMPAQUE Y ROTULADO O ETIQUETADO.

6.1. Características.

Todo medidor de agua potable considerado en el presente Reglamento Técnico, debe presentar la información descrita a continuación, misma que tendrán carácter de declaración jurada por parte del fabricante nacional o importador.

6.2. Marcado.

El medidor debe incluir marcas claras e indelebles con la información descrita a continuación, agrupada o distribuida en su carcasa, en el dispositivo indicador, en la placa de identificación, o en la cubierta si no es extraíble.

a) Unidad de medida: metro cúbico (m^3)

b) La clase de exactitud (cuando difiere de la clase de exactitud 2).

c) Valor numérico de Q_3 , la relación Q_3/Q_1 La relación Q_3/Q_1 puede expresarse como R (por ejemplo "R160").

- d)** Marca y modelo del fabricante.
- e)** Año de fabricación y número de serie (lo más cerca posible del dispositivo indicador)
- f)** Sentido de circulación del flujo (mostrar en ambos lados del cuerpo: o en un solo lado siempre que la flecha de sentido de circulación sea fácilmente visible en toda circunstancia).
- g)** Presión máxima admisible (PMA) si ésta excede a 1 MPa (10 bar).
- h)** Letra V o H si el medidor puede ser operado en posición vertical u horizontal.
- i)** Temperatura máxima permisible.
- j)** El fabricante puede indicar la máxima pérdida de presión.

6.3. Información documentada.

6.3.1. Todos los medidores deben estar provistos de manuales de instalación y de mantenimiento destinado al personal técnico que realizará la instalación del mismo o a la usuaria y/o usuario. Dicho manual (a criterio del fabricante), puede o no incluir recomendaciones del uso de reguladores de presión o acoplamientos de purgadores de aire complementarios al medidor.

6.3.2. El fabricante debe indicar el país de origen de fabricación de los medidores.

6.3.3. El fabricante debe proporcionar información, en caso de existir garantías del medidor y software relacionado, cuando éste cuente con dispositivos digitales.

6.3.4. La información debe ser de origen y si su contenido en idioma español o contar con la traducción correspondiente y para la simbología sus equivalentes en el Sistema Internacional de Unidades.

CAPITULO III.

PROCEDIMIENTOS ADMINISTRATIVOS.

Artículo 7. CERTIFICADO DE CUMPLIMIENTO DE REGLAMENTO TÉCNICO

Y

MARCA DE VERIFICACIÓN

7.1. Todo medidor contemplado en el presente Reglamento Técnico, sea fabricado en territorio nacional o importado, debe contar con el Certificado de Cumplimiento de Reglamento Técnico y la marca de verificación, que serán otorgados por IBMETRO como identificación de que los medidores de agua cumplen con el presente Reglamento Técnico y son aptos para su uso en el territorio nacional.

7.2. El Certificado de Cumplimiento de Reglamento Técnico, debe ser obtenido para cada lote de medidores importados o fabricados en territorio nacional en base a la Aprobación de Modelo correspondiente a cada tipo de medidor de agua aprobado, conforme lo establecido en el numeral 7.2.1 y según la Declaración de los números serie correspondientes a cada medidor de agua emitida por el Fabricante Nacional o Importador.

7.2.1. Aprobación de Modelo

Presentar a IBMETRO la Aprobación de Modelo (original o copia legalizada) conforme a una de las siguientes alternativas:

i. Aprobación de Modelo emitida íntegramente por IBMETRO:

- El fabricante nacional o importador, debe obtener de IBMETRO la Aprobación de Modelo, para cada tipo/modelo de medidor de agua según su diseño y características específicas. Esta aprobación debe ser registrada en el Sistema Informático de IBMETRO.
- Para la obtención de la Aprobación de Modelo, el fabricante nacional o

importador deberá proporcionar a IBMETRO la cantidad de muestras necesarias para realizar los ensayos, según lo establecido en los numerales 7.4.2 y 7.5.1 del presente Reglamento Técnico.

- En caso de que la muestra de medidores otorgada, apruebe satisfactoriamente todos los ensayos, IBMETRO otorgará la Aprobación de Modelo correspondiente, que tendrá una validez de cinco (5) años a partir de su registro en el Sistema Informático de IBMETRO, y solamente será válida para el tipo/modelo de medidor verificado. Cualquier modificación al modelo aprobado, conllevará al importador o al fabricante nacional a obtener una nueva Aprobación de Modelo.

ii. Homologación de la Aprobación de Modelo emitida por un Instituto Nacional de Metrología del extranjero:

- El fabricante nacional o importador, debe obtener de un Instituto Nacional de Metrología o por algún organismo que cuente con reconocimiento oficial del Instituto Nacional de Metrología de su país, la Aprobación de Modelo, para cada tipo/modelo de medidor de agua según su diseño y características específicas, que demuestre el cumplimiento de lo establecido en los numerales 7.4.2 y 7.5.1 del presente Reglamento Técnico.

- Para la obtención de la Homologación correspondiente, el fabricante nacional o importador deberá proporcionar a IBMETRO la Aprobación de Modelo (original o copia legalizada), para realizar las verificaciones correspondientes.

- En caso de que la Aprobación de Modelo demuestre entero cumplimiento con lo establecido en el presente Reglamento Técnico, IBMETRO otorgará la

Homologación de la Aprobación de Modelo correspondiente, cuya validez será establecida en función a la validez del documento emitido en origen por un plazo no mayor a cinco (5) años, y solamente será válida para el tipo/modelo de medidor verificado. Cualquier modificación al modelo aprobado, conllevará al importador o al fabricante nacional a obtener una nueva Aprobación de Modelo u Homologación, según sea el caso.

- Sin perjuicio de lo establecido anteriormente, previo a emitir la homologación correspondiente, IBMETRO se reserva el derecho de realizar algunos ensayos de forma excepcional.

7.2.2. Sin perjuicio de lo señalado, IBMETRO se reserva el derecho de realizar las verificaciones correspondientes del lote importado.

7.3. El fabricante nacional o importador, para la obtención de la Marca de Verificación, debe contar con la verificación inicial correspondiente, según lo siguiente:

7.3.1. Verificación inicial.

El importador o la EPSA que está adquiriendo los medidores de agua, antes de su comercialización o instalación, deberán someter el lote de medidores a la verificación metrológica inicial a cargo de IBMETRO, de acuerdo a lo siguiente:

- a)** La Verificación Inicial será realizada por IBMETRO, únicamente para aquellos medidores de agua que cuenten con el Certificado de Cumplimiento de Reglamento Técnico correspondiente.
- b)** IBMETRO realizará un muestreo en el recinto del importador o del fabricante nacional para realizar la verificación inicial.
- c)** IBMETRO realizará la verificación inicial de las muestras tomadas; en base a

dicha verificación, otorgará la marca de verificación a todos los medidores de agua detallados en el lote correspondiente.

d) Si durante la verificación inicial, los medidores de agua no cumplen con los ensayos establecidos en el punto 7.4.3, se realizará los ensayos a todo el lote y se otorgará la marca de verificación solamente a los medidores que hayan pasado las pruebas. Los otros medidores serán inutilizados.

e) Finalizada la verificación inicial se otorgará un certificado de verificación inicial donde se identifique el número de serie de los medidores aprobados.

7.4. Ensayos para Evaluar la Conformidad.

Para la demostración del cumplimiento de los requisitos solicitados en el presente Reglamento Técnico, los medidores deben ser sometidos a los ensayos que a continuación se detallan.

7.4.1. Inspección externa.

Se verificará que el medidor de agua cumple los requisitos de materiales de fabricación (tanto internos como externos), del tamaño del medidor (rosca de los extremos) y dimensiones generales (para cada tamaño de medidor hay un conjunto fijo correspondiente de dimensiones generales), así como con respecto a los requerimientos del dispositivo de indicación, el marcado del medidor y la aplicación del dispositivo de protección.

7.4.2. Ensayos para aprobación de modelo.

a) Para obtener la aprobación de modelo, todos los medidores deben estar sujetos a los ensayos descritos a continuación:

i. Prueba de presión estática

ii. Determinación de los errores intrínsecos de indicación

iii. Prueba de pérdida de presión

iv. Prueba de perturbación de flujo

v. Prueba de flujo inverso

vi. Prueba de durabilidad o desgaste – Flujo discontinuo

vii. Prueba de durabilidad o desgaste – Flujo continuo

b) Los ensayos deben realizarse en cualquier orden, de acuerdo a lo establecido en un procedimiento interno de IBMETRO, en base a las normas OIML R 49 vigentes.

c) Todos los ensayos deben realizarse en prototipo del medidor de agua con todos sus componentes presentados para la aprobación de modelo.

d) Para la Aprobación de Modelo, cuando se realiza un ensayo, la incertidumbre expandida del volumen medido, no debe exceder 1/5 del EMP.

7.4.3. Ensayos para la Verificación inicial.

a) IBMETRO realizará la verificación inicial solamente para aquellos medidores que cuenten con la aprobación de modelo correspondiente.

b) La verificación inicial será realizada llevando a cabo los ensayos determinados en los numerales i y ii del literal a) del numeral 7.4.2 del presente Reglamento Técnico

c) Para la verificación inicial, cuando se realiza un ensayo, la incertidumbre expandida del volumen medido, no debe exceder 1/3 del EMP.

d) IBMETRO colocará precintos de seguridad a cada medidor que haya pasado satisfactoriamente la verificación inicial correspondiente.

7.4.4. Requerimientos comunes a todos los ensayos.

a) **Calidad del agua**

Los ensayos deben realizarse con agua. El agua debe ser potable proveniente del servicio público o en su defecto poseer las mismas características. No debe contener ningún material capaz de dañar al medidor o afectar su operación. El agua no deberá contener burbujas.

b) Reglas generales de las instalaciones donde se realizarán los ensayos.

i. Ensayo de medidores en grupo.

Los medidores pueden ser ensayados en forma individual o en grupos. En el último caso las características individuales deberán determinarse en forma precisa. La interacción entre medidores y entre bancos de ensayo deberá eliminarse. Cuando los medidores se ensayan en serie, la presión a la salida de cada uno de ellos debe ser suficiente como para evitar cavitación.

ii. Ubicación.

Durante el ensayo, el laboratorio de ensayo debe encontrarse libre de influencias de perturbaciones involuntarias (p. ej. variaciones de la temperatura ambiente, vibraciones, otros).

iii. Condiciones de referencia para los ensayos.

Todas las otras magnitudes de influencia aplicables, excepto para las cantidades de influencia que fuesen ensayadas, debe estar dentro de los valores de la Tabla 4, durante ensayos de aprobación de prototipo en un medidor de agua.

Tabla 4. Condiciones de referencia para los ensayos.

Caudal (Caudal de referencia) $0,7 \times (Q2 + Q3) \pm 0,03 \times (Q2 + Q3)$

Temperatura del agua $20\text{ °C} \pm 10\text{ °C}$

Presión en el agua Para: $DN < 50\text{ mm}$

$0,03\text{ MPa} < PW < 1\text{ MPa}$

$(0,3\text{ bar} < PW < 10\text{ bar})$

Rango de temperatura ambiente 15 °C a 25 °C

Rango de humedad relativa

ambiente

$40\text{ \%}$ a $75\text{ \%}$

Rango de presión atmosférica

ambiente

650 mbar a 680 mbar o

65 kPa a 68 kPa

Durante cada prueba, la temperatura y humedad relativa no deben variar más de 5 °C o $10\text{ \%HR}$ respectivamente en el rango de referencia.